Thesis Plan

Decentralized Multi-Agent Reinforcement Learning for Power Grid Topology Control

Corentin Plumet

August 17, 2026

Contents

Chapter 1

Thesis Plan

1.1 Working Title

Graph Neural Networks for Decentralized Multi-Agent Reinforcement Learning in Power-Grid Topology Control

1.2 Central Thesis Question

How can graph neural networks help decentralized reinforcement learning agents learn reliable power-grid topology control policies that preserve survival performance while scaling to combinatorial action spaces?

Three terms fix what the thesis has to measure:

- **Reliable** is episodic survival on held-out chronics, always reported from the checkpoint with the best test survival and re-evaluated on the full test split.
- **Decentralized** means each agent owns a fixed group of substations, observes a local subgraph, and acts on its own discrete topology actions without having any information about the other agent actions; only the critic is centralized.
- **Scaling** means larger grids, which a graph encoder addresses because its parameters are independent of grid size.

1.3 Research Questions

1. **Can decentralized MAPPO first learn a strong, stable baseline?** The flat MLP actor must achieve reliable survival on `bus14` before either its objective or its representation is changed. (Chapter 3.)
2. **Can the agents intervene sparsely without losing survival?** Evaluation-time rho overrides, an intervention-gated actor, fixed non-idle penalties, and adaptive Lagrangian intervention budgets are evaluated on two axes at once, survival and intervention rate, since improving either alone is not a result. (Chapter 4.)
3. **Can a graph actor replace the flat one without paying for it?** An MLP's input width is tied to one grid, whereas a shared graph encoder is size-independent. That advantage matters only if the GNN first matches the tuned MLP on `bus14`. (Chapter 6.)
4. **How should the grid be represented as a graph for a local actor?** Node granularity, relations, substation-level augmentations, and graph readout are all design choices. Chapter 5 defines them and Chapter 6 measures which ones move survival.

5. **What is a local agent entitled to observe, and does enforcing that boundary matter?** A local graph carries contextual nodes from neighboring substations. Making context visible only through a live boundary connection, and choosing whether pooling covers the visible graph or only the controlled domain, determine what the policy conditions on. The distinction is between context and leakage. (Sections 5.2.5 and 5.4.)
6. **Does any of this transfer to a larger grid?** Moving from `bus14` to the 36-substation WCCI grid tests whether a graph encoder is portable in practice and not only in parameter shape, and what has to be corrected: mainly feature scaling and the size of the action space. (Chapter 8.)

1.4 Expected Contributions

- **A tuned decentralized MAPPO baseline on `bus14`.** The original framework trained unstably; the training and initialization changes that fix it are isolated one at a time.
- **An ablation of sparse intervention mechanisms** reported jointly on survival and intervention frequency, connecting the learned behavior to what a control room would accept: act rarely, act locally, and act when the grid is unsafe.
- **A shared GNN actor and a screened local graph design.** The GNN first matches the tuned MLP, then three representations are compared using a fixed candidate graph with a dynamic edge mask and an explicit information boundary. Encoder depth and width, message-passing operator, readout, and structural augmentations are screened with full-test evaluation.
- **A size-invariant mechanism for combinatorial action spaces.** A candidate-action head that scores each action from the encoded nodes it modifies, replacing the fixed-width layer over an action index.
- **A measurement of what actually breaks under transfer.** Encoder parameters cross grids unchanged; the input distribution they were calibrated on does not. Two findings generalize beyond power grids: per-feature standardization is harmful when a channel is a mixture dominated by structural zeros, and for gauge-dependent quantities such as voltage angle, changing the representation beats any choice of normalization.

1.5 Open Decisions

- Confirm the final thesis title and whether to emphasize GNNs, sparse-control, or decentralized MARL most strongly.
- Decide which experiment families are final and which are only exploratory.
- Decide whether the main claim is performance improvement, improved sparsity at equal survival, or reproducible ablation insight.
- Choose the final bibliography style required by the university.

Chapter 2

Introduction

2.1 Topology control

A transmission grid is not a fixed graph. Inside a substation, every connected element, each transmission line, generator, and load, is attached to one of several busbars, and an operator can move an element from one busbar to another or split a substation into two electrically separate nodes. Doing so changes which paths the power actually flows through, and therefore redistributes loading across the network. This is topology control, also called busbar switching or network reconfiguration.

Its appeal is economic. Congestion is classically relieved by redispatching generation or by curtailing renewable production, both of which cost money at every intervention, or by building new lines, which costs a great deal once and takes years. Topology control uses assets that already exist and switching equipment that is already installed, so its marginal cost is close to zero [1]. The energy transition makes this attractive rather than merely elegant: variable renewable generation makes flows less predictable and pushes existing corridors closer to their thermal limits, while grid expansion is slow [2], [3].

The catch is that topology is hard to exploit. Operators use it, but mostly through a small repertoire of remedial actions established by experience and offline study, so a large part of the available flexibility is never searched [3].

2.2 Why it has to be automated

Three properties make manual search impractical.

The action space is combinatorial. A substation with d connected elements and two busbars admits 2^d assignments, and the number of joint configurations of a grid is the product over its substations. Even the small IEEE 14-bus benchmark used throughout this thesis exposes 178 distinct unitary topology actions; the 36-substation grid of Chapter 8 exposes more than 66 000, which is why it has to be reduced offline before an agent can be trained on it at all (Section 5.6).

The effect of an action cannot be read off the grid. Power flows are governed by non-linear AC equations, so the consequence of moving one line onto another busbar is a global redistribution, not a local change. There is no reliable shortcut for ranking actions other than simulating them.

Finally, the decision is sequential and time-constrained. An action taken now changes which contingencies are survivable later, cooldown periods forbid acting on the same substation twice in quick succession, and an operator has minutes, not hours. Optimizing a single step greedily is not the same problem as keeping a grid alive for a day.

These are the conditions under which reinforcement learning is a reasonable candidate: a sequential decision problem, a simulator that is expensive but available, no analytic optimum, and a clear success

signal — the grid does not black out. The Learning to Run a Power Network (L2RPN) competition series was created by RTE to make exactly this problem tractable for machine-learning research, and the Grid2Op simulator provides the environment, the chronics of load and generation, and the operational rules that go with it [2], [4].

2.3 Why decentralized agents

Most published work treats the grid as a single agent controlling everything. Real grids are not operated that way. Transmission system operators divide the network into regions, each with its own control centre and its own operators, who act on their own substations with a detailed view of their own area and a much coarser view of the rest [3]. A decentralized formulation is therefore closer to how the job is actually done, and to how any automated assistant would eventually be deployed, as support inside one control room, not as a single controller over an entire interconnection.

It is also the formulation that scales. A monolithic controller has to represent the joint action space, which grows multiplicatively with the number of substations. Partitioning the grid into regions and giving each region its own policy replaces that product by a sum: each agent chooses among its own local actions only. This is the classical argument for decentralized multi-agent reinforcement learning, and it comes with the classical cost [5]. Because every agent learns while the others are also changing, each one faces an environment that is non-stationary from its own point of view, and because each observes only part of the grid, agents can take individually sensible actions that are jointly harmful.

The usual answer is *centralized training with decentralized execution*: a critic is allowed to see the global state during training, while each actor conditions only on what it will have access to at deployment time [6], [7]. This thesis uses that setting throughout.

2.4 Reinforcement learning for topology control

Work on RL for topology control falls into four groups, in roughly historical order. A broader survey of graph-based RL for power systems is given by Hassouna et al. [8].

Single-agent policies. The first approaches train one agent to control the whole grid. Subramanian et al. [9] showed that a plain deep RL agent acting on a curated set of topology actions already outperforms a do-nothing policy on L2RPN benchmarks. The strongest example is SMAAC [10], which won the L2RPN WCCI 2020 challenge with an actor-critic method over *afterstates* — the grid configuration that results from an action, evaluated before the physics resolves it — combined with a hierarchical decomposition of where to act and how. The common obstacle in this group is the action space: it has to be reduced, curated, or factorized before learning becomes feasible at all.

Learned policies wrapped in heuristics. A second group accepts that a learned policy should not be in charge at every step, and surrounds it with rules. The dominant pattern is an activation threshold: the agent is queried only when the grid is stressed, measured by the maximum line loading $\rho_{\max}$, and does nothing otherwise. PowRL added a heuristic overload manager and a reduced action set on top of an RL policy and topped the L2RPN NeurIPS 2020 robustness track [11]. Lehna et al. [12] compared advanced rule-based agents against RL agents directly and found comparable survival, with the RL agent far cheaper at inference — a useful reminder that beating a well-engineered heuristic is not automatic. Heuristics of this kind are now standard practice rather than a baseline, and this thesis uses one.

Hierarchical and centrally coordinated multi-agent methods. A third group decomposes the decision but keeps a coordinator. Manczak et al. [13] split control into choosing whether to act, choosing where, and choosing which local configuration to apply. van der Sar et al. [14] assign a low-level agent to each substation under a higher-level agent that selects which substation acts. de Mol et al. [15] make the coordination explicit: regional agents propose actions and a central agent selects among the proposals.

These methods recover much of the scalability of factorization, but they retain a component that sees the whole grid and decides for everyone, so they do not answer whether purely local policies suffice.

Decentralized multi-agent control. The remaining question — whether agents that act on local observations, without a coordinator and without communication, can operate a grid — is much less explored, and it is where this thesis sits. The gap is recognized: the MARL2Grid-TR benchmark was built specifically because prior work "focused on single-agent settings, neglecting the often decentralized, multi-agent nature of grid control" [16].

Graph representations. Cutting across all four groups is the question of how the grid is given to the network. A flat vector ties the policy to one grid size and discards the topology entirely. Graph neural networks are the natural alternative [17], [18], [19], and their parameters are independent of the number of substations, which is what makes transfer between grids conceivable at all. The choice of representation is not neutral, however: de Jong et al. [20] show that the standard homogeneous graph suffers from *busbar information asymmetry* — it encodes the connections that currently exist but not the ones an action could create — and that a heterogeneous representation resolves it. That observation is close to the subject of Chapters 5 and 6.

2.5 Starting point: MARL2Grid and MAPPO

This thesis builds on MARL2Grid, the multi-agent benchmark and codebase of Marchesini et al. [16], itself the multi-agent successor of RL2Grid [21]. It wraps Grid2Op in a multi-agent interface and supplies the decomposition, the observation and action spaces, the reward, and the training loop that the rest of this work modifies.

Decomposition. The grid is partitioned into disjoint regions, one per agent, fixed in advance. On the IEEE 14-bus environment (`bus14`, 14 substations and 20 lines) three agents own substations $\{0, 1, 2, 4\}$, $\{3, 6, 7, 8\}$ and $\{5, 9, \ldots, 13\}$, giving them 52, 50 and 76 non-idle actions respectively — the 178 actions of the full grid, partitioned rather than multiplied. Each agent observes only quantities belonging to its own region and chooses among the topology actions available at its own substations; action 0 is always do-nothing. Agents act simultaneously and are given no information about what the others are doing.

Reward and heuristic. The training reward is dominated by a flat $+1$ per surviving step, plus an overload term, so the objective is essentially to keep the episode alive; the reported metric is episodic survival, the fraction of evaluation episodes completed without a blackout. Following the practice described above, an idle heuristic fast-forwards the simulation while the grid is safe: agents are queried only once $\rho_{\max} \geq 0.90$.

MAPPO. The learning algorithm is multi-agent PPO [7], the natural multi-agent extension of PPO [22] under centralized training with decentralized execution. Each agent i has its own stochastic policy $\pi_{\theta_i}(a_i \mid o_i)$ over its discrete local action space, and is updated with the clipped surrogate objective

$$L^{\mathrm{CLIP}}(\theta_i) = \hat{\mathbb{E}}_t\Big[\min\Big(r_t(\theta_i)\,\hat{A}_t,\ \mathrm{clip}\left(r_t(\theta_i),\,1-\epsilon,\,1+\epsilon\right)\hat{A}_t\Big)\Big], \qquad r_t(\theta_i) = \frac{\pi_{\theta_i}(a_{i,t} \mid o_{i,t})}{\pi_{\theta_i^{\mathrm{old}}}(a_{i,t} \mid o_{i,t})},$$

where $\hat{A}_t$ is a generalized advantage estimate. The advantages come from a single *centralized* value function trained on the global state, which is available in simulation but never given to the actors. Only the actors are kept at execution time, so the deployed system is decentralized even though training was not.

What this thesis starts from. The baseline is that system with flat multilayer-perceptron actors and critic, trained on `bus14`. It is unstable at low survival out of the box, and it is tied to one grid: an MLP's input width is a function of the number of substations, so nothing it learns can be reused elsewhere. The chapters that follow address these limitations in the order they arose: first stabilize training, then make interventions more operator-like, and finally replace the fixed-width representation.

2.6 Outline

Chapter 3 tunes the flat baseline until it trains stably. Chapter 4 then changes the objective and asks whether the same agents can keep their survival while intervening rarely; its preliminary WCCI pilot exposes the limitations of retaining a fixed-width MLP actor. Chapter 5 consequently defines size-independent graph inputs: feature scaling, three grid representations, substation-level augmentations, the readout, and the candidate-action head. Chapter 6 first checks that a shared graph encoder remains competitive, then screens those design choices. Chapter 8 moves to the 36-substation WCCI grid and tests what actually transfers.

Chapter 3

Establishing a Baseline

This chapter establishes a robust MLP actor baseline for the IEEE 14-bus task.

3.1 Evaluation Protocol

All experiments use an 80/20 train/test chronic split. The 80% training split is used for policy updates, while the 20% test split estimates generalization. Evaluation rollouts run periodically during training using 10 episodes per split.

The primary performance metric is test episodic survival (expressed as a percentage), representing the fraction of evaluation timestep where the agent survives and successfully prevents a blackout. Results are reported using seed-aggregated mean curves, with shaded regions indicating variability. Final evaluations report the performance of the checkpoint that achieved the highest training survival.

3.2 IEEE 14-Bus Experiments

The initial MAPPO baseline from the MARL2Grid framework lead to unstable policies and low survival rates on the IEEE 14-bus topology-control task. The following experiments test training and architectural modifications to establish a high-performing baseline.

3.2.1 MLP Baseline

Question. Which changes are needed to obtain a stable MLP MAPPO baseline on the 14-bus task?

Baseline and changes. We start from the MLP MAPPO baseline of the MARL2Grid framework and test several main stabilization choices:

- using reward normalization;
- changing the critic update so that the critic is updated once for the whole batch instead of once per agent;
- tuning of optimization hyperparameters to stabilize the training, as summarized in Table 3.1.

Table 3.1: New hyperparameters for MAPPO training.

Hyperparameter	New Value	Paper Baseline
ENVIRONMENT		
n_envs (number parallel environment)	20	10
PPO OPTIMIZATION		
Total time step	15,000,000	25,000,000
Actor learning rate	1e-4	3e-5
Critic learning rate	1e-4	3e-5
Gamma	0.9	0.99
Update epochs	10	80
Minibatches	16	4
Maximum gradient norm	1.0	10.0

Evidence. Table 3.3 reports final and peak survival for the core ablations. Figure 3.1 overlays all core MLP baseline curves.

Interpretation. The `Optimized MLP Baseline`, which combines reward normalization and an optimized critic update, achieves the best performance and convergence speed, yielding the highest and most stable final survival (∼93%). Removing or reducing any of these three elements, using a smaller architecture, disabling reward normalization, or reverting to a non-optimized critic update, results in a drop in either learning speed or final survival. The original unoptimized baseline fails entirely to learn a competitive policy. Because the `Optimized MLP Baseline` consistently outperforms the ablations, it is established as the standard reference for all subsequent comparisons.

Table 3.2: MLP baseline run configurations. The reference run is `Optimized MLP Baseline`.

Run family	Actor/critic hidden layers	Reward norm.	Initial action-0 prob.	Optimized critic update	Difference from `Optimized MLP Baseline`
`Optimized MLP Baseline`	3×256 / 3×256	true	0.0	true	Reference baseline
`Disable optimized critic update`	3×256 / 3×256	true	0.0	false	Disables optimized critic updates
`Smaller MLP actor/critic`	2×256 / 2×256	true	0.0	true	Uses a smaller MLP actor and critic
`Disable reward normalization`	3×256 / 3×256	false	0.0	true	Disables reward normalization
`Old baseline (no val)`	3×256 / 3×256	true	0.3	false	Uses action-0 bias 0.3 and non-optimized critic updates
`Initial Bias 50%`	3×256 / 3×256	true	0.5	true	Uses action-0 bias 0.5
`Initial Bias 70%`	3×256 / 3×256	true	0.7	true	Uses action-0 bias 0.7

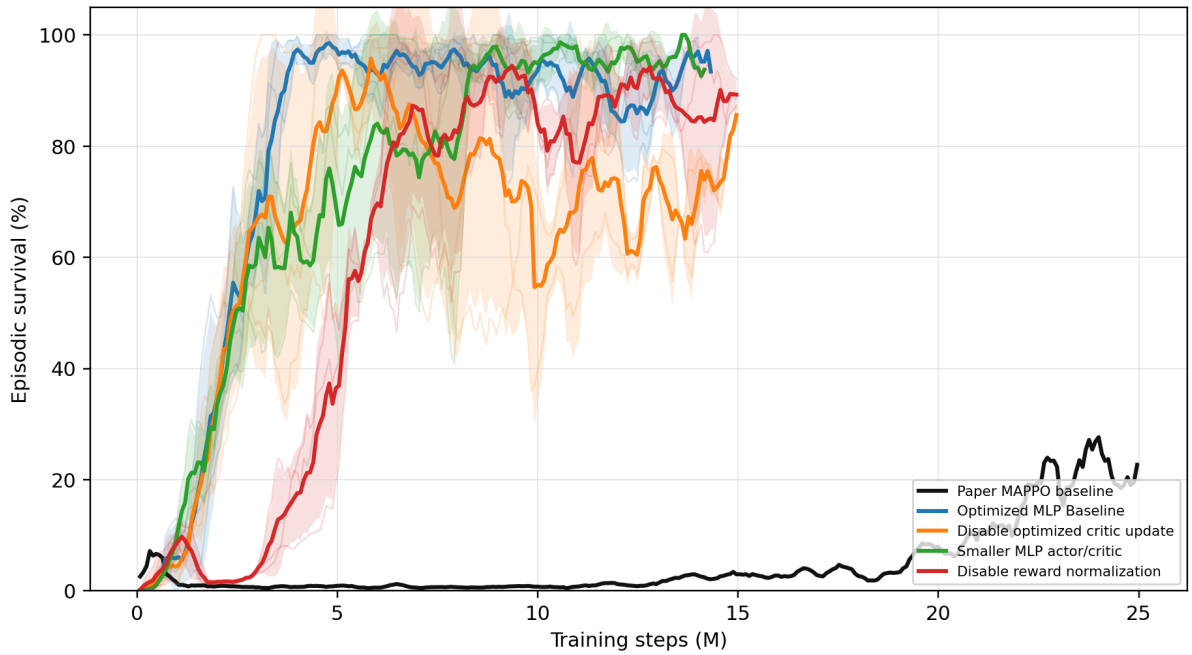

Figure 3.1: All MLP baseline core rerun survival curves shown on one axis.

Table 3.3: MLP baseline core rerun survival summary.

Run	Intended change	Final survival (%)	Peak survival (%)
`Optimized MLP Baseline`	Optimized MLP baseline	93.4	98.6
`Disable optimized critic update`	Non-optimized critic update	93.1	95.8
`Smaller MLP actor/critic`	Smaller MLP architecture	93.7	100.0
`Disable reward normalization`	No reward normalization	81.8	97.1

3.2.2 Initial Do-Nothing Bias

Question. Does biasing the initial policy toward action 0, the do-nothing action, help or hurt learning? In theory, a do-nothing bias makes sense because in reality, human operators most often do not operate the grid (i.e., they take no action) under normal conditions.

Baseline and changes. Our baseline uses no initial do-nothing bias in this rerun set. The bias controls keep the same general protocol but change the initial probability mass assigned to action 0.

Implementation detail. The bias is applied only at initialization: it changes the starting action distribution before learning, without changing the reward, architecture, or PPO objective. This is implemented by artificially increasing the initial pre-softmax logits for the do-nothing action, such that the policy initially selects this safe action with a higher probability before any learning updates occur. This makes the ablation a test of exploration bias rather than a test of representational capacity.

Evidence. Table 3.4 and Figure 3.2 compare the zero-bias baseline with `Initial Bias 50%` and `Initial Bias 70%`.

Interpretation. The effect of the action-0 bias is large and non-monotonic. The 0.5-bias curve remains unstable and substantially below the zero-bias baseline, whereas the 0.7-bias curve eventually reaches the highest final survival in this comparison. The current conclusion is that adding an initialization bias is generally hurtful or highly unstable for the models. Instead of aiding exploration, a strong initialization prior appears to restrict the agent's ability to reliably discover diverse beneficial actions and should be treated cautiously as a first-order ablation variable.

Table 3.4: Initial do-nothing bias survival summary.

Run	Final survival (%)	Peak survival (%)
`Optimized MLP Baseline`	93.4	98.6
`Initial Bias 50%`	50.8	70.2
`Initial Bias 70%`	99.3	100.0

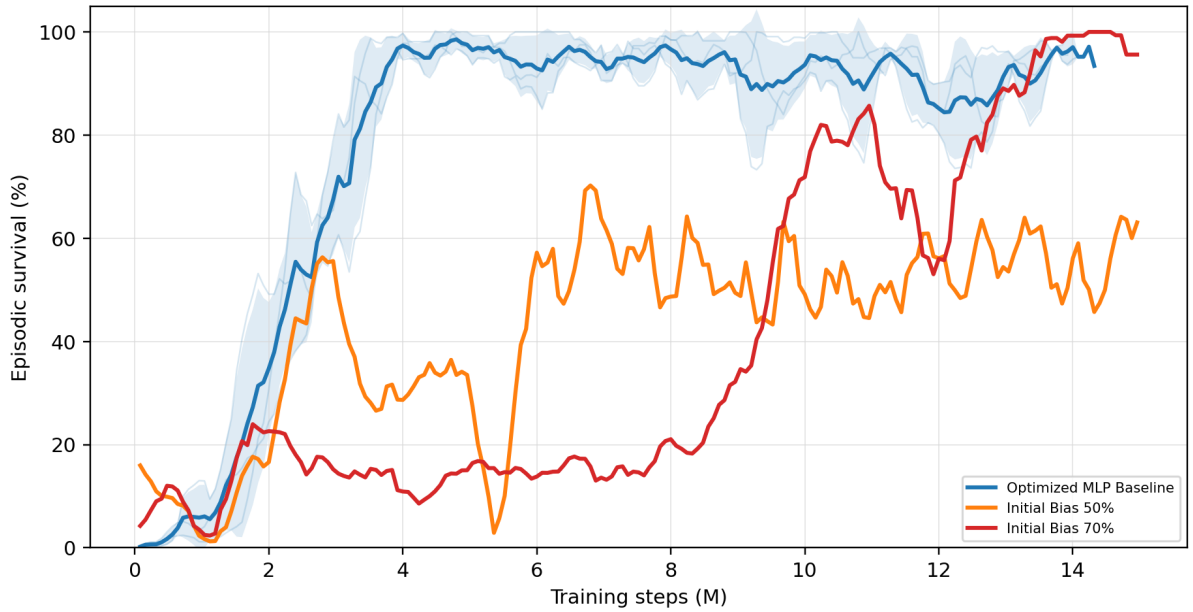

Figure 3.2: Optimized MLP baseline compared with initial do-nothing bias controls `Initial Bias 50%` and `Initial Bias 70%`.

Chapter Summary

Objective: Establish a stable, high-performing flat MAPPO baseline.

Findings:

- **MLP optimization:** Hyperparameter tuning, reward normalization, and optimized critic updates raise the flat MLP baseline to roughly 93% survival.
- **Do-nothing initialization:** The effect is large but non-monotonic; it is not a reliable substitute for learning when to intervene.

Conclusion: The tuned flat actor is a strong survival baseline. The next step is to ask whether it can behave more like an operator by intervening less often without losing that survival, which is the subject of Chapter 4.

Chapter 4

Sparse Intervention Control

The preceding chapter optimized survival on held-out chronics. This chapter introduces a secondary objective: sparsity. A policy that keeps the grid alive by reconfiguring substations at every opportunity is impractical for a control room: interventions carry operational costs, and an agent acting constantly is neither auditable nor trustworthy. The question is whether decentralized agents can maintain high survival while intervening rarely, locally, and only when necessary.

In the literature, sparse activity is most often enforced by hard-coding a physical threshold to dictate when the agent is permitted to act. Instead of relying on an external override, this chapter attempts to make sparse intervention behavior natively learnable.

Mechanisms are therefore evaluated on two competing axes: survival rate and intervention sparsity. An effective mechanism must improve sparsity without degrading survival.

Question. Can the agents keep high survival while behaving more like operators: acting rarely andmainly when the grid state makes an intervention useful?

Baseline and change. This block starts from the flat decentralized MAPPO topology-control actor. For agent i, the baseline policy is one categorical distribution over the full local action space:

$$\pi_i(a_i \mid o_i), \qquad a_i \in \{0, 1, \ldots, |\mathcal{A}_i| - 1\}.$$

Action 0 is the no-op action. During training, actions are sampled from the policy, while deterministic evaluation uses the greedy action by default:

$$a_{i,t}^{eval} = \arg\max_a \pi_i(a \mid o_{i,t}).$$

The sparse-control roadmap tests five ways to reduce unnecessary topology changes: evaluation-time rho overrides, an intervention-gated actor, fixed non-idle action penalties, adaptive intervention budgets, and behavioral cloning from heuristics.

The intervention gate separates whether to act from which action to execute, following hierarchical topology-control decompositions [13], [14]. The fixed and adaptive penalties instead treat non-idle interventions as a limited resource, following constrained and budgeted reinforcement learning [23], [24], [25].

4.1 Evaluation-Time Rho Heuristics

Implementation detail. The rho heuristic is an evaluation-only diagnostic. It does not change the training objective and it does not prove that the policy learned sparse behavior. The policy first proposes the deterministic action $\bar{a}_{i,t}$, then the evaluator may replace it with action 0.

The global version uses the pre-action maximum line loading

$$\rho_t^{global} = \max_{\ell \in \mathcal{L}} \rho_{\ell,t},$$

and executes

$$a_{i,t}^{exec} = \begin{cases} 0, & \rho_t^{global} < \rho_{safe}, \\ \bar{a}_{i,t}, & \text{otherwise}, \end{cases} \qquad \rho_{safe} = 0.90.$$

The local version is decentralized. Agent i computes the maximum rho on the lines touching its regional substations,

$$\rho_{i,t}^{local} = \max_{\ell \in \mathcal{L}_i} \rho_{\ell,t},$$

and only that agent is forced to no-op when $\rho_{i,t}^{local} < \rho_{safe}$. A boundary line can appear in the local neighborhood of both adjacent agents, but no communication is required: both agents can observe the line through their own physical neighborhood.

Interpretation. The heuristic is useful to ask what happens if safe-state interventions are blocked, but it is not a learned method. Its action-0 rates must therefore be reported as final executed evaluation actions, not as evidence that the policy itself prefers action 0.

4.2 Intervention-Gated Actor

Implementation detail. The gated actor changes the actor factorization. Instead of one categorical head over all actions, each agent has a gate head and a non-idle action head:

$$\pi_i^{gate}(g_i \mid o_i), \qquad g_i \in \{0, 1\},$$

where 0 means do nothing and 1 means intervene, and

$$\pi_i^{nonidle}(b_i \mid o_i), \qquad b_i \in \{1, \ldots, |\mathcal{A}_i| - 1\}.$$

During training, the gate is sampled first. If $g_{i,t} = 0$, then $a_{i,t} = 0$. If $g_{i,t} = 1$, the final action is sampled from the non-idle head. The PPO log-probability of the executed action is therefore

$$\log P(a_i = 0) = \log P(g_i = 0),$$

and, for $a_i > 0$,

$$\log P(a_i) = \log P(g_i = 1) + \log P(b_i = a_i \mid g_i = 1).$$

Two deterministic decoders are used. The `final_action_map` decoder selects the most likely final executed action:

$$P(a_i = 0) = P(g_i = 0), \qquad P(a_i = a > 0) = P(g_i = 1) P(b_i = a \mid g_i = 1).$$

The `hierarchical_greedy` decoder first chooses the most likely gate decision, then chooses the most likely non-idle action only if the gate selects intervention.

The original coupled entropy term is

$$H_i = H(g_i) + P(g_i = 1) H(b_i).$$

This can accidentally reward intervention because larger $P(g_i = 1)$ unlocks more non-idle entropy. The separated entropy variant is

$$H_i = \alpha_{gate} H(g_i) + \alpha_{nonidle} H(b_i),$$

and is used in the gate-plus-budget diagnostic runs.

Interpretation. The gate is learned and decentralized, but it can restrict exploration more strongly than a reward penalty. If it collapses early to no-op, the non-idle head is rarely executed and receives little learning signal. For this reason, the gate is treated as an important diagnostic rather than the main sparse control contribution.

4.3 Fixed Sparse-Action Penalty

Implementation detail. The fixed-penalty sweep adds a reward penalty to the final executed action of each agent:

$$r'_{i,t} = r_{i,t} - \lambda_{fixed}\mathbf{1}[a_{i,t} \neq 0],$$

with

$$\lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}.$$

The grid is

$$\text{actor} \in \{\text{flat}, \text{gated}\}, \qquad \lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}, \qquad \text{seed} \in \{0, 1, 2\},$$

which gives $2 \times 4 \times 3 = 24$ runs. The state-dependent safety penalty is disabled in every condition (`safe_intervention_penalty = 0.0`), so the penalty is not rho-weighted.

Interpretation. This mechanism changes only the training reward. It does not override actions during evaluation and it does not hard-mask exploration during training. The policy can still sample non-idle actions, but each such action must be useful enough to pay its fixed cost. Comparing the flat and gated actors at $\lambda_{fixed} = 0.010$ tests whether the gate adds value once a simple sparse objective already exists.

4.4 Adaptive Intervention Budget

Implementation detail. The adaptive intervention budget turns sparse topology control into a local constraint. For each agent, the cost is $c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0]w_i(s_t)$, and the desired constraint is $\mathrm{E}_{\pi_i}\left[c_i(s_t, a_{i,t})\right] \leq d$.

The implemented rollout reward removes the constant term that does not affect the PPO action gradient:

$$r'_{i,t} = r_{i,t} - \lambda_i c_i(s_t, a_{i,t}).$$

After each rollout, the local multiplier is updated by

$$\lambda_i \leftarrow \text{clip}\left(\lambda_i + \eta(\hat{C}_i - d), 0, \lambda_{\max}\right), \qquad \hat{C}_i = \frac{1}{T}\sum_{t=1}^{T} c_i(s_t, a_{i,t}).$$

If the observed cost $\hat{C}_i$ is above the budget d, future interventions become more expensive; if it is below the budget, the penalty relaxes. The default settings are `intervention_budget_lr = 0.02`, `intervention_budget_init_lambda = 0.0`, and `intervention_budget_max_lambda = 10.0`.

Two cost modes are used:

- **Non-idle cost:** $w_i(s_t) = 1$, so $c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0]$. This is an adaptive version of a plain sparse-action penalty.
- **Local-safe cost:** $w_i^{local}(s_t) = \sigma(k(\rho_{safe} - \rho_{i,t}^{local}))$. This is the decentralized version: safe local states make interventions expensive, while stressed local states make them cheap.

In the AIB setting, $\rho_{safe} = 0.90$ is not a hard action override. It is the center of a smooth sigmoid safety weight.

AIB experiment grid. The main local-budget ablation contains five conditions:

- flat actor, local-safe cost, $d = 0.20$;
- flat actor, stricter local-safe budget, $d = 0.10$;
- flat actor, looser local-safe budget, $d = 0.35$;
- gated actor, hierarchical-greedy evaluation, separated entropy, local-safe cost, $d = 0.20$;
- flat actor with adaptive plain non-idle cost, $d = 0.20$.

Comparing the local-safe and plain non-idle costs at $d = 0.20$ isolates the value of local rho weighting.

4.5 Behavioral Cloning from Heuristics

Implementation detail. The evaluation-time rho heuristics (Section 4.1) produce highly sparse, safe behavior, but they require a manual override. To internalize this behavior into the policy weights, a teacher-student distillation approach is tested. The teacher is a MAPPO model running the local rho heuristic, and the student is trained via Behavioral Cloning (BC) to match the teacher's executed actions. The student is then evaluated without the heuristic override to see if it learned to voluntarily imitate the sparsity. Two balancing configurations are evaluated to manage the severe class imbalance caused by the teacher's high action-0 rate.

Interpretation. As reported in Appendix A.1, the student policies reach roughly 91–92% survival and 94–96% sparsity. While this shows that sparse behavior can be partially distilled into the actor weights, the resulting student is weaker than the AIB methods and sacrifices a portion of the teacher's survival (97.1%). Consequently, the distillation route is less reliable than natively learning the sparse objective through the Adaptive Intervention Budget.

4.6 Results

Figure 4.1 summarizes survival and the mean action-0 fraction across agents. Exact values and per-agent action-0 fractions are reported in Appendix A.1.

Table 4.1: Sparse-control mechanisms and where they affect the policy.

Method	Training effect	Evaluation effect	Main risk
Baseline	Samples from the flat policy	Greedy argmax by default	Standard entropy-controlled exploration
Rho heuristic	No training change	Hard no-op override in safe states	Evaluation is manually changed
Intervention gate	Changes action factorization and sampling	Final-action MAP or hierarchical greedy	Gate collapse can starve non-idle exploration
Fixed penalty	Fixed reward penalty on non-idle actions	No action override	Penalty coefficient must be tuned
AIB non-idle	Adaptive reward penalty on non-idle actions	No action override	Multiplier can grow if target is too strict
AIB local-safe	Adaptive state-dependent reward penalty	No action override	Depends on the local rho safety-weight design
Behavioral cloning	Distills heuristic into actor weights	No action override	Struggles with extreme action-0 class imbalance

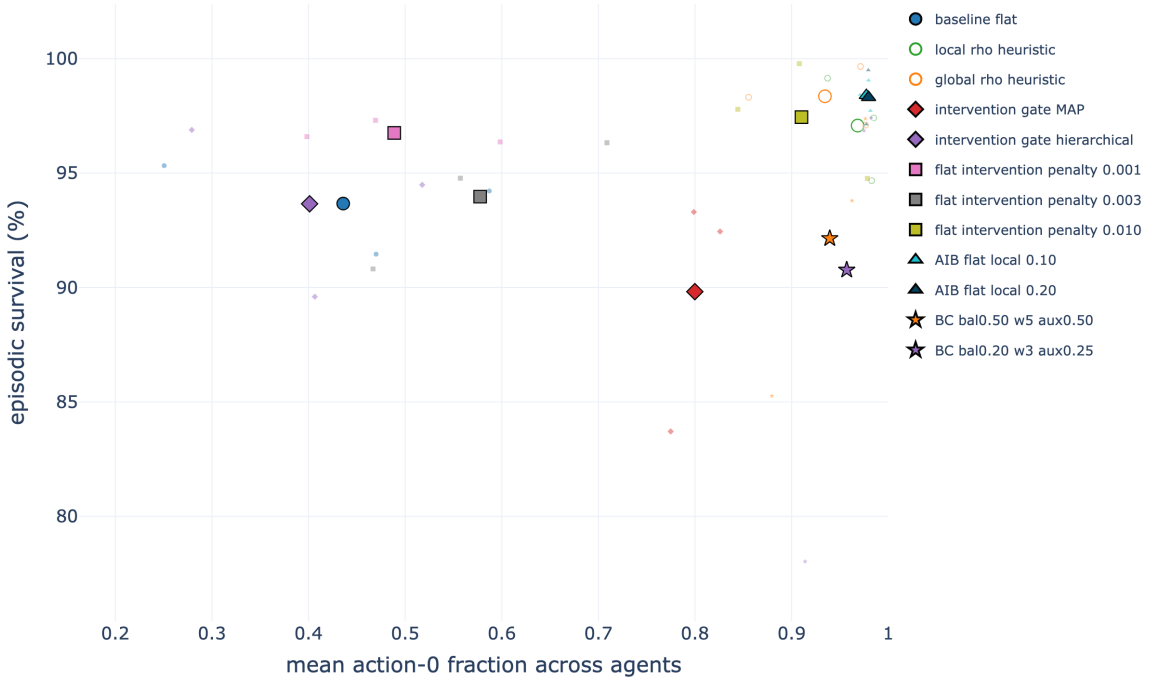

Figure 4.1: Joint action-0/survival comparison of the selected sparse-control mechanisms on `bus14`. Large markers denote condition means and faint markers the individual seeds. The plot uses checkpoint-aligned action-0 rates and full-test survival, so it visualizes the chapter's two objectives.

Interpretation. The current roadmap interpretation is that the gate is not the strongest contribution by itself. Sparse reward shaping already improves action-0 usage, the adaptive budget makes the sparse penalty self-tuning, and the local-safe cost makes the penalty more physically meaningful. In particular, the flat actor with $\lambda_{fixed} = 0.010$ is a strong fixed-penalty baseline, while the flat local-safe AIB policy with $d = 0.20$ is the most principled learned sparse-control candidate: it keeps the original decentralized actor, does not rely on an evaluation-time override, learns the intervention penalty adaptively, and uses a local state-dependent safety weight. Figure 4.1 also makes the main failure mode visible: high action-0 usage is not sufficient when the gate collapses and survival falls.

4.7 Preliminary WCCI Stress Test

As a preliminary scale test, representative mechanisms were retrained from scratch with an MLP actor on the maintenance-enabled WCCI grid; no `bus14` weights were transferred. All runs use the same reduced action space, three seeds, 60M training steps, and learning-rate schedule. The reported values are the final deterministic test evaluation, averaged over seeds and ten held-out episodes per seed; they are not full-test estimates.

Table 4.2: Preliminary WCCI stress test with scheduled maintenance at the final training-time test evaluation.

Method	Survival (%)	Action-0 fraction
Matched MLP baseline	23.75 ± 7.70	0.168
Global rho heuristic, $\rho_{safe} = 0.90$	28.35 ± 11.06	0.974
Local rho heuristic, $\rho_{safe} = 0.90$	9.46 ± 3.54	0.995
Intervention gate, final-action MAP	25.92 ± 10.69	0.400
Flat fixed penalty, $\lambda_{fixed} = 0.010$	27.00 ± 12.13	0.489
Flat local-safe AIB, $d = 0.20$	21.65 ± 2.21	0.927

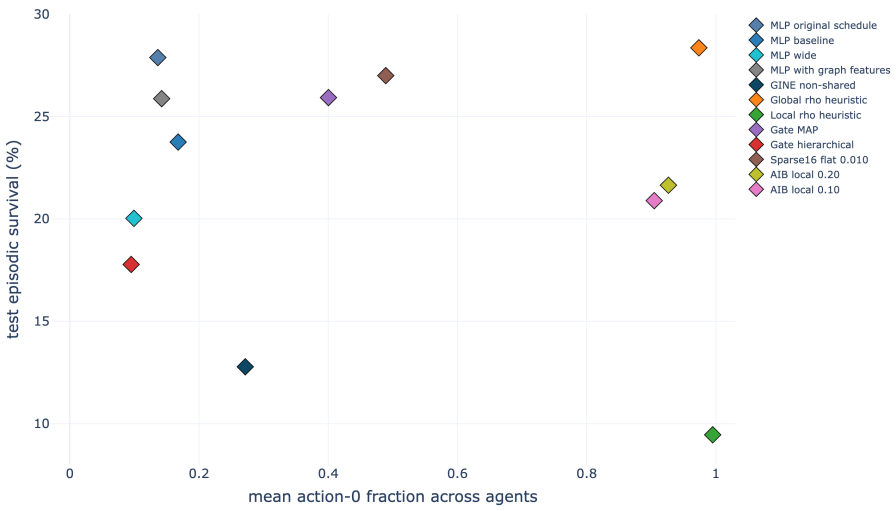

Figure 4.2: Action-0/survival trade-off for the preliminary maintenance-enabled WCCI runs. Each point is a seed-aggregated run variant. The upper-right region remains preferred, but no method combines high sparsity with strong survival on this larger grid.

The mechanisms change intervention frequency, but none reliably improves survival over the weak MLP baseline: the differences are small relative to the seed variation. The local rho heuristic and AIB are highly sparse but remain weak controllers, suggesting that sparsity transfers more readily than control quality. This failed pilot changes grid size, action space, and maintenance at once, so it cannot identify the cause. It nevertheless exposes the limitation of continuing with a fixed-width MLP actor on the larger grid. The next two chapters therefore introduce a size-independent graph representation and test its design choices. Chapter 8 later returns to WCCI and isolates the confounded factors before studying cross-grid transfer.

Chapter Summary

Objective: Preserve survival while reducing unnecessary topology interventions.

Findings:

- **Evaluation heuristics** reach 97–98% survival with action 0 in 94–97% of decisions, but sparsity is imposed after the policy acts.
- **A fixed penalty** already gives a strong learned controller: $\lambda_{fixed} = 0.010$ reaches 97.44% survival and a 0.905 action-0 fraction without an evaluation override.
- **Local-safe AIB** gives the best learned trade-off: budgets $d = 0.10$ and $d = 0.20$ reach about 98.4% survival and 0.98 action-0 usage. The gated variants are less reliable and can collapse to almost permanent no-op behavior.

Conclusion: Sparse behavior can be learned on `bus14`, but the same MLP mechanisms remain weak on WCCI. This motivates the graph representation introduced next.

Chapter 5

Methods

Chapter 4 made the flat policy more operator-like on `bus14`, but its preliminary WCCI stress test remained weak. Sparse intervention control changes when the policy acts; it does not remove the fixed-width MLP representation that ties the actor to one grid. This chapter therefore introduces the size-independent graph inputs and graph-actor variants used in the remaining experiments. The presentation separates four design choices: feature scaling, the base graph representation, substation-level augmentations, and graph pooling.

5.1 Physical Scaling and Voltage-Angle Representation

Two input choices are used to make observations more comparable across grids: fixed physical scaling and a reference-independent representation of voltage angles. Both operate before the graph encoder and are independent of the graph schema.

5.1.1 Physical scaling

Each scaled continuous feature x_f is divided by a fixed reference with the same unit,

$$x'_f = \frac{x_f}{s_f}.$$

This changes the unit of the feature without changing the information it contains and, importantly for sparse graph features, keeps zero at zero. The references are listed in Table 5.1. In all reported experiments that use scaling, the power reference is the rating of the largest installed generator in the corresponding grid, $s_P = \max_g P_g^{\max}$.

Table 5.1: Deterministic physical scales used for continuous graph features.

Features	Scale s_f	Source
`gen_p`, `load_p`, `p`	Power base in MW	Largest installed generator rating.
`gen_theta`, `load_theta`, `theta`, `theta_diff`	180 degrees	Fixed angular scale.
`time_before_cooldown_sub`	Maximum substation cooldown	Grid2Op environment parameter.
`time_before_cooldown_line`	Maximum line cooldown	Grid2Op environment parameter.
`timestep_overflow`	Allowed overflow duration	Grid2Op environment parameter.
Maintenance time and duration	Episode horizon	Grid2Op chronic horizon.

The dimensionless loading ratio `rho` and all binary, mask, type, and relation indicators are left unchanged.

5.1.2 Angle representation

An absolute voltage angle requires an arbitrary zero, normally fixed by the slack bus. Adding the same constant to every bus angle therefore describes the same physical state. Absolute angles depend on this choice of reference, whereas the angle difference across a line does not. For transfer, the graph instead uses

$$\Delta\theta_\ell \;=\; \theta_\ell^{\mathrm{or}} - \theta_\ell^{\mathrm{ex}}$$

This quantity is *gauge-invariant*, meaning that it is unchanged when the angle reference is shifted. It also provides one angle value per line instead of only at nodes with an attached generator or load. The reference-dependent absolute-angle channels are removed when the angle drop is enabled.

Chapter 8 shows why this special treatment is necessary in practice: the source and target grids exhibit a 5–7° offset in absolute angles due largely to their different reference choices. Fixed scaling cannot remove such an offset, whereas the line-angle difference is independent of the reference.

Table 5.2: Where the voltage angle enters each representation, under the two ways of expressing it.

Representation	Absolute angle	Angle drop
Busbar	On busbar nodes, averaged over the attached assets	On the physical line edge
Disaggregated	On generator and load nodes	On the physical line edge
Disaggregated with line nodes	On generator and load nodes	On the *line node*

In the busbar and disaggregated schemas, physical lines are edges, so the angle drop is stored on those edges. In the explicit-line schema, each line is a node and its attachment edges contain no physical measurements; the drop is therefore stored on the line node. This also gives each local actor the angle drop for every line it observes, including boundary lines. A disconnected line is assigned a zero drop because the simulator may retain stale endpoint angles after an outage. These placements are the only schema-specific part of the transformation; the encoder and graph readout are unchanged.

5.1.3 Use across the experiments

The screening experiments of Chapter 6 and the sparse-control experiments of Chapter 4 use the original raw features and absolute angles. The cross-grid transfer study of Chapter 8 uses physical scaling and angle drops on both grids. The code also supports running standardization, but no reported policy run uses it. Section 7.6.1 analyzes it offline as a rejected alternative and finds that it increases the mismatch between grids.

5.2 Graph Representations

The graph observation can use one of three base representations. The `bus` representation aggregates equipment values onto busbar nodes. The `heterogeneous` representation disaggregates generators and loads into explicit asset nodes while keeping physical lines as busbar-to-busbar edges. The `heterogeneous_line` representation further disaggregates transmission lines into explicit line nodes.

Table 5.3 states what separates them. The node schema also determines how a boundary transmission line is represented in a local actor graph. Consequently, the three representations are not information-equivalent: they expose different amounts of context about the substation at the far end of a shared line. This distinction is made explicit in Section 5.2.5.

Table 5.3: The three graph schemas. Each row is a consequence of how much equipment receives its own node.

Aspect	`bus`	`heterogeneous`	`heterogeneous_line`
Node types	Busbar	Busbar, generator, load	Busbar, generator, load, transmission line
Generator and load state	Summed and averaged onto the busbar	One node per asset	One node per asset
Physical line	Direct busbar-to-busbar edge	Typed direct busbar-to-busbar edge	Typed node between its endpoint busbars
Line state	Continuous edge attributes	Continuous edge attributes	Features of the line node
Minimum useful depth	One layer for adjacent busbar exchange	One layer for adjacent busbars; two for asset-to-remote-busbar flow	Two layers for adjacent busbar exchange through a line node
Hidden line state	None: the edge modifies a message	None: the edge modifies a message	Yes: each line obtains and updates an embedding
Far-end context in a local actor graph	Live neighboring busbar, including aggregated power and angle	Live neighboring busbar, but no neighboring generator or load nodes	None; the shared line node replaces the need for a far-end busbar
Node count	SB	$SB + N_g + N_d$	$SB + N_g + N_d + L$

5.2.1 Candidate graph and dynamic mask

All three schemas share one construction.

Grid2Op exposes the instantaneous electrical state and topology through a structured observation [4]. For each representation, the graph construction keeps the set of candidate nodes and edges fixed throughout an episode while using a dynamic edge mask to express the topology that is electrically active at time step t.

For agent a, the graph observation can be written as

$$\mathcal{G}_t^{(a)} = \left(\mathcal{V}^{(a)}, \mathcal{E}_{\text{cand}}^{(a)}, \mathbf{X}_t^{(a)}, \mathbf{Z}_t^{(a)}, \mathbf{m}_t^{(a)}, \mathbf{n}_t^{(a)}, \mathbf{c}^{(a)}\right), \tag{5.1}$$

where $\mathcal{V}^{(a)}$ and $\mathcal{E}_{\text{cand}}^{(a)}$ are static, $\mathbf{X}_t^{(a)}$ contains node features, $\mathbf{Z}_t^{(a)}$ contains edge features, $\mathbf{m}_t^{(a)}$ indicates which candidate edges are active, $\mathbf{n}_t^{(a)}$ marks currently visible nodes, and $\mathbf{c}^{(a)}$ marks nodes belonging to the agent's controlled domain. The controlled mask is static, whereas the edge and visibility masks change with line status and busbar assignments. This separates the fixed candidate layout from the electrical topology that can change after every action.

The mechanism is easiest to see on a transmission line in the `bus` and `heterogeneous` schemas, where a physical line is an edge. Consider a line ℓ whose origin and extremity belong to substations $o(\ell)$ and $e(\ell)$. For every pair of possible endpoint busbars (b_o, b_e), the graph reserves the two directed candidate edges

$$i(o(\ell), b_o) \to i(e(\ell), b_e) \quad \text{and} \quad i(e(\ell), b_e) \to i(o(\ell), b_o), \tag{5.2}$$

so each physical line contributes $2B^2$ candidate directed edges and the two directions share the same line attributes. For the usual case $B = 2$, one line reserves eight candidate edges even though at most two, one per direction, are electrically active at any time.

Which of them is active is decided by the mask

$$m_{\ell,b_o,b_e,t} = \mathbf{1}[\text{line } \ell \text{ is connected at } t]\, \mathbf{1}\left[b_{\ell,t}^{\text{or}} = b_o\right]\, \mathbf{1}\left[b_{\ell,t}^{\text{ex}} = b_e\right]. \tag{5.3}$$

A disconnected line has all of its candidates masked; a connected one keeps only the pair matching its current origin and extremity busbars, provided both endpoint nodes are visible. A topology action therefore changes connectivity by changing $\mathbf{m}_t^{(a)}$, never by rebuilding the graph, and the node and edge tensors keep a fixed shape for the whole episode. The same candidate and mask pattern governs the asset attachments of Section 5.2.3 and the line-node attachments of Section 5.2.4; Appendix C gives the resulting sizes and the masking conventions.

Why a fixed shape rather than rebuilding the graph. A fixed candidate graph keeps every observation at the same tensor shape, so parallel rollouts can be stacked and stored without rebuilding

or re-indexing the graph. Stable rows also allow each action to be mapped to the physical assets it modifies once, before training. Most importantly, masked candidates retain connections that an action could create; rebuilding only the active topology would omit empty destination busbars and hide feasible alternatives, which is the busbar information asymmetry identified by de Jong et al. [20]. The trade-off is a larger edge tensor: with $B = 2$, each line reserves eight directed candidates although at most two are active.

The following subsections define the three schemas.

5.2.2 Busbar Representation

The `bus` schema is the compact busbar representation. Busbars are the only nodes, active transmission lines form the graph edges, and generator and load quantities are aggregated into their currently assigned busbars.

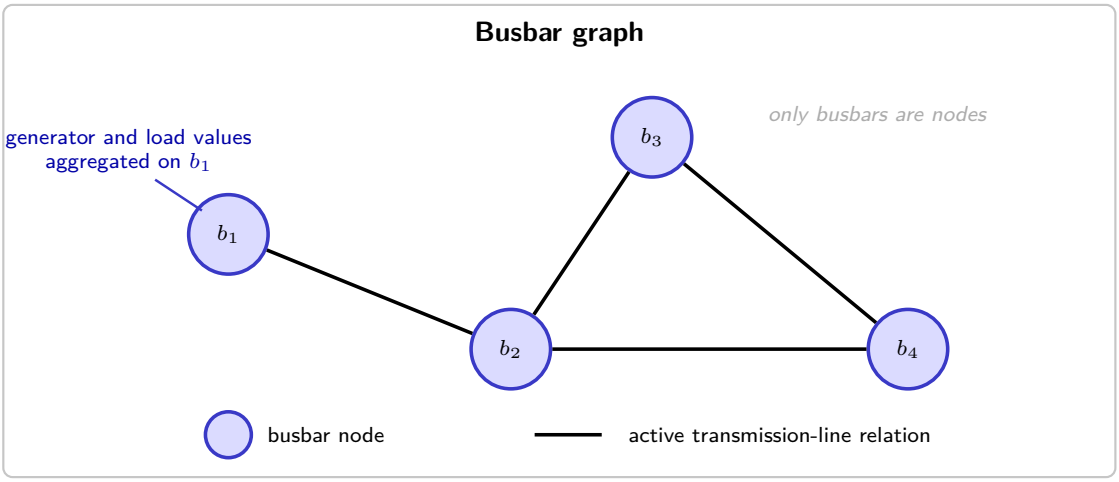

Figure 5.1: Busbar graph representation. Blue nodes denote busbars and black connections denote active transmission-line relations. Generator and load quantities are aggregated into the features of their associated busbars.

Nodes and features

Let S denote the number of substations and let B denote the maximum number of busbars per substation. A node is created for every substation–busbar pair, including busbars to which no asset is connected at the current time. A global node identifier is assigned as

$$i(s, b) = sB + b, \qquad s \in \{0, \dots, S-1\}, \quad b \in \{0, \dots, B-1\}. \tag{5.4}$$

The complete graph therefore contains SB nodes.

Table 5.4 lists the six node features. Generator and load quantities are assigned to the busbar specified by the current topology vector. Active powers are summed when several assets occupy the same busbar, whereas the corresponding angles are averaged. Disconnected assets do not contribute to any node. The substation cooldown is repeated on every busbar node of that substation.

Table 5.4: Features attached to each busbar node.

Feature	Definition
`gen_p`	Sum of active generator power connected to the busbar.
`gen_theta`	Mean voltage angle of generators connected to the busbar.
`load_p`	Sum of active load power connected to the busbar.
`load_theta`	Mean voltage angle of loads connected to the busbar.
`time_before_cooldown_sub`	Remaining substation cooldown.
`domain_mask`	One for a substation controlled by the agent and zero for a contextual neighbor.

This aggregation produces a compact representation but deliberately removes the identities and counts of individual generators and loads.

Relations and edge features

The only physical relations (edges) of the busbar schema are the busbar-to-busbar transmission line. The dynamic attributes copied onto active physical-line edges are given in Table 5.5. The two maintenance features are present only for environments with scheduled maintenance.

Table 5.5: Features attached to physical-line edges.

Feature	Definition
`line_status`	Binary connection status of the physical line.
`rho`	Line loading relative to its thermal limit.
`timestep_overflow`	Number of consecutive time steps in overload.
`time_before_cooldown_line`	Remaining cooldown before another line action is legal.
`time_next_maintenance`	Time until scheduled maintenance, when available.
`duration_next_maintenance`	Duration of scheduled maintenance, when available.

The edge embedding of a busbar-to-busbar transmission edge is exactly the row of Table 5.5. Optional same-substation edges use the same feature width: all physical channels are zero throughout. The homogeneous schema does not append relation one-hots, so these optional relations are distinguished only by their endpoints and neutral physical attributes.

Local information boundary

For an agent-local `bus` graph, all busbars of controlled substations are present. A live boundary line additionally requires exactly one contextual node: the far-end busbar to which that line is currently attached. This node is needed because the shared line is an edge, and both endpoint nodes must exist for its features to reach the controlled busbar through message passing. Because generator and load quantities are aggregated onto busbars, the visible far-end busbar also exposes aggregated neighboring `gen_p`, `load_p`, and angle features. The other busbar candidates at that neighboring substation are masked, even if they carry generation or load. If the boundary line is disconnected, no neighboring busbar remains visible.

5.2.3 Disaggregated Representation

The `heterogeneous` schema keeps busbars as the backbone and adds one node per generator and load. Transmission lines remain busbar-to-busbar edges. The node set is

$$\mathcal{V} = \mathcal{V}_{\text{bus}} \cup \mathcal{V}_{\text{gen}} \cup \mathcal{V}_{\text{load}}, \tag{5.5}$$

with typed directed relations between these sets. Generator and load states are therefore preserved individually.

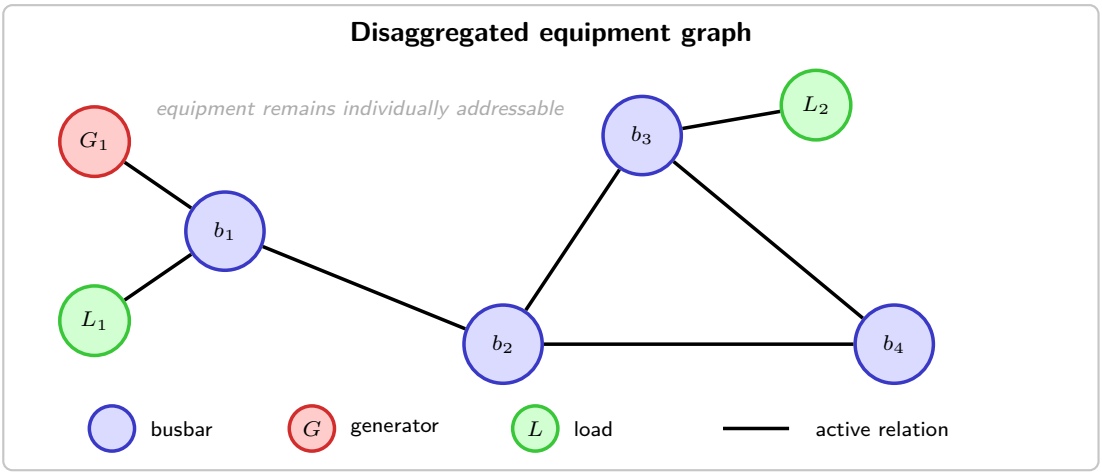

Figure 5.2: Heterogeneous equipment representation. Busbars are blue, generators are red, and loads are green. Physical transmission lines remain direct relations between busbar nodes.

Nodes and features

For a graph with S included substations, B busbars per substation, N_g included generators, and N_d included loads, the number of nodes is

$$|\mathcal{V}| = SB + N_g + N_d. \tag{5.6}$$

Busbar nodes are created exactly as in Section 5.2.2. In addition, each generator and load whose substation is present in the graph receives its own node.

Every busbar, generator, and load node is represented by the same ordered eight-dimensional feature vector

$$\begin{aligned}\mathbf{x}_v = [\,&\texttt{is\_busbar},\ \texttt{is\_generator},\ \texttt{is\_load},\ \texttt{p},\ \texttt{theta},\\ &\texttt{time\_before\_cooldown\_sub},\ \texttt{domain\_mask}\,]^{\mathsf{T}} \in \mathbb{R}^7.\end{aligned} \tag{5.7}$$

The width and meaning of this vector are identical for all node types. Table 5.6 shows the value placed in every shared column. A zero in the table means that the column is still present for that node type but is explicitly zero-filled.

Table 5.6: Values placed in the eight feature columns shared by every node in the direct-line heterogeneous graph. Disconnected assets retain their node row but have zero power and angle.

Shared feature column	Busbar node	Generator node	Load node
`is_busbar`	1	0	0
`is_generator`	0	1	0
`is_load`	0	0	1
`p`	0	`gen_p` if connected; 0 otherwise	`load_p` if connected; 0 otherwise
`theta`	0	`gen_theta` if connected; 0 otherwise	`load_theta` if connected; 0 otherwise
`time_before_cooldown_sub`	Substation value	0	0
`domain_mask`	1 controlled; 0 contextual	1 controlled; 0 contextual	1 controlled; 0 contextual

Relations and edge features

Every candidate directed edge is represented by the same ordered feature vector. Its physical block is

$$\begin{aligned}\mathbf{p}_{uv} = [\,&\texttt{line\_status},\ \texttt{rho},\ \texttt{timestep\_overflow},\\ &\texttt{time\_before\_cooldown\_line},\\ &(\texttt{time\_next\_maintenance},\ \texttt{duration\_next\_maintenance})_{\text{optional}}\,]^{\mathsf{T}}.\end{aligned} \tag{5.8}$$

The two maintenance columns are appended only when scheduled maintenance is available. The relation block is the same six-column vector for every edge:

$$\begin{aligned}\mathbf{r}_{uv} = [\,&\texttt{relation\_physical\_line},\\ &\texttt{relation\_same\_substation\_busbar},\\ &\texttt{relation\_generator\_to\_busbar},\ \texttt{relation\_busbar\_to\_generator},\\ &\texttt{relation\_load\_to\_busbar},\ \texttt{relation\_busbar\_to\_load}\,]^{\mathsf{T}}.\end{aligned} \tag{5.9}$$

The complete common edge vector is therefore

$$\mathbf{e}_{uv} = [\,\mathbf{p}_{uv}^{\mathsf{T}} \mid \mathbf{r}_{uv}^{\mathsf{T}}\,]^{\mathsf{T}} \in \begin{cases}\mathbb{R}^{10}, & \text{without maintenance columns},\\ \mathbb{R}^{12}, & \text{with maintenance columns}.\end{cases} \tag{5.10}$$

Its width, column order, and meaning do not change with the relation type. An active physical-line edge fills $\mathbf{p}_{uv}$ with its measured line state. Every active structural or asset-attachment edge instead uses

$$\mathbf{p}_{\text{neutral}} = \mathbf{0}, \tag{5.11}$$

so a relation carrying no flow reports none. In every active edge row, exactly one column of $\mathbf{r}_{uv}$ is one and the other five are zero.

Table 5.7 shows the values placed in these common blocks for every relation.

Table 5.7: Values placed in the common edge-feature vector shared by every active directed relation in the direct-line heterogeneous graph. "Other five" refers to the remaining columns of the six-column relation block.

Directed relation	Active rule	Shared physical block	Shared relation block
$i_o \to i_e$ and reverse (physical line)	Line online and candidate busbars match the current endpoints	Measured $\mathbf{p}_{uv}$	`relation_physical_line`= 1; other five = 0
$i(s,b) \to i(s,b')$, $b \neq b'$	Optional; every ordered same-substation busbar pair	$\mathbf{p}_{\text{neutral}}$	`relation_same_substation_busbar`= 1; other five = 0
$g \to i(s(g),b)$	Generator connected to candidate busbar b; direction enabled	$\mathbf{p}_{\text{neutral}}$	`relation_generator_to_busbar`= 1; other five = 0
$i(s(g),b) \to g$	Same mask; reverse direction enabled	$\mathbf{p}_{\text{neutral}}$	`relation_busbar_to_generator`= 1; other five = 0
$d \to i(s(d),b)$	Load connected to candidate busbar b; direction enabled	$\mathbf{p}_{\text{neutral}}$	`relation_load_to_busbar`= 1; other five = 0
$i(s(d),b) \to d$	Same mask; reverse direction enabled	$\mathbf{p}_{\text{neutral}}$	`relation_busbar_to_load`= 1; other five = 0

Generator and load directions are selected independently as `bidirectional`, `asset_to_busbar`, or `busbar_to_asset`. The default is `bidirectional`. Physical-line and same-substation relations remain bidirectional.

Local information boundary

The agent-local `heterogeneous` graph contains all busbars, generators, and loads of controlled substations. For each live boundary line it also contains the single far-end busbar to which the line is attached, since the line is still represented as a busbar-to-busbar edge. It never constructs a generator or load belonging to a neighboring substation, even when that asset is connected to the visible neighboring busbar. Neighboring private power and angle therefore cannot enter message passing or graph readout. Inactive far-end busbar candidates are masked in the same way as in the `bus` schema.

The consequence: a contextual busbar here carries no measurement. Unlike in the `bus` schema, a contextual busbar in the `heterogeneous` graph contains only type and role indicators. Power and angle are stored on asset nodes, and neighboring assets are omitted, so all neighboring measurements arrive through the boundary-line edge instead. This preserves the decentralized information boundary but leaves the contextual busbar largely structural. The explicit-line schema removes that busbar and places the shared measurements on the line node. Thus, comparing the `bus` and `heterogeneous` schemas also changes the available context, as discussed in Section 5.2.5.

5.2.4 Disaggregated Representation with Line Nodes

The `heterogeneous_line` schema replaces each direct physical-line edge with a transmission-line node between its endpoint busbars. Its node set is

$$\mathcal{V} = \mathcal{V}_{\text{bus}} \cup \mathcal{V}_{\text{gen}} \cup \mathcal{V}_{\text{load}} \cup \mathcal{V}_{\text{line}}. \tag{5.12}$$

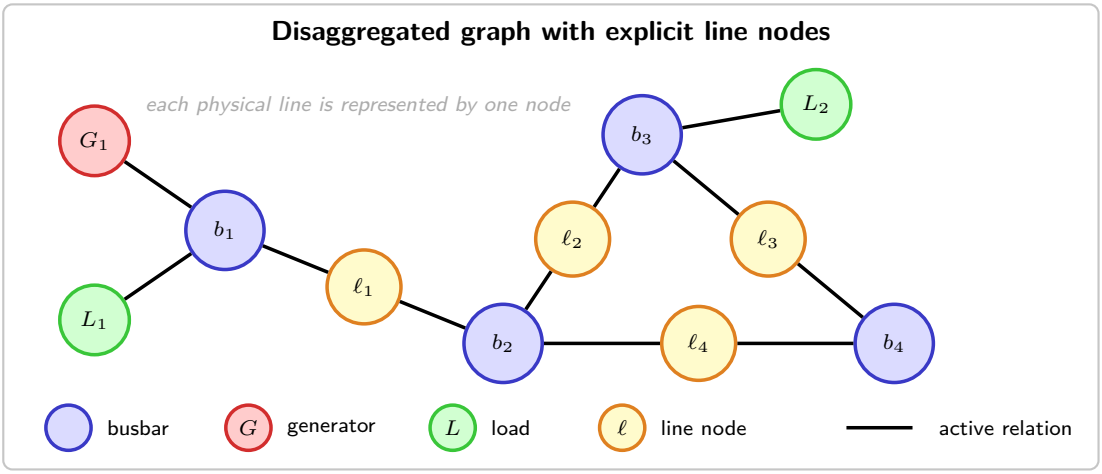

Figure 5.3: Heterogeneous representation with explicit transmission-line nodes. Yellow nodes represent physical lines and connect the blue busbars at their endpoints; generators and loads remain explicit terminal nodes.

Nodes and features

For S included substations, B busbars per substation, N_g generators, N_d loads, and L included transmission lines, the graph contains

$$|\mathcal{V}| = SB + N_g + N_d + L \tag{5.13}$$

nodes.

Every node type uses the same ordered feature vector:

$$\begin{aligned}
\mathbf{x}_v = [&\texttt{is_busbar}, \texttt{is_generator}, \texttt{is_load}, \texttt{is_transmission_line},\\
&\texttt{p}, \texttt{theta}, \texttt{line_status}, \texttt{rho}, \texttt{timestep_overflow},\\
&\texttt{time_before_cooldown_line},\\
&(\texttt{time_next_maintenance}, \texttt{duration_next_maintenance})_{\text{optional}},\\
&\texttt{time_before_cooldown_sub}, \texttt{domain_mask}]^{\mathsf{T}},\\
\mathbf{x}_v \in &\begin{cases} \mathbb{R}^{12}, & \text{without maintenance columns,} \\ \mathbb{R}^{14}, & \text{with maintenance columns.} \end{cases}
\end{aligned} \tag{5.14}$$

The width and meaning of this vector are identical for busbar, generator, load, and transmission-line nodes. Table 5.8 specifies every entry; 0 denotes an explicitly zero-filled column, not an omitted feature. For generators and loads, power and angle are set to zero when the asset is disconnected.

Table 5.8: Values placed in the common node-feature vector of the explicit-line heterogeneous graph, when voltage angles are carried as absolute per-asset values. The two maintenance rows are included only when maintenance observations are enabled. When the angle is instead carried as the drop along each line, the angle row changes owner: the generator and load entries become 0 and the transmission line holds the drop across it, set to 0 while the line is out (Section 5.1.2).

Shared feature column	Busbar	Generator	Load	Transmission line
`is_busbar`	1	0	0	0
`is_generator`	0	1	0	0
`is_load`	0	0	1	0
`is_transmission_line`	0	0	0	1
`p`	0	`gen_p` if connected; 0 otherwise	`load_p` if connected; 0 otherwise	0
`theta`	0	`gen_theta` if connected; 0 otherwise	`load_theta` if connected; 0 otherwise	0
`line_status`	0	0	0	Observed line value
`rho`	0	0	0	Observed ρ_ℓ
`timestep_overflow`	0	0	0	Observed line value
`time_before_cooldown_line`	0	0	0	Observed line value
`time_next_maintenance`	0	0	0	Observed line value
`duration_next_maintenance`	0	0	0	Observed line value
`time_before_cooldown_sub`	Substation value	0	0	0
`domain_mask`	1 controlled; 0 contextual	1 controlled; 0 contextual	1 controlled; 0 contextual	1 if either endpoint is controlled; 0 otherwise

Relations and edge features

Every candidate directed edge uses the same ordered nine-dimensional feature vector:

$$\begin{aligned}\mathbf{e}_{uv} = [&\texttt{relation\_same\_substation\_busbar},\\ &\texttt{relation\_busbar\_to\_line\_origin}, \texttt{relation\_line\_origin\_to\_busbar},\\ &\texttt{relation\_busbar\_to\_line\_extremity}, \texttt{relation\_line\_extremity\_to\_busbar},\\ &\texttt{relation\_generator\_to\_busbar}, \texttt{relation\_busbar\_to\_generator},\\ &\texttt{relation\_load\_to\_busbar}, \texttt{relation\_busbar\_to\_load}]^{\mathsf{T}} \in \mathbb{R}^9.\end{aligned} \tag{5.15}$$

These edges contain no continuous line-state attributes because the transmission-line node already contains `rho`, status, overflow, cooldown, and maintenance. For an active edge, exactly one relation column is one and the other eight are zero. An inactive candidate has all nine entries set to zero and is removed by `edge_mask`$= 0$ before message passing. Table 5.9 gives the active rule and the nonzero entries for every relation.

Table 5.9: Nonzero entries in the common 9-column edge-feature vector of the explicit-line heterogeneous graph. All relation columns not named in a row are zero.

Directed relation	Active rule	Relation column set to 1
$i(s,b) \to i(s,b')$, $b \neq b'$	Optional; every ordered same-substation pair	`relation_same_substation_busbar`
$i_o \to \ell$	Online line, current origin busbar, direction enabled	`relation_busbar_to_line_origin`
$\ell \to i_o$	Same mask; reverse direction enabled	`relation_line_origin_to_busbar`
$i_e \to \ell$	Online line, current extremity busbar, direction enabled	`relation_busbar_to_line_extremity`
$\ell \to i_e$	Same mask; reverse direction enabled	`relation_line_extremity_to_busbar`
$g \to i(s(g), b)$	Generator attached to b; direction enabled	`relation_generator_to_busbar`
$i(s(g), b) \to g$	Same mask; reverse direction enabled	`relation_busbar_to_generator`
$d \to i(s(d), b)$	Load attached to b; direction enabled	`relation_load_to_busbar`
$i(s(d), b) \to d$	Same mask; reverse direction enabled	`relation_busbar_to_load`

Local information boundary

In the agent-local `heterogeneous_line` graph, every line touching a controlled substation is itself part of the agent's observation/action graph. Only controlled busbars, generators, loads, and the corresponding line nodes are constructed. A boundary line node is connected only to its endpoint busbar inside the controlled domain; no candidate edge to its non-controlled endpoint and no neighboring busbar, generator,

or load node is created. The line node already carries the shared measurements, so no far-end context is needed. If the line is disconnected, the line node remains visible with `line_status`= 0, but all of its busbar-attachment edges are inactive. This construction is unchanged by the neighbor-inclusion option.

5.2.5 Local actor information boundary and the global state graph

Each agent a controls a set of substations $\mathcal{C}_a$. A boundary line connects $\mathcal{C}_a$ to another agent's domain, and its far-end busbar is the only possible neighboring busbar. The local graph exposes only the boundary information required by its representation.

Why one hop, and why not more. The one-hop boundary is a methodological choice: it keeps each actor regional while the centralized critic still observes the global state. Power flow has no natural one-hop cutoff, but the loading and angle drop of boundary lines provide the immediate interaction with the exterior without exposing neighboring asset states. One hop is therefore not claimed to be optimal.

Visibility and masking

Candidate rows are retained for fixed tensor shapes, but a contextual busbar is visible only while a live boundary line connects it to the controlled region. Otherwise its entire feature row is zeroed, every incident edge is deactivated and removed before message passing, and the row is excluded from pooling—including the denominator of a mean. It therefore affects neither message passing nor the readout. Same-substation edges cannot restore visibility.

Controlled nodes remain visible when disconnected. In `heterogeneous_line`, a shared-line node also remains visible because it belongs to the agent graph, although its attachment edges are inactive when the line is offline. Physical and attachment relations are active only when the line is online and the current busbar assignments match their candidate endpoints. Neighboring generators and loads are never constructed, and the explicit-line representation creates no far-end attachment. Pooling is thus a summary operation, not the information-security boundary.

Representation-dependent information

The representations are not information-equivalent. The busbar schema exposes the most neighboring information because it aggregates assets at the far-end busbar; the heterogeneous schema exposes only that busbar; and the explicit-line schema exposes only the shared line.

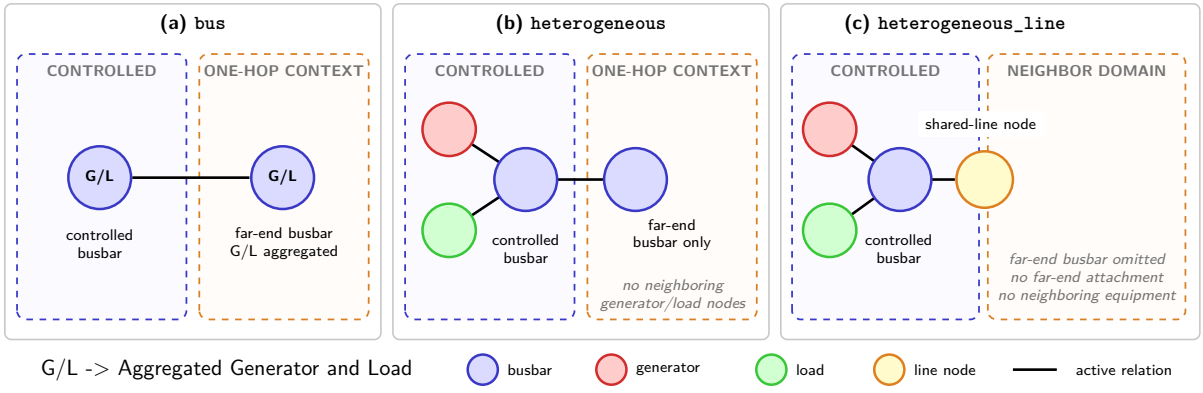

Figure 5.4: The same live boundary line for an agent controlling two substations, s_1 and s_2. The blue dashed region is the complete controlled zone; the solid inner boxes identify its substations. The busbar graph exposes aggregated measurements at the far end, the disaggregated graph exposes only the far-end busbar, and the explicit-line graph exposes only the shared line node. If the boundary line disconnects, the contextual busbar is masked in the first two graphs; in the last graph the line node remains visible but its attachment edge becomes inactive.

Table 5.10: Information from another agent's domain visible to a local actor.

Representation	Neighbor information available
`bus`	Shared-line edge and attached far-end busbar, including aggregated generator/load power and angles
`heterogeneous`	Shared-line edge and attached far-end busbar; no neighboring generator/load nodes or private-asset state
`heterogeneous_line`	Shared-line node only; no neighboring busbar, equipment, or far-end attachment

Comparing these representations therefore compares both architectures and actor information sets.

Legacy compatibility and centralized state

The legacy construction kept all neighboring busbars, and in heterogeneous graphs their private assets, visible even without a live boundary connection. Such rows could enter pooling even when isolated from message passing. Setting `gnn_context_requires_connection=false` restores this legacy visibility for A/B tests; legacy and leakage-free checkpoints are therefore not strictly comparable.

This boundaries apply only to decentralized actor graphs. The centralized critic still uses the full-grid state graph during training, while execution uses each actor's local graph, as required by MAPPO's centralized- training, decentralized-execution setup [7].

5.2.6 Graph actor and centralized critic

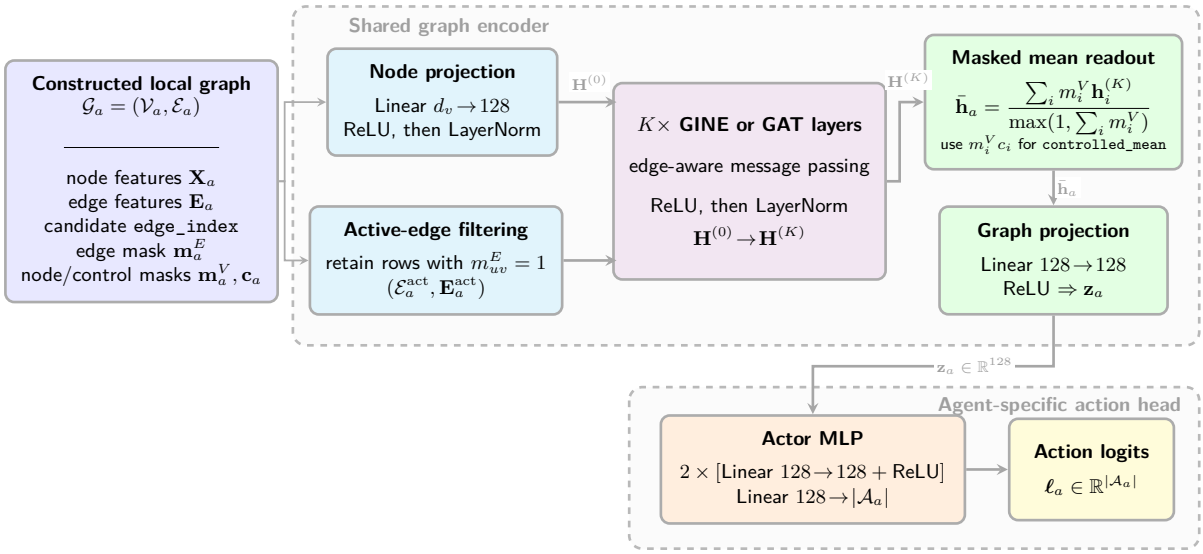

Figure 5.5: End-to-end graph-actor architecture. Candidate edges masked by the current topology are removed before message passing. The graph encoder and its output projection are shared across agents, whereas the final MLP has one output per action available to agent a and is therefore agent-specific.

GINE and GAT use the same actor pipeline and differ only in the message-passing operator. In the reported configuration, the input node projection and the two message-passing layers have 128 features. LayerNorm is applied after the input projection and after each message-passing layer. The graph readout produces a 128-dimensional embedding, followed by an agent-specific action head with two 128-unit hidden layers. The graph encoder and output projection are shared across agents, whereas the action heads are separate because their action-set sizes may differ.

This is the default configuration. The candidate-scoring alternative of Section 5.5 replaces this head.

The centralized critic is a separate MLP over the normalized flat state. It does not reuse the actor graph embedding and is used only during centralized training. Its flat-state normalization is independent of the graph-input normalization. The exact GINE and GAT updates, graph readout, output projection, and action-head equations are given in Appendix B.

Chapter Summary (1/3): Graph Construction and Model

Purpose: Define how the electrical state becomes the input of each actor and how that input is encoded.

- **Input processing:** Continuous physical quantities can be divided by fixed physical scales and optionally standardized from training statistics. The loading ratio `rho`, binary features, and masks remain unchanged. The graph-screening experiments use raw features; the cross-grid transfer study uses deterministic physical scaling.
- **Dynamic topology:** Nodes and candidate edges remain fixed, while time-dependent edge and node masks retain only the relations and contextual nodes that are electrically connected to the controlled region. An inactive contextual candidate has a zero feature row and participates in neither message passing nor readout.
- **Graph representations:** The `bus` schema aggregates equipment on busbars; `heterogeneous` gives generators and loads their own nodes; and `heterogeneous_line` also represents each transmission line as a node. This changes both the feature location and the number of message-passing steps between physical objects.
- **Actor and critic:** Each actor receives its controlled region and only the schema-specific boundary context of Table 5.10. The reference architecture uses a two-layer, 128-feature GINE or GAT encoder shared across agents, followed by an agent-specific action head. The centralized critic remains a separate MLP over the normalized full-grid flat state.

Takeaway: The graph variants change both how state is represented and how much neighboring information is available. The MAPPO training structure and agent partition remain unchanged, but the three representations must not be described as information-equivalent.

5.3 Substation Representation: Direct Edges and Summary Nodes

Two optional factors change how information is exchanged inside a substation. The factor e adds direct same-substation busbar edges. The factor n adds one learned substation-summary node per included substation. Both augmentations are part of the constructed graph G_a shown at the left of Figure 5.5; their nodes and edges therefore enter the same node projection, edge filtering, and message-passing pipeline as the base graph.

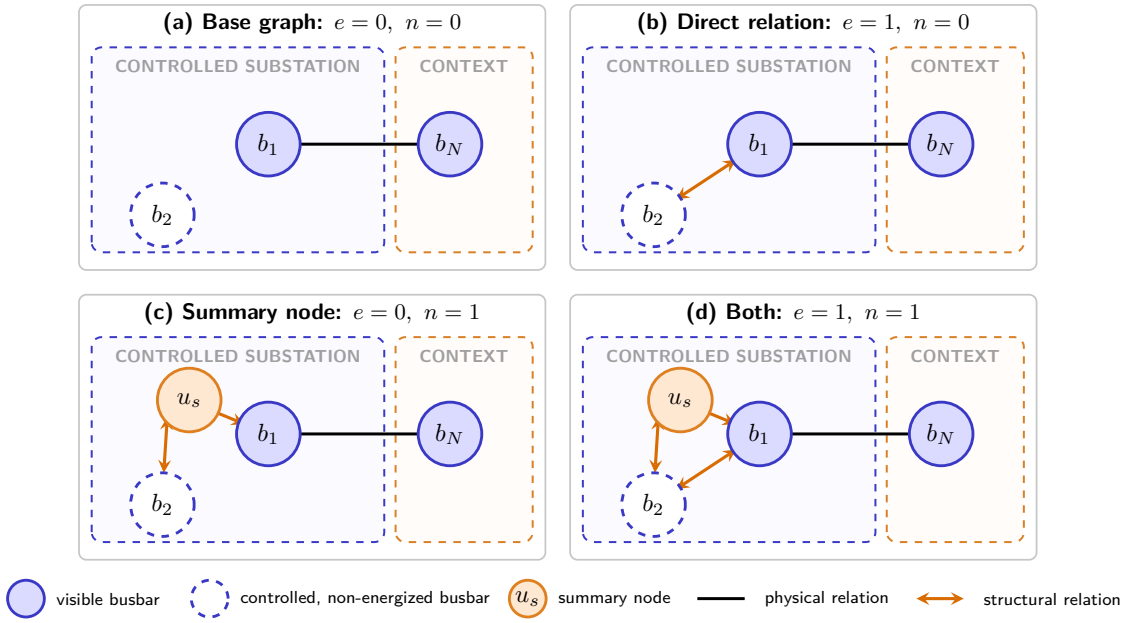

Figure 5.6: The four combinations of the independent substation factors e and n. The hollow node b_2 is controlled but non-energized. Factor e adds a bidirectional relation between the controlled busbars, while factor n connects a summary node u_s to both. Neither augmentation extends to the contextual busbar b_N, which retains only its physical relation.

Figure 5.7 places the combined augmentation in each graph representation.

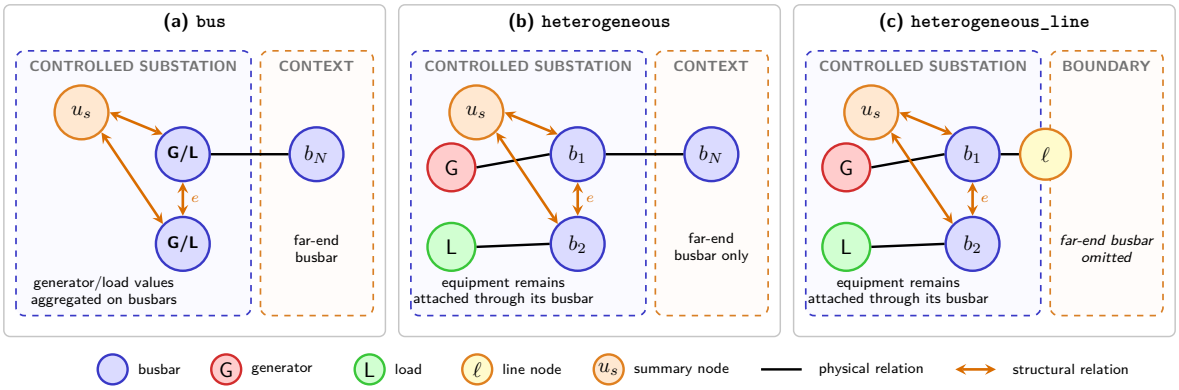

Figure 5.7: Placement of the two augmentations when $e = n = 1$. In every graph representation, the direct relation and summary node attach only to controlled busbars. Equipment and explicit line nodes retain their ordinary busbar relations, and no augmentation is added in the contextual domain.

5.3.1 Direct same-substation edges (e)

In addition to physical-line relations, the augmentation inserts $i(s, b) \to i(s, b')$ for every ordered pair $b \neq b'$ of busbars in the same included substation. The edges are therefore bidirectional between each pair of busbars, remain active throughout the episode, and do not represent an electrical connection. Consequently, a non-energized controlled busbar remains connected to the other busbar of its substation through this structural relation, without being considered electrically energized. Figure 5.7 shows this relation across the three graph representations.

Table 5.11 gives the exact feature values of the added edges. Every physical attribute is zero: a relation that carries no flow reports none.

These edges are restricted to controlled substations. The relation represents the actor's ability to move elements between busbars, not an electrical connection. Adding it inside a neighboring substation would therefore encode another agent's action space and create a nonphysical message path. The current construction applies the augmentation only within the actor's controlled domain, and these structural edges do not make contextual nodes visible.

Table 5.11: Node and edge features introduced by the direct same-substation augmentation. "None" means that the augmentation neither adds nodes nor changes the features of existing nodes.

Graph schema	Added node features	Feature values on each added directed edge
Busbar	None	All physical-line features are 0.
Disaggregated	None	`relation_same_substation_busbar`= 1; all other entries are 0.
Disaggregated with line nodes	None	`relation_same_substation_busbar`= 1; the other eight relation entries are 0.

5.3.2 Substation summary nodes (n)

In all experiments, augmentation n adds one learned summary node u_s for each included substation s and connects it in both directions to every busbar in $\mathcal{B}(s)$:

$$\mathcal{E}_{\text{sub}} = \{(i, u_s), (u_s, i) \mid i \in \mathcal{B}(s)\}\,; \tag{5.16}$$

For S included substations with B busbars each, this adds S nodes and $2SB$ directed edges. Each u_s gathers information from its busbars and sends the result back, so two busbars in the same substation communicate in two steps: $i(s, b) \to u_s \to i(s, b')$. It therefore provides the same intra-substation communication role as the direct relation, but over two message-passing steps. Only busbars connect directly to u_s; other node types reach it through their busbar attachments. Both augmentations are shown across the three graph representations in Figure 5.7.

Summary nodes exist only for controlled substations. Like the direct edges of Section 5.3.1, summary nodes serve for communication within a substation and are therefore created only in the actor's controlled domain. Adding them to neighboring substations would let contextual busbars exchange through the summary node and bypass that restriction. Each summary gathers only visible controlled busbars, so it introduces no contextual state directly; external information can affect it only through messages that first reach the controlled region.

After the raw node projection in Figure 5.5, the model initializes u_s from a *structural descriptor* of substation s rather than from its identity. Let $\boldsymbol{\phi}_s \in \mathbb{R}^4$ collect the number of busbars, incident lines, attached generators, and attached loads of that substation, each divided by a constant that is the same on every grid. The summary node starts at

$$u_s^{(0)} = \mathbf{W}_{\text{sub}}\, \boldsymbol{\phi}_s + \mathbf{b}_{\text{sub}}, \qquad \mathbf{W}_{\text{sub}} \in \mathbb{R}^{d_h \times 4},\ \mathbf{b}_{\text{sub}} \in \mathbb{R}^{d_h}, \tag{5.17}$$

and carries no current grid measurement. Messages from the busbars then produce an observation-dependent state for u_s, while PPO learns $\mathbf{W}_{\text{sub}}$ and $\mathbf{b}_{\text{sub}}$ with the other parameters.

An embedding table indexed by substation identity would have shape $S \times d_h$ and could not transfer between grids with different values of S. Equation 5.17 instead applies one shared projection to four normalized structural counts. Its parameter shapes are independent of grid size, while substations with different structures still receive different initial states. This makes the augmentation compatible with the cross-grid transfer of Chapter 8.

Table 5.12: Node and edge features introduced by the substation-summary augmentation. Here d_h is the hidden node width; $d_h = 128$ in the screened GINE models.

Graph schema	Added summary-node state	Feature values on $i \leftrightarrow u_s$
Busbar	$\mathbf{W}_{\text{sub}}\boldsymbol{\phi}_s + \mathbf{b}_{\text{sub}} \in \mathbb{R}^{d_h}$, from the substation's structural descriptor; no measured feature.	All physical-line features are 0.
Disaggregated	Same structural projection; no measured feature.	All physical and relation entries are otherwise 0.
Disaggregated with line nodes	Same structural projection; no measured feature.	All nine relation entries are 0.

In runs with mean readout, each u_s is marked as a valid node and is included once in the mean together with all other valid nodes. In runs with a virtual node, there is no terminal mean: the virtual node connects to the summary nodes, and its final state is used as the graph representation.

The summary-node and direct-edge augmentations may also be enabled together, and either may be combined with the virtual-node readout.

5.4 Pooling and Graph Readout

The final graph embedding is obtained either by pooling node embeddings directly or by reading out a learned virtual node. The output projection and action head are the same in both cases.

5.4.1 Mean, maximum, and sum pooling

Mean pooling averages the final embeddings of the valid nodes:

$$\overline{\mathbf{h}}_G = \frac{\sum_v m_v \mathbf{h}_v^{(K)}}{\max(1, \sum_v m_v)}, \tag{5.18}$$

where $m_v = n_{v,t}^{(a)}$ is the visibility mask. A single mean is computed over all visible node types; there is no separate pooling operation for each type. An inactive fixed-shape candidate contributes neither an embedding to the numerator nor one unit to the denominator. The mask is therefore an additional enforcement of the construction boundary, not an averaging of zero placeholders.

Maximum pooling instead selects the largest value independently in every hidden feature channel:

$$\left(\overline{\mathbf{h}}_G^{\max}\right)_c = \max_{v\,:\,m_v=1} \left(\mathbf{h}_v^{(K)}\right)_c . \tag{5.19}$$

It therefore keeps the strongest activation in each channel without depending on the number of nodes, but it does not preserve how many nodes produced a similar activation.

Sum pooling is available but is not used in any reported experiment. Its magnitude grows with the number of nodes. That number differs across agents, graph representations, and grids, so the scale of a sum-pooled embedding would also change across the transfer studied in Chapter 8.

For ordinary mean, maximum, or sum pooling, the pooled set is every *visible* node in the agent's local graph. It contains controlled nodes, including empty controlled busbars and disconnected controlled assets; the currently connected far-end busbars in `bus` and `heterogeneous`; and all boundary/internal line nodes in `heterogeneous_line`. It never contains an inactive contextual candidate, a neighboring private asset, or any neighboring equipment in the explicit-line schema. Valid substation-summary nodes are included when that augmentation is enabled. Direct same-substation edges add no nodes and do not change the pooled set.

The alternatives `controlled_mean`, `controlled_sum`, and `controlled_max` restrict terminal readout to the action domain. For controlled mean, define $q_v = m_v c_v$, where c_v is the static `controlled_node_mask`; then

$$\overline{\mathbf{h}}_G^{\text{ctrl}} = \frac{\sum_v q_v \mathbf{h}_v^{(K)}}{\max(1, \sum_v q_v)}. \tag{5.20}$$

Contextual busbars can still affect the controlled embeddings through active message-passing paths, but do not enter the final aggregation directly. The controlled pooled set is fixed for an episode, whereas the ordinary visible set can change when a neighboring line switches busbar or goes offline. This stabilizes the denominator, but it is a readout choice rather than the leakage fix: the graph construction and visibility masks enforce the information boundary for every pooling mode.

The optional `energized_mean` readout further requires a node to be incident to an active electrical relation; self and same-substation relations do not qualify. `controlled_energized_mean` additionally intersects this dynamic mask with the controlled-node mask. These masks affect only the terminal mean and its denominator, not message passing. They are therefore pooling ablations, not leakage fixes. Virtual-node runs do not use terminal pooling and are described in Section 5.4.2.

Comparison of pooling choices. The aggregation rule, pooled population, and contextual scope address distinct questions. Table 5.13 summarizes their main trade-offs before empirical comparison.

Table 5.13: Theoretical advantages and limitations of the pooling choices.

Choice	Main advantage	Main limitation
Mean	Smoothly summarizes the regional state and gives gradients to every pooled node.	A rare but critical activation may be diluted.
Maximum	Preserves the strongest activation and is insensitive to the number of pooled nodes.	Discards multiplicity, gives sparse gradients, and may combine channel maxima from unrelated nodes.
Controlled nodes	Aligns the readout with the action domain and keeps its population stable; context still enters through message passing.	Relies on message passing to transmit all useful boundary information.
All visible nodes	Inserts contextual information directly into the readout.	Context can dilute a mean or dominate a maximum, and the pooled set may change with boundary topology.
All controlled nodes, including non-energized	Retains empty busbars and disconnected equipment that remain relevant action targets.	Inactive nodes may dilute a mean, although their embeddings are not necessarily zero after message passing.
Energized controlled nodes only	Focuses the readout on the network currently carrying power.	Removes available action targets and makes the pooled set change after switching or disconnection.
Include one-hop context	Supplies boundary conditions that affect flows in the controlled region.	Adds external information that the agent cannot directly control.
Exclude one-hop context	Gives a simpler, strictly local, and more stable observation.	Increases partial observability of boundary conditions.

Theoretical default. Pending ablation results, the most defensible choice is `controlled_mean` over all controlled nodes, with one-hop context used for message passing but excluded from terminal pooling. This representation is stable, aligned with the action domain, and retains empty busbars that actions may use. `controlled_max` is a plausible risk-sensitive alternative when decisions are expected to depend on one critical condition. Without one-hop context, the controlled and visible sets coincide; in the explicit-line schema, the context option has little effect because boundary line nodes are already present.

5.4.2 Virtual-node graph readout

A virtual-node readout replaces terminal mean, sum, maximum, or attention pooling with one learned graph-summary node. It can be combined with any of the three graph schemas.

Augmented graph

Let $v_\star$ denote the virtual node. Define the summary set $\mathcal{V}_{\text{sum}}$ as the controlled substation nodes when the augmentation from Section 5.3.2 is enabled, and as the visible controlled busbar nodes otherwise. The augmented graph is

$$\mathcal{V}^+ = \mathcal{V} \cup \{v_\star\}, \tag{5.21}$$

$$\mathcal{E}^+ = \mathcal{E} \cup \{(i, v_\star) \mid i \in \mathcal{V}_{\text{sum}}\}. \tag{5.22}$$

All reported experiments use a toward-virtual direction: every added edge points from a node in $\mathcal{V}_{\text{sum}}$ to $v_\star$. The virtual node therefore receives information but never sends messages back. It connects to visible controlled busbars, or to the controlled substation-summary nodes when that augmentation is enabled. It has no direct edge to contextual busbars, generators, loads, or transmission-line nodes; those nodes can affect the readout only through ordinary message-passing paths into the controlled region.

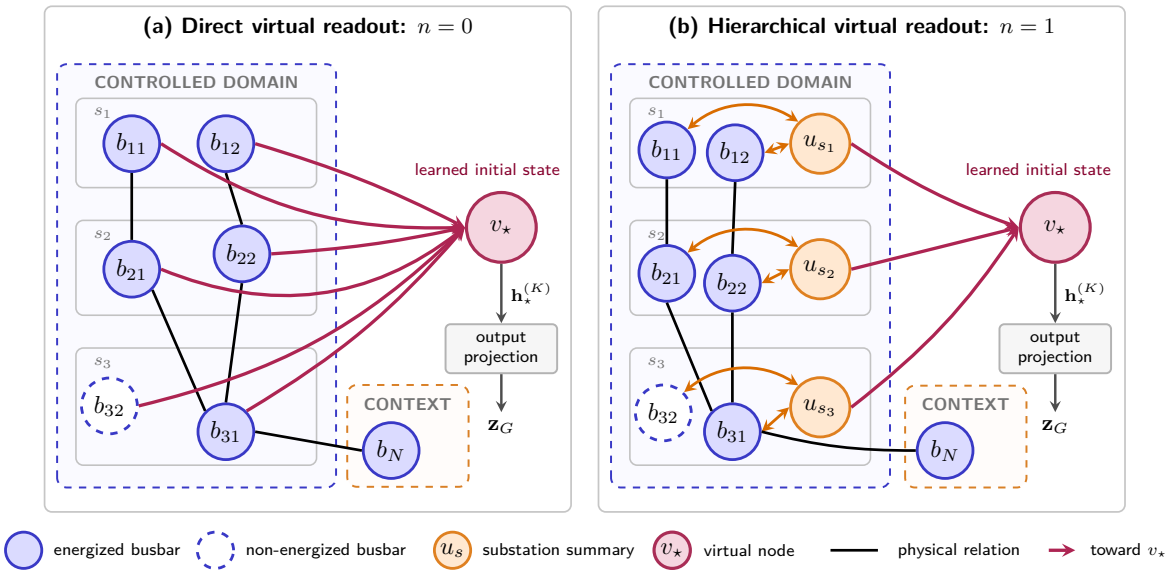

Figure 5.8: Virtual-node readout in the reported one-way setting. Without substation nodes, every controlled busbar—including non-energized b_{32}—sends directly to $v_\star$. With augmentation n, the same busbars communicate through u_s, and only the substation nodes send to $v_\star$. The final state $\mathbf{h}_\star^{(K)}$ replaces terminal pooling.

Learned summary instead of terminal pooling

The initial virtual representation $\mathbf{h}_\star^{(0)}$ is learned. A message-passing layer jointly updates the physical and virtual nodes:

$$\left(\{\mathbf{h}_i^{(k+1)}\}_{i\in\mathcal{V}}, \mathbf{h}_\star^{(k+1)}\right) = \text{GNN}^{(k)}\left(\{\mathbf{h}_i^{(k)}\}_{i\in\mathcal{V}}, \mathbf{h}_\star^{(k)}, \mathcal{E}^+\right). \tag{5.23}$$

Because every virtual edge points toward $v_\star$, it only aggregates the node states and cannot influence them during message passing. After K layers, the graph embedding is

$$\mathbf{z}_G = \text{ReLU}\left(\mathbf{W}_o \mathbf{h}_\star^{(K)} + \mathbf{b}_o\right), \tag{5.24}$$

rather than from $\text{Pool}(\{\mathbf{h}_i^{(K)}\}_{i\in\mathcal{V}})$. The output dimension is unchanged.

Relation attributes and encoder compatibility

Virtual connections are structural: their continuous edge attributes are zero, except for a neutral unit value in the `rho` position when that feature exists.

Without substation nodes, a graph containing $N_{\text{bus}}^{\text{ctrl}}$ visible controlled busbars receives one virtual node and $N_{\text{bus}}^{\text{ctrl}}$ directed edges. With the hierarchy enabled, the virtual-node stage instead adds S edges

from the S controlled substation-summary nodes. After one layer the virtual node has aggregated its immediate busbar or substation neighbors.

Chapter Summary (2/3): Augmentations and Graph Readout

Purpose: Separate changes to information exchange inside the graph from changes to the final graph-level representation.

- **Direct substation edges (e):** Add structural relations between busbars of the same substation, without adding nodes or representing a physical transmission line.
- **Substation nodes (n):** Add one learned node per substation. It has no raw electrical measurement, is updated from its busbars, and can be included in the graph readout.
- **Terminal pooling:** Ordinary mean pooling averages every visible node, including electrically connected contextual busbars and valid added substation nodes; masked contextual candidates are excluded. A `controlled_*` readout instead aggregates only the fixed action-domain nodes, while still allowing visible context to influence them through message passing. Maximum pooling keeps the largest value in each hidden channel. Sum pooling is not used because its scale changes with the number of nodes.
- **Virtual readout (v):** Replace terminal pooling with one learned grid-summary node. Messages travel only toward this node, from the busbars or from the added substation nodes, and its final state becomes the graph embedding.

Takeaway: Factors e and n modify message passing, whereas pooling and factor v determine how the encoded local graph is reduced to one vector for action selection. These choices can be combined with any base graph schema.

5.5 Candidate-Action Scoring from the Explicit-Line Graph

The default action head produces one output column per discrete action. That column is tied to an action index and knows nothing about the physical objects the action modifies. This section describes an alternative head that scores each action from the graph nodes it touches. It is available only for the explicit-line schema of Section 5.2.4 and is selected with `actor_action_head=candidate_pool`; the default remains the action-index head of Appendix B.5. The PPO objective, the categorical policy, sampling, and deterministic evaluation are unchanged.

Checkpoint compatibility. The `cas_hl_NL` and `cas_hl_NLS` checkpoints were trained with the former 13-column explicit-line node schema, which still included the redundant `connected` feature. The current schema has 12 columns; full-test evaluation detects these checkpoints and restores the legacy column, thereby preserving their original input representation.

5.5.1 From action indices to candidate scoring

Throughout this section, $\mathbf{z}_a$ denotes the graph embedding of agent a: the node states of its local graph pooled as in Section 5.4 and passed through the output projection (Equation B.10). It is one 128-dimensional vector per agent per step, and it is the only thing the actor has ever seen of the graph.

With the default head, agent a produces its logit vector in one step from $\mathbf{z}_a$:

$$\boldsymbol{\ell}_a = \mathrm{MLP}_a(\mathbf{z}_a), \qquad \boldsymbol{\ell}_a \in \mathbb{R}^{|\mathcal{A}_a|}. \tag{5.25}$$

The last layer has one row per action index, so what makes an action good must be learned separately for every index, and the head grows with $|\mathcal{A}_a|$.

Candidate scoring replaces that by one scalar-valued network shared across all non-idle actions. The idle

action may use the same network or a dedicated one:

$$\ell_{a,j} = \text{score}\left(\underbrace{\mathbf{z}_a}_{\text{whole graph}}, \underbrace{\mathbf{c}_j}_{\text{nodes action } j \text{ touches}}, \underbrace{\boldsymbol{\phi}_j}_{\text{optional action descriptor}} \right), \qquad j \geq 1. \tag{5.26}$$

The same parameters score all non-idle actions, so the head no longer grows with $|\mathcal{A}_a|$, and two actions are ranked by the difference between their touched-node contexts rather than by two independent output rows. This is what makes the explicit line nodes usable: an action next to a loaded line is scored from that line's own embedding.

The implementation separates three choices, summarized in Table 5.14. They can be combined independently, so the two uniform pooling rules give eight variants. Learned attention adds a third pooling family, described after the uniform controls.

Table 5.14: Independent choices in the candidate-action scorer.

Component	First choice	Alternative
Touched-node context	Average all touched nodes together into one vector.	Average busbars, lines, loads, and generators separately, then concatenate the four vectors.
Static action descriptor	Omit it and score from the graph state and touched nodes alone.	Append the action indicators and counts listed in Table 5.16.
Idle action	Use the shared scorer with an empty touched-node context.	Use a dedicated scorer that reads only the global graph embedding.

Figure 5.9 shows how these independent choices enter one candidate score.

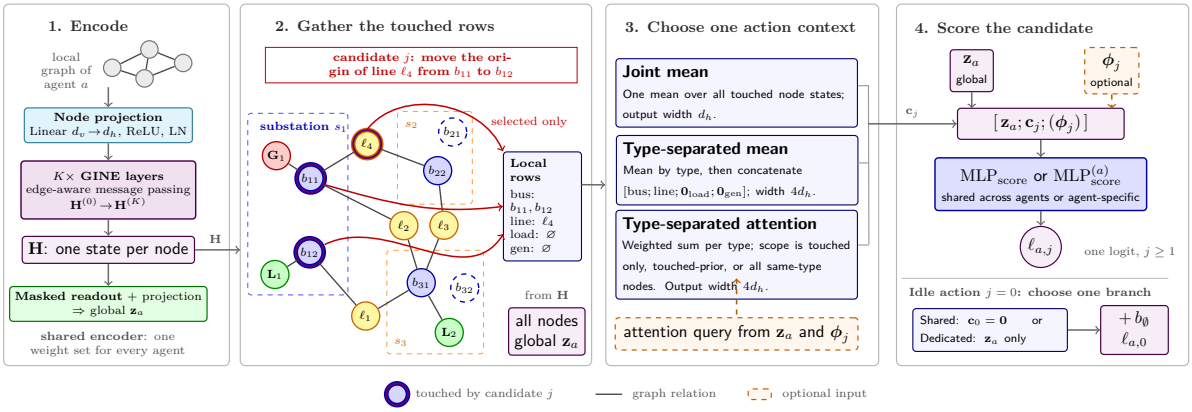

Figure 5.9: Candidate-action scoring for an endpoint-move candidate. Stage 1 is the shared GNN encoder, given in full in Figure 5.5: it maps the local graph to node states $\mathbf{H}$, and the same stack yields the global embedding $\mathbf{z}_a$ by masked readout and projection, so the two inputs to the scorer come from one set of weights rather than two. In stage 2 the violet outlines mark the only rows gathered from $\mathbf{H}$ for the local context: both busbars of the modified substation and the touched explicit-line node. One pooling construction then produces $\mathbf{c}_j$. The static descriptor $\boldsymbol{\phi}_j$ may be appended before the shared non-idle scorer. The idle action uses either zero local context in that scorer or a dedicated scorer.

The following subsections define these inputs and explain their consequences.

5.5.2 What each action touches

The first ingredient is one lookup table per agent, with one row per action, answering a single question: which nodes of the local graph does this action physically touch? It is built once, when the environment is created, and never recomputed during training.

A Grid2Op topology action states which entries of the topology vector it changes, and each entry belongs to a known object: a line endpoint, a generator, or a load. Reading those entries gives the objects the action modifies, and the graph builder's row maps turn each object into the row of the graph that holds its embedding. Table 5.15 follows one action through this.

Table 5.15: One action, from Grid2Op to node rows: moving the origin end of line 4 onto the other busbar of substation 6. Row indices are illustrative.

Step	Result for this action
Objects the action changes	Substation 6, line 4
Their rows in the graph	Both busbars of substation 6, here 12 and 13, and the line node of line 4, here 22
Stored for scoring	Busbar $[12, 13]$, line $[22]$, generator and load empty

Two decoding rules are worth stating. A touched substation contributes *all* of its busbar rows, not only the busbar in use, because the action changes which of them the equipment sits on. A boundary line contributes its explicit line-node row and the controlled endpoint rows that are present in the local graph; scoring does not add the far-end busbar or any neighboring private asset. The rows are stored as fixed-size arrays with a mask, padded to the widest action, so that a single gather serves the whole action space.

Write $\mathcal{N}_{\text{bus}}(j)$, $\mathcal{N}_{\text{line}}(j)$, $\mathcal{N}_{\text{load}}(j)$, and $\mathcal{N}_{\text{gen}}(j)$ for the four sets of node rows that action j touches. Because a topology change moves line endpoints, the explicit line node must be present whenever the line touches the agent's controlled domain. The leakage-free construction guarantees that membership without constructing the non-controlled endpoint.

Each action also carries a static feature vector $\boldsymbol{\phi}_j \in \mathbb{R}^{12}$, listed in Table 5.16. It describes what the action does independently of the current grid state, so the scorer can tell a single endpoint move from an action that reconfigures a whole substation even when both touch the same nodes.

Table 5.16: The static action-feature vector $\boldsymbol{\phi}_j$. The five indicators are binary; the seven counts are divided by their largest value over that agent's action space, so every entry lies in $[0, 1]$ and is independent of the grid size.

Entry	Meaning
Indicators	
`is_do_nothing`	The action is the idle action
`has_busbar_context`	It modifies at least one substation
`has_line`	It touches at least one line
`has_load`	It touches at least one load
`has_generator`	It touches at least one generator
Scaled counts	
`n_substations`	Substations modified
`n_topology_changes`	Topology-vector entries changed
`n_line_status_changes`	Lines connected or disconnected
`n_line_endpoint_changes`	Line endpoints moved to another busbar
`n_generator_changes`	Generators moved to another busbar
`n_load_changes`	Loads moved to another busbar
`n_other_changes`	Changed objects of any other type

The descriptor is optional in the final scorer. Including it tells the model what operation is proposed and can distinguish actions that touch similar nodes; omitting it isolates what can be learned from the graph state and the touched-node context alone. This choice is independent of the pooling rule and of the idle-action treatment.

5.5.3 Pooling the touched nodes

The second ingredient turns the touched rows into $\mathbf{c}_j$. In $\mathbf{z}_a$ the nodes have already been averaged away, so the encoder also returns the node states themselves:

$$(\mathbf{z}_a, \mathbf{H}) = \text{encoder}_{\text{nodes}}(\mathcal{G}_t^{(a)}), \qquad \mathbf{H} \in \mathbb{R}^{|\mathcal{V}| \times d_h}, \tag{5.27}$$

where row v of $\mathbf{H}$ is the state $\mathbf{h}_v^{(K)}$ of node v after the last message-passing layer. These are exactly the vectors that mean pooling averages into $\mathbf{z}_a$; nothing new is computed, and a run with the default head is unaffected. Only the physical nodes appear in $\mathbf{H}$: substation-summary and virtual nodes are excluded, since an action touches equipment and not a learned hub.

For each object type $t \in \{\text{bus}, \text{line}, \text{load}, \text{gen}\}$, the masked mean is

$$\mathbf{c}_j^{(t)} = \frac{1}{\max\left(1, |\mathcal{N}_t(j)|\right)} \sum_{v \in \mathcal{N}_t(j)} \mathbf{h}_v^{(K)}, \tag{5.28}$$

where the clamped denominator makes an untouched type give the zero vector instead of a division by zero. Uniform pooling can either average all touched nodes jointly,

$$\mathbf{c}_j^{\text{joint}} = \frac{\sum_t \sum_{v \in \mathcal{N}_t(j)} \mathbf{h}_v^{(K)}}{\max\left(1, \sum_t |\mathcal{N}_t(j)|\right)}, \tag{5.29}$$

or preserve node roles by concatenating the four type-specific means,

$$\mathbf{c}_j^{\text{sep}} = \left[\mathbf{c}_j^{(\text{bus})}; \mathbf{c}_j^{(\text{line})}; \mathbf{c}_j^{(\text{load})}; \mathbf{c}_j^{(\text{gen})}\right]. \tag{5.30}$$

The available local-context constructions are summarized in Table 5.17.

Table 5.17: Implemented constructions of the action-specific node context.

Construction	Eligible rows	Aggregation	Width
Joint uniform mean	All nodes touched by the action	One equal-weight mean; node roles are mixed	d_h
Type-separated uniform mean	All nodes touched by the action	One equal-weight mean per type, then concatenation	$4d_h$
Type-separated learned attention	Determined by the attention scope	One action-conditioned weighted sum per type, then concatenation	$4d_h$

With learned attention, an action-specific query assigns normalized weights within each node type:

$$\mathbf{c}_j^{(t)} = \sum_{v \in \mathcal{E}_t(j)} \alpha_{jv}^{(t)} \mathbf{V}\mathbf{h}_v^{(K)}, \qquad \sum_{v \in \mathcal{E}_t(j)} \alpha_{jv}^{(t)} = \mathbb{1}[|\mathcal{E}_t(j)| > 0]. \tag{5.31}$$

Here $\mathcal{E}_t(j)$ is the eligible set; an empty set returns the zero vector. The implemented choices are shown in Table 5.18. The action query is formed from the global graph state and the static action descriptor in the reported attention comparisons. This is separate from whether the descriptor is also appended to the final scorer.

Table 5.18: Possible eligibility rules for learned action-specific attention.

Eligibility rule	Nodes allowed to receive weight	Purpose
Fixed physical support	Only nodes identified as touched by the action	Learn their relative importance without changing the physical mapping.
Physical support as a prior	Every local node of the same type, with a score bonus for touched nodes	Allow attention to extend the physical mapping while favoring it initially.
Fully learned support	Every local node of the same type, without a score bonus	Test whether the model can discover the relevant nodes without the mapping as a prior.

The two uniform means are parameter-free controls. Learned attention can retain a critical node that a mean would dilute, but adds parameters and must also learn a sensible weighting.

5.5.4 Scoring and the idle action

All non-idle actions are scored by one network with a single output unit. Its input depends on whether the static action descriptor is included:

$$\mathbf{x}_{a,j} = \begin{cases} [\mathbf{z}_a; \mathbf{c}_j; \boldsymbol{\phi}_j], & \text{with the descriptor,} \\ [\mathbf{z}_a; \mathbf{c}_j], & \text{without it,} \end{cases} \qquad \ell_{a,j} = \mathrm{MLP}_{\mathrm{score}}(\mathbf{x}_{a,j}), \quad j \geq 1. \tag{5.32}$$

The idle action touches no node, so its local context is the zero vector. It can either use the same scorer as the non-idle actions,

$$\ell_{a,0} = \mathrm{MLP}_{\mathrm{score}}(\mathbf{x}_{a,0}) + b_{\varnothing}, \qquad \mathbf{c}_0 = \mathbf{0}, \tag{5.33}$$

or use a dedicated scorer over the global graph embedding:

$$\ell_{a,0} = \mathrm{MLP}_{\varnothing}(\mathbf{z}_a) + b_{\varnothing}, \tag{5.34}$$

In both cases $b_{\varnothing}$ is a learned scalar applied only to the idle action. Sharing the scorer preserves parameter sharing across the full action set, but presents it with an empty context not seen for non-idle actions. A dedicated scorer instead treats "do nothing" as a global safety decision, at the cost of extra parameters and a different scoring mechanism.

5.5.5 Cost and current limits

The candidate contexts form a tensor of shape $[\text{batch}, |\mathcal{A}_a|, d_c]$, so memory grows with the size of the action space. This is the reason to start on `bus14` and on reduced action spaces before the full WCCI one.

The compared variants do not have identical parameter counts: separating node types widens the scorer input, static descriptors add input columns, learned attention adds projections, and a dedicated idle scorer adds another network. These counts should therefore be reported. When the dedicated idle scorer is used, its different mechanism also makes the action-0 frequency an explicit comparison metric rather than something assumed comparable.

The head currently requires the explicit-line graph and does not combine with the intervention gate, which is refused at construction rather than silently ignored. One-hop context is not required when using the angle-drop representation: the neighbor-inclusion setting adds no far-end equipment to this representation, because the shared-line state is supplied by the line node itself. Auxiliary heads over the same node embeddings, such as predicting which lines are about to overload, are a possible later addition.

5.6 Action-Space Reduction for Large Grids

The representations above change what an agent sees. This section changes what it can do, and is what makes the larger grids tractable at all: the WCCI bus36 setting has a far larger discrete topology-action space than `bus14`, so before training MAPPO on it the action space is reduced offline from simulated action outcomes. During rollouts, states are collected when the grid is stressed,

$$\rho_{\max} > 0.90.$$

For each collected state s, candidate actions are simulated and compared against do nothing:

$$\Delta(s, a) = \rho_{\mathrm{after}}(s, a) - \rho_{\mathrm{after}}(s, 0).$$

An action is counted as useful when it lowers the post-action loading relative to do nothing, i.e. $\Delta(s, a) < -\epsilon$. Each action is then scored by its empirical improvement rate,

$$\mathrm{score}(a) = \frac{\#\{\text{tested states where } a \text{ improves the grid}\}}{\#\{\text{tested states where } a \text{ is evaluated}\}},$$

with a minimum-count filter to avoid selecting rarely sampled actions. The final reduced action space keeps the best actions per agent, while always preserving action 0. The candidate sets are then sanity-checked with a one-step simulator-greedy controller: at each risky state, the controller simulates the available reduced actions and executes the best improving one. That check also sizes the reduced space, since it shows how much survival is still reachable once the action set is cut. Table 5.19 reports it for five sizes; the survival of trained agents on this environment is a separate question and is not covered here.

Table 5.19: One-step greedy evaluation of WCCI reduced action spaces on 50 random test chronics. All rows use the same chronic sample (`chronic_sample_seed=0`); the do-nothing mean survival on this sample is 28.30%.

Reduced action space	Kept actions per agent (a_0, a_1, a_2, a_3)	Greedy survival (%)	Do-nothing survival (%)	Survival delta (%)	Greedy full survival (%)
Do-nothing reference	$1, 1, 1, 1$	–	28.30	–	–
WCCI $h = 1$, `mk64`	$64, 64, 64, 64$	55.24	28.30	26.94	22.00
WCCI $h = 1$, `mk128`	$77, 128, 127, 128$	60.65	28.30	32.35	22.00
WCCI $h = 1$, `mk256`	$77, 256, 127, 256$	68.61	28.30	40.31	36.00
WCCI $h = 1$, `mk512`	$77, 512, 127, 512$	71.21	28.30	42.90	40.00
WCCI $h = 1$, `mk1024`	$77, 1024, 127, 1024$	78.18	28.30	49.87	54.00

Chapter Summary (3/3): Action Scoring and Large-Grid Reduction

Purpose: Define how graph embeddings become action logits and how the large WCCI action space is made tractable.

- **Default action head:** An agent-specific MLP maps the graph embedding directly to one logit per discrete action.
- **Candidate-action head:** The optional explicit-line variant scores each non-idle action from the encoded nodes it modifies and from static action descriptors. The idle action has a separate score. The PPO objective and categorical policy are unchanged.
- **Current scope:** Candidate scoring requires the explicit-line graph, and its memory cost still grows with the number of candidate actions. One-hop context is optional, since the line node already carries the shared measurements.
- **WCCI reduction:** Actions are ranked offline by how often they reduce loading in simulated stressed states. Action 0 is always retained, and the reduced sets are checked with a one-step simulator-greedy controller before policy training.

Takeaway: Graph representation, action scoring, and action-space size are separate design choices. This chapter defines them; the following chapters evaluate the resulting policies.

Chapter 6

Graph-Design Screening

This chapter begins by verifying that a shared GNN actor can match the tuned MLP baseline, then reports the graph-design screens. Screens A and B are the completed seed-0 campaigns on the corrected information boundary; Screens C and D retain the earlier three-seed legacy campaigns; Screen E is a seed-0 factorial on the corrected boundary. All ask whether a specific choice in the actor of Chapter 5 changes survival on held-out chronics.

Information-boundary status of the reported screens

The preliminary viability runs and Screens C–D predate the corrected local-graph construction of Section 5.2.5. Their fixed candidate graphs marked all rows of a neighboring substation as visible, including unattached busbars and, in heterogeneous graphs, neighboring private assets. Those isolated rows could enter graph readout even when no live boundary line connected them to the agent. Screens C–D therefore characterize the *legacy, information-leaking observation* and must not be presented as strictly comparable to checkpoints trained with the corrected boundary. Screens A, B, and E use connected context and controlled-only structural relations, and are the leakage-free results reported in this chapter. The corrected Screens A and B supersede the earlier campaigns under those names.

- **Screen A** varies the depth and width of the actor encoder, which applies to every graph schema: a 3×4 grid, 12 seed-0 runs. *Complete.*
- **Screen B** varies which nodes enter graph readout and whether they are reduced by a mean or a maximum: a 4×2 factorial, seven new seed-0 runs plus one reused Screen-A cell. *Complete.*
- **Screen C** varies the three structural augmentations of the busbar graph – same-substation edges, substation-summary nodes, and virtual-node readout (Sections 5.3 and 5.4.2): a 2^3 factorial, 24 runs. *Complete.*
- **Screen D** varies the direction of the generator and load attachment relations in the disaggregated graph (Section 5.2.3): a 3×3 factorial, 27 runs. *Complete.*
- **Screen E** varies candidate-node pooling, the action descriptor, and the idle-action head on the leakage-free explicit-line graph: a 2^3 factorial, eight runs at seed 0, repeated with corrected graph inputs. *Complete.*

Screens A and B form one corrected-boundary sequence on the busbar graph and share the nominal `mp2_h128` GINE/mean reference. Screen E is the other corrected-boundary screen, on the explicit-line graph. Screens C and D are kept as separate legacy evidence and are not compared numerically with the corrected campaigns.

6.1 Preliminary GNN Encoder Viability

Before screening individual graph-design choices, an exploratory study checks that a graph actor can match the tuned MLP baseline. This is a prerequisite for transfer: the agents must be able to share one size-independent encoder without sacrificing survival.

Question. Can a shared graph encoder perform as well as the MLP? If each agent requires a separate encoder, or if the GNN cannot train stably, its structural transferability is lost.

Baseline and changes. The study compares several GINE variants. It varies encoder size (light or heavy), actor sharing (shared or unshared encoder), critic architecture (MLP or GNN), and the training optimizations established for the MLP baseline, including optimized critic updates and reward normalization.

Protocol. All GINE runs use `gnn_type = gine`, 15,000,000 total timesteps, evaluation every 80,000 timesteps, 2,000 rollout steps, deterministic evaluation, and the same 80/20 chronic split used by the MLP experiments. GINE is used because it incorporates edge features and is expressive over graph structures [19]. The complete graph-actor pipeline is described in Appendix B; Appendix D gives the full configurations in Tables D.1 and D.2.

Evidence. Table 6.1 and Figure 6.1 compare the main architectural improvements with the `MLP Baseline`. Figure 6.2 shows the broader search over isolated changes.

Table 6.1: GINE rerun survival summary.

Run	Intended change	Final survival (%)	Peak survival (%)
`Heavy GINE Baseline`	Historical GINE baseline	59.2	93.9
`Light GINE (Concat, MLP Critic)`	Light GINE with MLP critic	78.9	90.4
`No Bias, Opt Critic`	Optimized critic and bias 0.0	82.5	85.7
`Optimized Light GINE`	Light optimized GINE with MLP critic	95.7	97.4

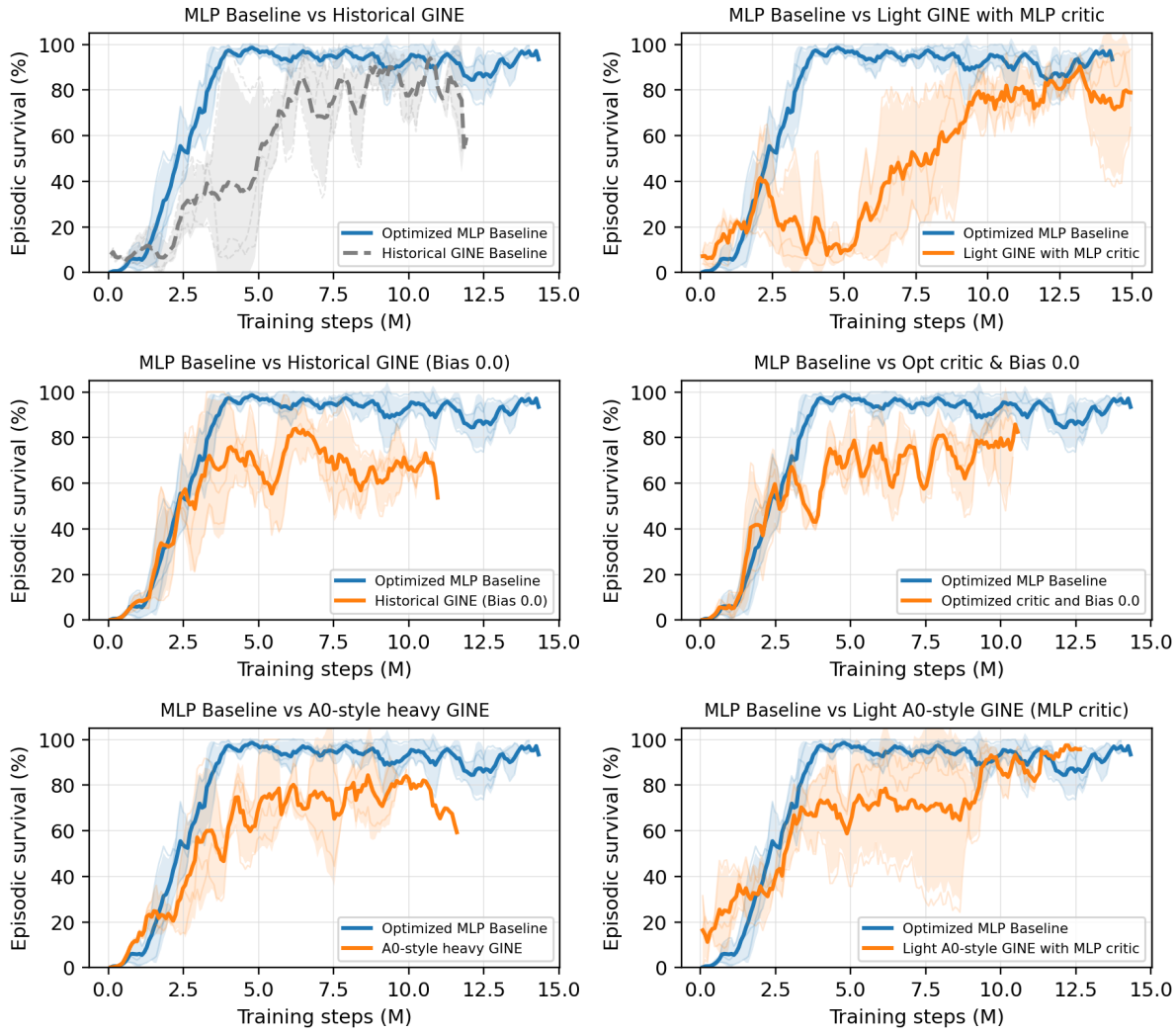

Figure 6.1: Seed-aggregated test episodic survival comparing the `MLP Baseline` with high-performing GINE architectural variants.

Interpretation. The broad search gives three conclusions:

- **Shared encoder viability.** The unshared-actor variant does not outperform the shared variants, so one encoder can serve all agents without becoming the performance bottleneck.
- **Critic architecture.** A GNN centralized critic is unstable and degrades performance. The strongest variants pair the shared GNN actor with the standard MLP critic.
- **Optimization dependence.** Optimized critic updates, reward normalization, and removal of the initial do-nothing bias are also important for stable GNN training.

The strongest result is the `Optimized Light GINE`, which combines the shared light GINE actor, the MLP critic, and the baseline optimizations to reach 95.7% final survival. A shared GNN is therefore competitive on `bus14`, justifying the representation screens that follow.

6.2 How the Screens Are Organized

Each screen is presented as one block containing its design and its outcome together:

1. **Question.** Which design choice is uncertain?
2. **Design.** Which factors are varied, at which levels, and what is held fixed.

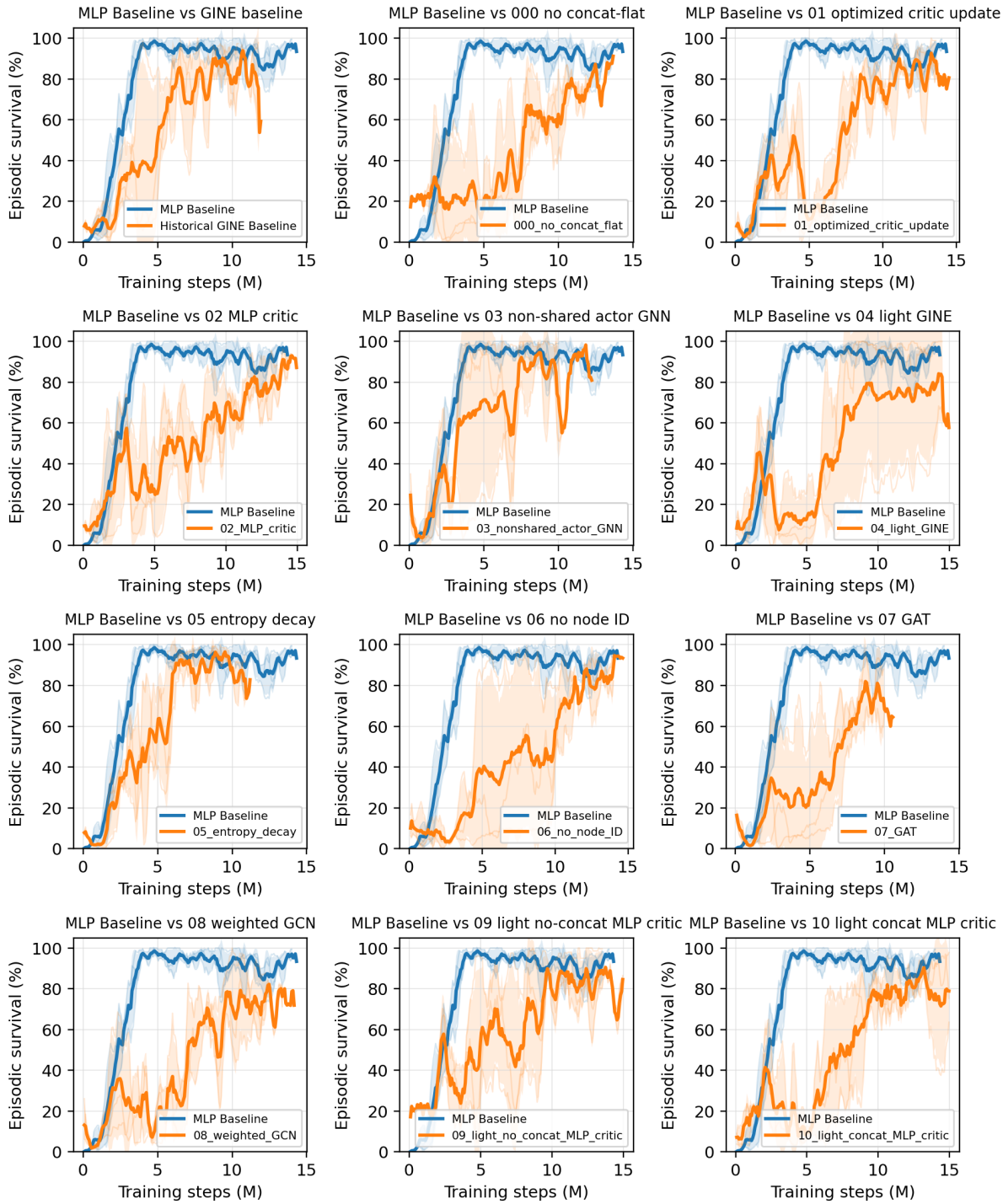

Figure 6.2: GINE survival comparisons. Each subplot compares the `MLP Baseline` with one variant from `configs/gine_s0_s1_s2`. Full hyperparameters and architectural details are given in Appendices B and D.

3. **Result.** The measured tables.
4. **Reading.** What follows, and what does not.

6.3 Shared Protocol

Table 6.2 separates the common optimization protocol from the boundary and seed settings that differ by campaign. Within a screen, every setting not named as a screened factor is fixed, so each screen is a comparison against its own baseline cell.

Table 6.2: Common optimization settings and campaign-specific boundary and seed settings.

Component	Fixed setting
Environment	`bus14`, difficulty 0, topology actions, decentralized agents, normalized observations and rewards, 80/20 train/test chronic split with split seed 0
Actor	One GNN encoder shared by all agents; LayerNorm; node pre-encoder; two 128-unit action-head layers
Information boundary	Screens A, B, E: connected context and controlled-only structural relations (corrected); Screens C, D: legacy whole-substation one-hop context
Critic	Centralized MLP with three 256-unit hidden layers
Graph preprocessing	None: neither deterministic physical scaling nor running standardization
Training budget	15,000,000 environment steps, 72 parallel environments, 576 rollout steps
MAPPO update	10 epochs, 16 minibatches, actor and critic learning rates 10^{-4} with linear annealing, $\gamma = 0.9$, $\lambda = 0.95$, clip 0.2, target KL 0.02, entropy coefficient 0.01
Exploration	No initial do-nothing prior and no action-0 logit bonus
Sparse control	Disabled: no intervention gate, no intervention penalty, no evaluation-time rho heuristic
Evaluation	Deterministic evaluation every 82,944 environment steps, 10 episodes on each split
Seeds	Screens A, B, E: 0; Screens C, D: 0, 1, 2

6.3.1 Endpoint statistics

Survival and the train/test split are as defined in Section 3.1: survival is the percentage of an episode's maximum length that the agent keeps the grid alive, reported on the test split. What differs here is how a run is reduced to one number. Each run saves the checkpoint with the highest test survival seen during training. That checkpoint is then re-evaluated deterministically over all 201 held-out test chronics.

Every run reaches the 15M-step budget, to within a negligible margin, so the screens do not report how far their runs got. Screens A and B report one seed per cell; Screens C and D aggregate three seeds per cell. Every seed-level endpoint is evaluated over the same 201 held-out chronics.

6.3.2 Compute

The campaigns were executed on JED (CPU) and IZAR (GPU). Within each factorial, the execution setup was held fixed across the compared configurations; the three-seed legacy campaigns also distribute seeds across both systems.

Evaluation is deterministic at a fixed checkpoint and seed. The corrected Screens A and B nevertheless have only one training seed per cell, so their differences are descriptive rather than estimates of between-seed effects.

What limits the campaign is simulation throughput rather than model size. A training step is dominated by stepping 72 Grid2Op environments, so the actor forward and backward passes are a small part of the wall clock: across the depth and width grid of Section 6.4 the actor varies by a factor of three in

parameters, 83.6k to 244.9k, without a comparable effect on run time. Parameter count is therefore not the cost to minimize when choosing an architecture here.

6.4 Screen A: Encoder Depth and Width

Question

This screen measures how message-passing depth K and encoder width d_h behave after the information-boundary correction. On the busbar graph, depth controls reach: one layer crosses one transmission-line edge, while two layers let each busbar combine information across paths of two physical edges. The graph-level readout then pools all valid node embeddings, so K need not equal the diameter of the local graph. Width controls how much information the node and pooled embeddings can retain.

The result applies directly only to the busbar schema; using the same pair for graphs with explicit asset or line nodes remains an assumption.

Design

A complete 3×4 grid at seed 0, 12 runs. Table 6.3 lists the screened parameters and their levels. The graph output dimension is matched to the hidden dimension, so a cell varies one width rather than two. Every other setting is inherited from the corrected-boundary reference configuration of Section 6.3.

Table 6.3: Screen A. Screened parameters and the values tested. All other settings are held at Table 6.2.

Parameter	Setting	Values tested
Message-passing layers K	`gnn_layers`	1, 2, 3
Encoder width d_h	`gnn_hidden_dim`	16, 32, 64, 128

Result

The complete endpoint and marginal tables, the training curves, and the parameter-count diagnostic are reported in Appendix A.2.

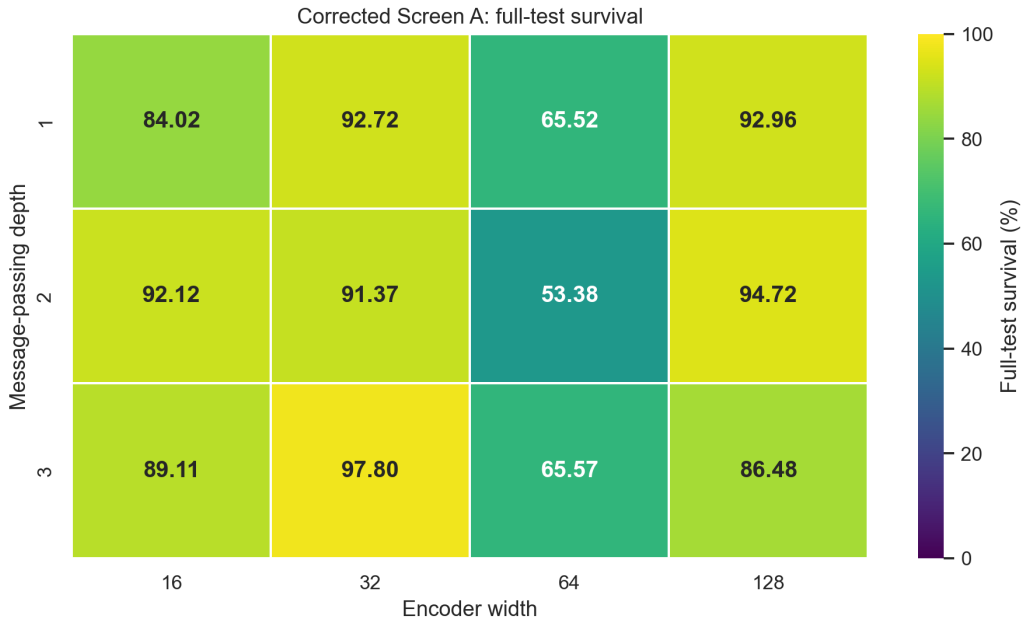

Figure 6.3: Screen A. The same twelve cells as a heatmap of the full-test survival of each run's best checkpoint. All cells use corrected graph inputs and seed 0.

Reading

The endpoint leader is `mp3_h32`. Its best checkpoint reaches 97.80% full-test survival, 3.08 points above `mp2_h128` at 94.72%.

Depth does not separate cleanly. The row means are 83.80%, 82.90%, and 84.74% for $K = 1, 2, 3$. Their ranges overlap heavily, and the best and second-worst cells both occur at $K = 3$. These four single-seed cells per row do not establish a reliable depth ordering.

Width is strongly non-monotonic. Averaged across depths, $d_h = 32$ leads at 93.97%, followed by 128 at 91.39% and 16 at 88.42%; every $d_h = 64$ cell is weak, yielding a 61.49% marginal mean.

The factorial does not yield a simple architecture rule. The depth marginals differ by less than two points, width is non-monotonic, and the ranking changes across depth–width combinations. The grid is therefore better used to select a robust operating point than to claim a monotonic depth or capacity effect.

Each endpoint covers all 201 test chronics, but it still reflects a checkpoint selected from roughly 180 ten-episode evaluations and only one training seed. Table 6.4 therefore complements it with metrics computed from the complete training curves.

Table 6.4: Screen A. Convergence measured from the training curves rather than from a single checkpoint. Steps to 90% is the first evaluation at which the smoothed test survival reaches 90%; the curve mean is the time-average of the smoothed curve over training. The final-window mean averages the raw last 20 periodic evaluations, spanning approximately the final 1.58M environment steps. This is the campaign's late-stability window: it reduces sensitivity to the last few evaluations while remaining local to the converged regime. Dashes mark architectures that never reached the threshold.

Architecture	Steps to 90% (M)	Curve mean (%)	Final-window mean (%)
`mp2_h128`	**6.22**	**71.40**	**94.04**
`mp1_h16`	6.88	70.92	84.93
`mp3_h32`	7.88	69.63	91.28
`mp2_h16`	8.79	69.16	88.21
`mp1_h128`	11.03	68.50	91.11
`mp1_h32`	6.64	63.13	93.77
`mp3_h128`	–	58.58	80.20
`mp1_h64`	–	51.60	63.37
`mp3_h64`	–	49.80	67.58
`mp3_h16`	–	48.82	60.00
`mp2_h32`	13.69	46.51	91.08
`mp2_h64`	–	45.00	53.41

The curve summaries separate a best-checkpoint endpoint from sustained late training behavior. `mp2_h128` leads all three curve criteria: it has the earliest 90% crossing, the highest time-averaged curve, and the highest 20-evaluation final-window mean. Over that same final window its standard deviation is 5.29 points, compared with 10.45 for `mp3_h32`. The latter still has the best full-test checkpoint, but its 3.08-point endpoint advantage comes from one seed and does not persist across the late trajectory.

Why carry `mp2_h128` forward. The selection combines the training evidence with an architectural prior rather than treating the noisy factor marginals as a law. In the busbar schema, $K = 2$ is the smallest depth that represents interactions across two consecutive transmission-line edges; the global readout already combines all node embeddings, while a third layer adds another round of mixing. This does *not* mean that every pair of nodes is at most two hops apart. Conditional on this deliberately chosen depth, $d_h = 128$ is the strongest width in the $K = 2$ row both at full test and over the late window, and its additional parameter cost is negligible beside Grid2Op simulation. We therefore select $(K, d_h) = (2, 128)$ as the conservative default for the subsequent screens, while retaining `mp3_h32` as the seed-0 endpoint challenger that would need a multi-seed confirmation.

6.5 Screen B: Graph Readout Scope and Reducer

Question

The corrected local graph may contain nodes that provide message-passing context without being equally useful in the final graph summary. Should readout pool every visible node, only controlled nodes, only energized nodes, or the intersection of controlled and energized nodes? For each scope, should the valid embeddings be averaged or reduced by a per-channel maximum?

The scope mask is applied only at graph readout. Context nodes remain in the message-passing graph, so this screen changes which embeddings form the policy summary without changing what information can propagate to a controlled node.

Design

A 4×2 factorial crosses four readout scopes—all visible, controlled, energized, and controlled $\cap$ energized—with mean and max reduction at seed 0. All cells use GINE with $K = 2$ and $d_h = 128$ on the corrected busbar graph. Seven cells are new; the all-visible/mean cell reuses `mp2_h128` from Screen A because it is

exactly the same configuration.

Result

Table 6.5: Screen B. Full-test survival by readout scope and reducer at seed 0. The all-visible/mean cell is inherited from Screen A.

	Reducer	
Readout scope	Mean	Max
All visible	94.72[†]	96.31
Controlled	94.60	94.79
Energized	98.93	**99.39**
Controlled ∩ energized	91.45	96.47

[†]Screen-A `mp2_h128` reference; not a new run.

Table 6.6: Screen B training-curve diagnostics at seed 0, ranked by curve mean. "To 90%" is the first crossing of 90% by the five-evaluation trailing mean; curve mean is its time-weighted mean over training. Final-20 statistics use the raw final 20 periodic evaluations (approximately 1.58M environment steps).

Scope/reducer	To 90% (M)	Curve mean (%)	Final-20 mean (%)	Final-20 std (pp)
Energized/max	**3.40**	**79.26**	97.79	4.12
Energized/mean	8.71	73.59	**98.01**	**2.90**
Controlled ∩ energized/mean	5.97	72.97	91.18	8.59
All visible/mean[†]	6.22	71.40	94.04	5.29
Controlled/mean	6.47	70.94	91.21	6.08
Controlled ∩ energized/max	8.88	68.65	86.31	8.80
All visible/max	4.73	68.38	88.57	8.08
Controlled/max	10.87	68.27	90.67	6.81

The corresponding training curves are reported in Appendix A.3.

Reading

Energized-node readout leads with both reducers. Its mean-pooling cell reaches 98.93% and its max-pooling cell reaches 99.39%, only 0.46 points apart. Relative to pooling all visible nodes, energized-only pooling improves survival by 4.21 points under mean and 3.08 points under max.

Max is directionally consistent across scopes. For every matched scope, max pooling exceeds mean pooling on the full test. The gain is 1.59 points for all visible nodes, 0.19 for controlled nodes, 0.46 for energized nodes, and 5.01 for the controlled-and-energized intersection. The magnitude therefore depends strongly on scope, but the direction does not.

Why carry energized/max forward. It combines the best scope with the reducer that wins every within-scope full-test comparison. Table 6.6 also identifies it as the strongest overall training curve: it is the earliest to cross 90% (3.40M steps) and has the highest time-weighted curve mean (79.26%). Energized/mean has the better final-20 mean and standard deviation, so the late-window evidence alone does not select max, and this remains a seed-0 screening choice rather than a general claim that max pooling must always dominate. Taken together with the 99.39% full-test endpoint, `energized_max` is the most consistently supported operating point and is fixed for the corrected reruns of Screens C and D, preventing readout from becoming an additional factor in those experiments.

6.6 Screen C: Structural Augmentations of the Busbar Graph

Question

Three optional augmentations add computational structure to the busbar graph without adding any physical measurement: same-substation edges (e), which let the busbars of one substation exchange messages directly; substation-summary nodes (n), which insert one learned hub per substation; and virtual-node readout (v), which replaces mean pooling with a learned graph-summary node. Each is motivated by the same intuition, shorten the path between related busbars, so the question is whether any of them earns its cost, and whether they compose.

Design

A complete 2^3 factorial with three seeds per cell, 24 runs. The graph type is `bus` throughout and the baseline cell is the unaugmented `e0n0v0`. It has the same nominal $K = 2$, width-128, GINE/mean design as the corrected reference, but predates the boundary fix and is not numerically comparable to Screens A and B. Cells are named by their three switches, so `e1n0v1` means same-substation edges on, summary nodes off, virtual-node readout on.

Result

The detailed result tables, training curves, and per-seed heatmap are reported in Appendix A.4.

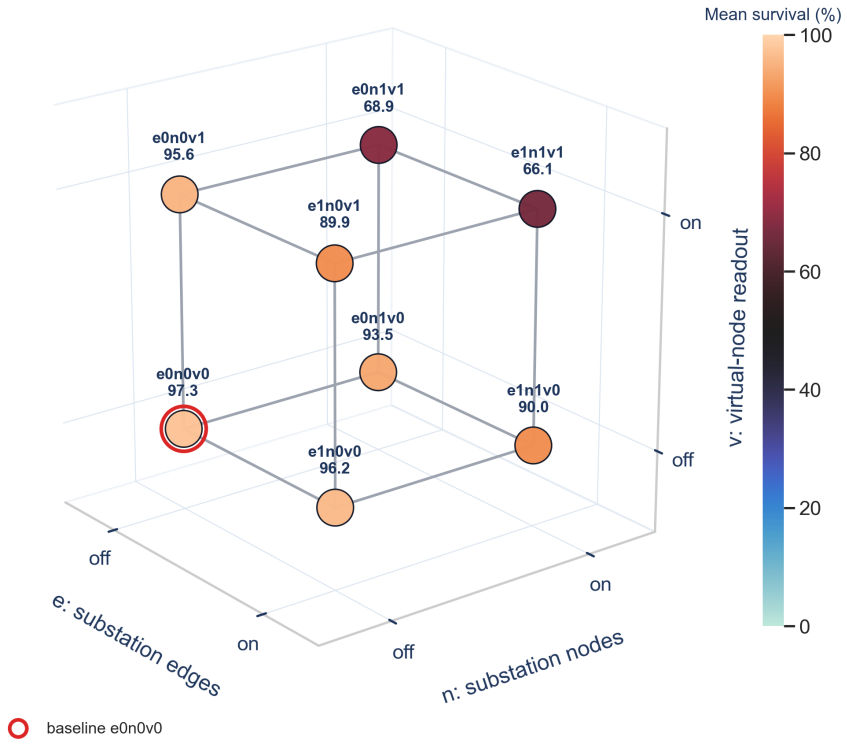

Figure 6.4: Screen C. The factorial as a cube: each vertex is one of the eight cells and each edge flips a single factor with the other two held fixed. The baseline `e0n0v0` is circled in red.

Reading

No augmentation improves the result. The unaugmented `e0n0v0` baseline is the best cell at 97.31%, and all twelve single-factor flips in Table A.10 are negative. It is also the most stable cell, with a seed standard deviation of 1.22 points against 3.9–33.9 for the augmented cells. Figure A.5 shows that the cells separate well before the training budget ends.

The hierarchy factors have the largest effects: −15.13 points for summary nodes and −14.11 for virtual-node readout. Each is relatively mild alone but severe when combined, with both $n1v1$ cells between 66% and 69%. Their four conditional flips span about 23 points, so the main effects hide a strong interaction visible on the upper face of Figure 6.4.

The likely explanation is redundancy: both mechanisms add a learned aggregation node, and together they stack two hubs between the busbars and the readout (Figure 5.8). With only two message-passing layers, most physical nodes cannot reach that readout; Screen A's $K = 3$ row directly tests this interpretation.

Same-substation edges are milder and more consistent: their mean effect is −3.28 points and the four flips range from −1.11 to −5.71. The best edge cell is only 1.11 points below the baseline, within the observed seed variability (Figure A.6); they therefore show no reliable benefit and add candidate edges without measured return.

The subsequent stages therefore keep the plain busbar graph with mean readout. Any future hierarchy should be tested with a deeper encoder.

6.7 Screen D: Generator and Load Message Direction

Question

In the disaggregated graph, each generator and load is a node attached to its busbar. Section 5.2.3 leaves the direction of that attachment open: the asset may send messages to the busbar, receive them from it, or both. The two directions are not symmetric in effect, because the mean readout of Section 5.4.1 pools asset nodes as well as busbars. Does the choice matter, and do the generator and load relations behave independently?

Design

A complete 3×3 factorial over the generator and load directions with three seeds per cell: 27 runs. The graph type is `heterogeneous` throughout; the line and summary directions stay `bidirectional` and are inert for this schema. The encoder matches the reference configuration: GINE, two layers, 128 features, mean readout, no augmentations, so the only screened factors are the two attachment directions. The baseline cell is bidirectional on both relations. Directions are abbreviated `bi` (bidirectional), `a2b` (asset to busbar), and `b2a` (busbar to asset).

Result

The training curves and complete endpoint ranking are reported in Appendix A.5.

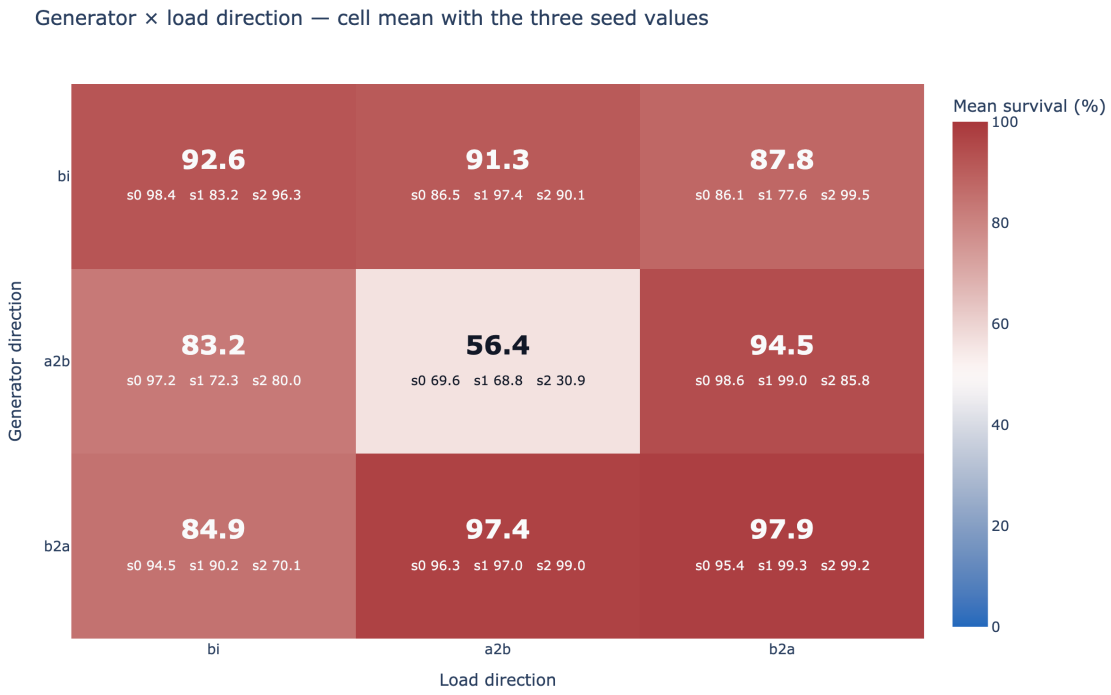

Figure 6.5: Screen D. Full-test survival of the best checkpoint by generator direction (rows) and load direction (columns), with the three individual seed values printed under each cell mean. The baseline cell is `bi/bi`.

Reading

Busbar-to-asset is the better direction for both relations. Averaging over the other relation, the generator marginal is 93.43% for `b2a`, 90.58% for `bi`, and 78.01% for `a2b`; the load marginal follows the same order at 93.39%, 86.91%, and 81.73%. The strongest part of the result is not the mean but the spread: the two best cells, `b2a/b2a` and `b2a/a2b`, have seed standard deviations of 2.20 and 1.42 percentage points against 8.24 for the bidirectional baseline, and their worst seeds (95.40% and 96.32%) are above the baseline's mean. The seed values printed in Figure 6.5 show this at a glance: the bottom row holds three tightly grouped seeds per cell, while the baseline cell spans 83.2% to 98.4%.

The failure case is equally clear. Making both relations asset-to-busbar collapses the policy to 56.41% with a 22.12-point spread and a worst seed at 30.88%, which is 36 points below the baseline and the largest single effect measured anywhere in this chapter. Its panel in Figure A.7 separates from the baseline within the first two million steps and never recovers, so this is a training failure rather than a late collapse; one of its seeds never improved on a checkpoint from 4.31M steps.

A plausible mechanism, consistent with the readout defined in Section 5.4.1 but not directly measured here, is that the mean pooling includes generator and load nodes. Under `b2a` an asset node receives busbar context and its updated embedding enters the readout, so the asset row carries information about its local neighborhood. Under `a2b` the asset node never receives a message, so after K layers its embedding is still a projection of its own raw features, and it dilutes the pooled vector with context-free rows; with both relations set that way, every generator and load row is inert.

6.8 Screen E: Candidate-Action Scoring on the Corrected Boundary

Question. The candidate-action head of Section 5.5 scores every action from the encoded nodes it modifies rather than from a fixed-width layer over an action index. Three choices in that head are unresolved: how the touched nodes are pooled, whether the static action descriptor is appended to the score, and whether the idle action gets its own head. This screen varies all three.

Design. A 2^3 factorial on `bus14`, seed 0, eight runs. Pooling is `mean` or `typed_mean`; the action descriptor is appended (`f1`) or withheld (`f0`); the idle action is scored by a dedicated head (`a0h1`) or by the shared scorer (`a0h0`). Every run uses the explicit-line graph without one-hop context, a two-layer GINE encoder of width 128, and the 15M-step budget of Section 6.3. All eight reached 15M steps.

Result. Table 6.7 reports the full-test survival of each best checkpoint over the 201 held-out chronics. The corresponding training curves and endpoint ranking are reported in Appendix A.6.

Table 6.7: Screen E, full-test survival of the best checkpoint on 201 held-out chronics. One seed per cell.

Pooling	Descriptor	Idle head	Survival (%)	Best checkpoint
`mean`	`f0`	`a0h1`	99.52	14,929,920
`typed_mean`	`f1`	`a0h0`	99.50	14,929,920
`mean`	`f0`	`a0h0`	99.18	14,929,920
`mean`	`f1`	`a0h1`	98.66	14,929,920
`mean`	`f1`	`a0h0`	98.32	14,846,976
`typed_mean`	`f0`	`a0h1`	98.17	14,764,032
`typed_mean`	`f0`	`a0h0`	92.50	12,358,656
`typed_mean`	`f1`	`a0h1`	57.84	11,612,160

Reading. **Six of the eight cells reach** 98.2% **or better** and three exceed 99%, so the candidate-action head is a viable actor on the corrected boundary. No factor separates. The apparent marginals, `mean` at 98.9% against `typed_mean` at 87.0% and `f0` at 97.3% against `f1` at 88.6%, are artifacts of two low cells; removing the single failure at 57.84% brings the pooling gap to about a point. At one seed the defensible statement is that the head works and that the three choices do not order themselves.

The descriptor is the one asymmetry worth recording: the best cell withholds it and both failing cells include it, against a prior that favours the more elaborate option. It is not redundant information, since every action still reaches the scorer with its own pooled context – agent 0's 57 actions produce 57 distinct row-sets, because an action toggles one to three of a substation's lines rather than a single busbar. Withholding it removes a static label and leaves the learned description intact.

6.8.1 Replication with Corrected Graph Inputs

At seed 0, the eight cells were repeated with corrected graph inputs (NLS): deterministic physical scaling and line-angle differences, without running standardization. The GNN remains shared, but each agent retains its own scoring MLP and optional idle head; this tests preprocessing, not scorer sharing.

Table 6.8: Full-test survival over 201 held-out `bus14` chronics with original (NL) and corrected (NLS) graph inputs. The final column is NLS minus NL in percentage points. One seed per cell.

Pooling	Descriptor	Idle head	NL (%)	NLS (%)	Δ
`mean`	`f1`	`a0h0`	98.32	**100.00**	+1.68
`typed_mean`	`f0`	`a0h0`	92.50	99.84	+7.33
`mean`	`f0`	`a0h1`	99.52	99.46	−0.07
`mean`	`f0`	`a0h0`	99.18	99.43	+0.25
`typed_mean`	`f1`	`a0h1`	57.84	90.71	+32.87
`mean`	`f1`	`a0h1`	98.66	83.81	−14.85
`typed_mean`	`f1`	`a0h0`	99.50	71.67	−27.82
`typed_mean`	`f0`	`a0h1`	98.17	69.67	−28.50

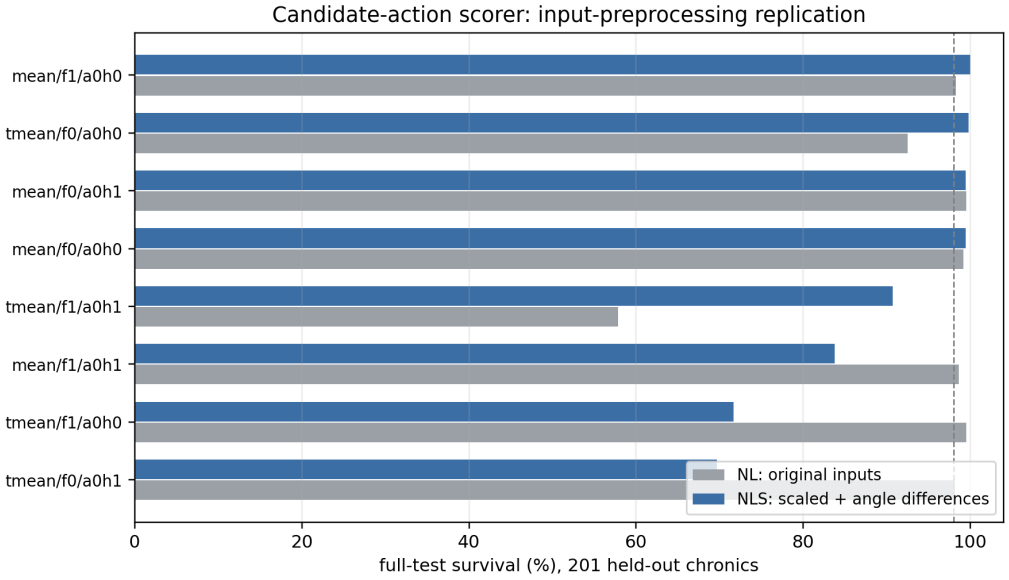

Figure 6.6: The paired comparison of Table 6.8. NLS changes both power scaling and angle representation; the differences cannot be attributed to scaling alone.

Reading. Four NLS cells exceed 98%, but three remain below 91%, with changes above 25 points in either direction. At one seed, this indicates an interaction with the candidate head rather than a stable preprocessing effect. Only `mean/f0` is robust: both idle-head choices exceed 99.1% with NL and 99.4% with NLS. Chapter 8 separately studies transfer with a scorer shared across agents.

Chapter 7

What the Shared Encoder Reads

One encoder serves every agent, and the same encoder is what a transfer study moves between grids. Its inputs are raw physical quantities, so before either form of sharing can be interpreted the inputs themselves have to be measured. This chapter does that: it establishes what the encoder consumes, how far those distributions differ between two grids, which of the differences a network absorbs and which it does not, and what corrections follow.

Chapter 8 then applies the corrections and runs the transfer. The measurements here are the reason that study is interpretable at all: without them, a deficit under transfer could not be told apart from a representation reading miscalibrated inputs.

7.1 Why the input distribution matters

The graph encoder is grid-size independent and can therefore be transferred across grids. A conventional action head cannot: it has one output per action index, so its shape changes with the agent partitions and action sets. The candidate scorer of Section 5.5 escapes that by applying shared weights and action-conditioned pooling to a variable set of actions, which makes the action head transferable too. Only the flat centralized critic stays grid-specific, its input dimension being set by the grid.

The two input paths must be distinguished. The critic receives the normalized flat observation; the transferred actor encoder receives only graph node and edge features, for which physical scaling and running standardization are disabled here. A frozen `bus14` encoder therefore reads WCCI's raw graph quantities, and whether that matters depends on how closely the two distributions match.

7.2 Measurement protocol

Both environments were rolled out under a do-nothing policy for 6,000 environment steps, with a cap of 300 steps per chronic so the sample spreads over roughly twenty scenarios per grid rather than concentrating in one. At each step the per-agent local graphs were rebuilt and every node and active physical edge recorded, pooling agents together because a single encoder is shared across them. This yields 222,000 node observations for `bus14` and 552,000 for WCCI, and 312,000 and 899,494 edge observations respectively.

Four statistics summarize each feature. Writing F_A and F_B for the two empirical distributions with means μ_A, μ_B and standard deviations σ_A, σ_B, and $\sigma_p = \sqrt{(\sigma_A^2 + \sigma_B^2)/2}$ for their pooled spread:

$$\text{offset} = \frac{\mu_B - \mu_A}{\sigma_p}, \qquad \text{scale ratio} = \frac{\sigma_B}{\sigma_A}, \tag{7.1}$$

$$\text{KS} = \sup_x |F_A(x) - F_B(x)|, \qquad W_1/\sigma_p = \frac{1}{\sigma_p} \int_0^1 |F_A^{-1}(q) - F_B^{-1}(q)| \, \mathrm{d}q. \tag{7.2}$$

Reading the symbols. A is always `bus14`, the source grid, and B always WCCI, the target: an offset is therefore positive when WCCI sits higher, and a scale ratio above one means WCCI is the wider of the two. F_A and F_B are the two *empirical cumulative distribution functions*, one per grid, built from the samples of Section 7.2: $F_A(x)$ is the fraction of `bus14`'s sampled values that fall at or below x, so it rises from 0 to 1 as x sweeps the axis. Its inverse $F_A^{-1}(q)$ runs the other way, returning the value below which a fraction q of the sample lies — $F_A^{-1}(0.5)$ is the median. The two definitions are what let the last two statistics compare the same pair of samples from perpendicular directions: KS measures the gap between the curves vertically at fixed x, W_1 between their inverses horizontally at fixed q.

Naming the last two. KS is the Kolmogorov–Smirnov statistic: the largest vertical gap between the two empirical cumulative distribution functions. It lies in $[0, 1]$, reaching 0 when the distributions coincide and 1 when their supports are disjoint, and the last panel of Figure 7.1 draws the gap it measures. W_1 is the 1-Wasserstein distance, also called the earth mover's distance: the average horizontal gap between the two quantile functions, or equivalently the least mass $\times$ distance needed to rearrange one distribution into the other. The subscript is the exponent applied to that displacement — W_2 would square it and so weigh far excursions more heavily — and this work uses $p = 1$ throughout. Because W_1 carries the units of the feature, it is reported in pooled standard deviations so that channels measured in MW and in degrees stay comparable.

The pairing is deliberate: the offset is blind to a change in spread and the scale ratio is blind to a displacement, so a channel is only unproblematic when both sit near their neutral values. KS measures disagreement in the bulk and is comparatively insensitive to tails, which W_1 does weigh. Section 7.5 shows why the two failures need separating.

7.3 What the encoder reads

The measurements are taken on the `bus` graph, whose ten channels are specified in Tables 5.4 and 5.5 and repeated with their units in Table A.12. What matters here is not their definition but how often each is zero, because four of the six node channels are structurally sparse: a busbar carries generation or load only when an asset is attached to it, and an angle is recorded only where that asset connects.

The zero fraction is a sparsity diagnostic, not an information score. 100 % means every sampled value is zero, whereas 0 % only means zero was never observed: `line_status` is constant at one because the measurement retains only active physical edges, and `rho` is nonzero on every retained edge. The overflow channel is not constant but nearly so, with 8 nonzero entries out of 312,000 on `bus14` and 6 out of 899,494 on WCCI. The two cooldown channels are exactly zero only because the sample uses a do-nothing policy; under control a topology change sets the substation cooldown to 3 and a reconnection sets the line cooldown to 3.

One feature is dimensionless by construction: `rho` divides each line's flow by that line's own thermal limit. Every other physical channel carries the units of the grid it came from.

7.4 Results

Table 7.1 reports the shift statistics twice: over all sampled entries, and over only the nodes that carry the asset in question. The two readings differ because a sparse channel is a point mass at zero mixed with a continuous part, and the aggregate averages the two together.

Table 7.1: Distribution shift between the two grids, ordered by KS. Offset is in pooled standard deviations and the scale ratio is WCCI over `bus14`. The right-hand block restricts the four sparse channels to nodes carrying the corresponding asset. *dense* marks a channel with no structural zeros to remove: every entry is a genuine measurement, so the restriction would return the same rows and the same numbers. *constant* marks a channel with no variation in this sample, whose offset and ratio are therefore undefined. Means, standard deviations and W_1 for both readings are in Table A.13.

	Zeros (%)		All sampled entries			Asset-carrying nodes		
Channel	`bus14`	WCCI	offset	ratio	KS	offset	ratio	KS
`load_theta`	45.9	48.9	+1.194	0.90	0.412	+2.105	1.65	**0.757**
`gen_theta`	81.1	76.1	+0.566	0.93	0.139	+1.756	1.49	**0.723**
`gen_p`	80.1	85.4	+0.017	2.45	0.111	+0.255	**5.84**	0.497
`load_p`	45.9	46.7	−0.078	2.42	0.055	−0.105	2.87	0.093
`rho`	0	0	−0.259	1.05	0.141	dense		
`domain_mask`	24.3	21.7	+0.061	0.96	0.026	dense		
`timestep_overflow`	> 99.99		−0.004	0.70	0.000	dense		
`line_status`	0	0	—	—	0.000	constant		
`time_before_cooldown_line`	100	100	—	—	0.000	constant		
`time_before_cooldown_sub`	100	100	—	—	0.000	constant		

Read carelessly the left block looks mild: only `load_theta` exceeds KS = 0.2 and the power channels sit near KS = 0.1. Their scale ratios, however, are 2.4 — two distributions sharing a centre while differing in width, which is precisely what KS alone cannot see.

The right block is the comparison the weights respond to, and it widens every shift sharply: `load_theta` moves from 0.412 to 0.757 in KS and from 0.90 to 1.65 in scale ratio, and `gen_p`'s spread ratio goes from 2.45 to 5.84. Restricting to asset-carrying nodes removes the structural zeros the two grids share, leaving only the physics on which they differ. This is why the mixture structure matters for any per-feature normalization, a point Section 7.6.1 returns to.

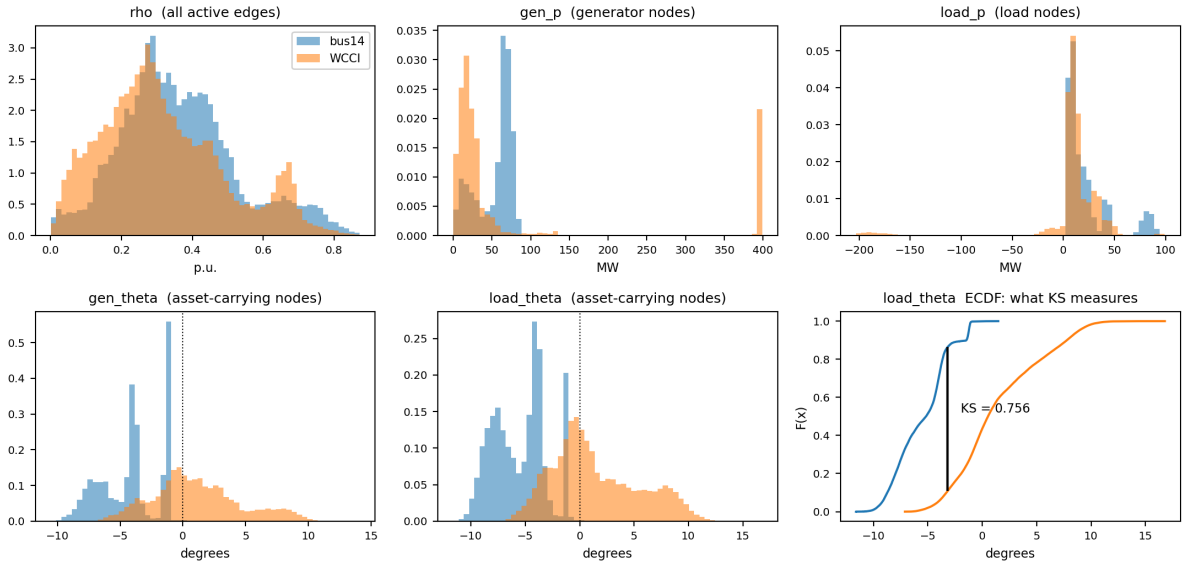

Figure 7.1: The channels the encoder reads, with the four sparse ones restricted to nodes carrying the corresponding asset. `rho` lands almost on top of itself, as expected from a quantity already normalized by each grid's own thermal limits. The power channels share a centre but not a width, and WCCI alone produces negative `load_p`. The angles are displaced rather than rescaled: `bus14` sits almost entirely on the negative side. The last panel reads that displacement as the KS statistic. Tails beyond the plotted range appear on log density in Figure A.10.

7.5 Two failure modes

The left panel of Figure 7.2 separates the channels along the two axes; the right panel repeats the diagnostic after the corrections and is read in Section 7.6.5. The distinction is not cosmetic, because the architecture responds to the two axes differently.

Reading the axes. Each point is one channel, placed by the two statistics of Equation (7.1) taken over all sampled entries. The horizontal axis is the *magnitude* of the offset, $|\mu_B - \mu_A|/\sigma_p$, in pooled standard deviations: how far the two distributions have moved apart, with the sign discarded because only the size of a displacement matters here. The vertical axis is the scale ratio σ_B/σ_A, WCCI over `bus14`: how much wider or narrower WCCI's spread is, with 1 meaning identical spread. It is drawn on a logarithmic scale so that halving and doubling sit the same distance from 1, which a linear axis would misrepresent. A channel needing no correction sits at the origin of this reading — near-zero offset and ratio near 1 — so distance from the bottom-left corner is distance from comparability, and the direction of that distance names which of the two failures it is.

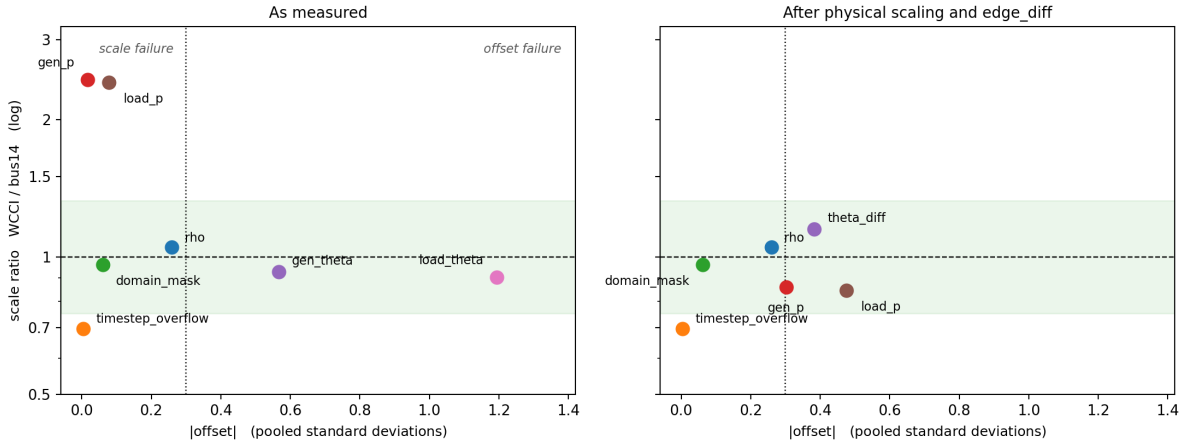

Figure 7.2: Absolute offset against scale ratio, over all sampled entries. The shaded band and the dashed line mark a scale ratio within a third of unity; the dotted vertical line marks an offset of 0.3 pooled standard deviations. *Left:* as measured. Channels outside those bounds fail in one of two distinct ways — the power channels keep their centre and widen, the angles keep their width and move. *Right:* the same diagnostic after the two corrections of Section 7.6, discussed in Section 7.6.5. Every channel now sits inside the scale band, and the worst displacement falls from 1.19 to 0.48, but the power channels acquire an offset they did not have.

The actor's node pre-encoder is a linear map followed by ReLU and LayerNorm. LayerNorm absorbs much of a uniform change of *scale*: multiplying every input channel by a constant leaves the normalized activations largely unchanged. It does not absorb an *offset*. Displacing a channel by a constant moves the pre-activations to a different region of the ReLU, changing which units fire at all, and normalization applied afterwards cannot undo that.

This makes the angles, not the powers, the binding problem. The power channels differ in width by factors of 2.9 to 5.8, which the architecture partially self-corrects, though the negative `load_p` region on WCCI reaching −281 MW is genuinely unseen territory rather than a wider version of something familiar. The angles differ by roughly two pooled standard deviations of displacement, which it does not correct at all.

`rho`, meanwhile, needs no correction at all: scale ratio 1.050, offset −0.26. The single most decision-relevant channel is already comparable across grids because it was defined in per-unit terms — an argument for expressing features in grid-relative units wherever the physics permits it.

7.6 Correcting the inputs

Two preprocessing mechanisms exist in the codebase. Deterministic physical scaling divides each channel by a per-environment physical constant. Running standardization maintains per-environment, per-node-type statistics over both power and both angle channels, excluding `rho`, which the measurements above confirm needs no help. On the diagnosis alone the second looks like the match, being the only one that can move a displaced channel. It is not.

7.6.1 Running standardization makes the mismatch worse

Standardizing each environment with its own statistics aligns the aggregate moments exactly: every channel comes out at offset 0.00 and scale ratio 1.00. On three of the four channels it touches it nonetheless moves the distributions several times further apart (Table 7.2).

Table 7.2: KS between the two grids before and after per-environment standardization. The moments align perfectly, and three of the four channels degrade by a factor of five or more. The last column is the number a busbar with *no asset attached* presents to the encoder once standardized. Raw, that busbar reads 0 on both grids. Standardizing sends it to $(0-\mu)/\sigma = -\mu/\sigma$, which is a property of the grid, so the same physical fact arrives as two different numbers — and on the angle channels with two different signs.

Channel	KS raw	KS standardized	"No asset" encoding `bus14` vs WCCI
`gen_p`	0.111	**0.805**	−0.450 vs −0.197
`gen_theta`	0.139	**0.700**	+0.409 vs −0.148
`load_p`	0.055	**0.590**	−0.599 vs −0.188
`load_theta`	0.412	0.326	+0.910 vs −0.252

The cause is the mixture structure documented in the right-hand block of Table 7.1. These channels are a point mass at zero, meaning no generator or load is attached to this busbar, plus a continuous part. Subtracting a per-grid mean moves that point mass to a grid-dependent location. Since 80–85 % of busbars carry no generator, the encoder's single most frequent input stops meaning the same thing on the two grids, which is a larger mismatch than the displacement being corrected. On the two angle channels it is starker still: an unattached busbar reads +0.409 and +0.910 on `bus14` but −0.148 and −0.252 on WCCI, so the absence of an asset lands on opposite sides of zero depending on which grid the encoder is reading.

The one channel that improves confirms the mechanism rather than contradicting it. `load_theta` alone has grids that agree on how often it is zero (45.9 against 48.9 %) and a raw scale ratio already near unity, so its point mass moves by nearly the same amount on both sides and standardization does little there but remove the offset. The three that degrade lack exactly that coincidence. Even so it is not the better fix: the representation change of Section 7.6.3 brings the same channel to 0.230 without touching the point mass at all.

Two further properties rule the mechanism out independently of these numbers. Standardization is scale-invariant, so it erases any scaling applied before it — dividing by a constant and then standardizing reproduces standardizing alone to four decimals — meaning the two mechanisms cannot be combined. And running statistics begin empty and converge during training, so a frozen encoder would spend early target training reading a distribution that is itself still moving, where a deterministic scaling is fixed at construction and identical on both grids from the first step.

7.6.2 Physical scaling for the power channels

Physical scaling, defined in Section 5.1, divides the power channels by each environment's own maximum generator capacity. Two properties answer the failure diagnosed above: being multiplicative it leaves zeros at zero, so the point mass stays aligned on both grids, and being fixed at construction it does not drift the way estimated moments do. The divisor is a single generator's nameplate rating, a crude proxy

for grid scale, so it is worth checking against the alternatives (Table 7.3); it is the closest of the four to the divisor that would equalize the two spreads exactly.

Table 7.3: Candidate power normalizers and the `gen_p` scale ratio each would produce. A ratio of 1.00 would be exact; the raw ratio is 2.45.

Divisor	`bus14`	WCCI	Resulting ratio
Exact	—	—	1.00
Maximum $P_{\max}$ (used)	140	400	**0.86**
Total $P_{\max}$	540	2,450	0.54
Generator count	6	22	0.67
Median $P_{\max}$	85	71	2.93

7.6.3 The angle displacement is a gauge artifact

Scaling cannot repair the angle displacement, and the implementation does not attempt it: angle channels are divided by a constant 180, identical on both grids, so both sides move together and the displacement is untouched. The measurement explains why. An absolute voltage angle is referenced to whichever bus is slack, and the two grids do not place theirs alike, so the 5 to 7° displacement in Figure 7.1 is largely gauge rather than physics. Nothing that rescales or recentres a reference-dependent quantity can fix that; the quantity has to be replaced.

The replacement is the angle drop along each line, defined in Section 5.1.2. Being a difference it is gauge-invariant, and being per line it is dense, so it removes the mixture structure from the angle channels outright rather than working around it. This study therefore enables it, alongside physical scaling, in both source and target environments.

7.6.4 The action descriptor must not be normalized per agent

One input reaches the actor without passing through the encoder: the candidate-action head of Section 5.5 appends a static descriptor of each action to the pooled node context. Its count columns were divided by the largest value in that agent's *own* action set — the construction Section 7.6.1 rejects for node features, applied to a different input, with the agents as the populations.

Table 7.4: Divisors applied to the action count descriptors under the previous per-agent rule, `bus14`.

Agent	Actions	Topology changes	Line endpoints
0	57	5	3
1	51	5	4
2	83	6	3

An action moving three elements therefore reached the shared scorer as 0.60 for agent 0 and 0.50 for agent 2, and every agent's busiest action reached it as exactly 1.0 whether it moved five elements or six. On another grid the divisors change again. The columns are now divided by a constant, selected by `candidate_action_feature_scaling`; the busiest actions then read 1.25, 1.25 and 1.5. This is the only correction here that applies within a grid as well as across grids, and it qualifies Screen E of Chapter 6, whose runs predate it: their `f1` cells appended a descriptor inconsistent between the agents sharing one scorer.

7.6.5 The corrected distribution

Table 7.5 measures both changes together, under the same protocol as Section 7.2, and the right panel of Figure 7.2 places the result back on the offset–scale diagnostic. The angle channel lands in the band occupied by `rho`, the power spreads come within 15 % of parity, and every channel now sits inside the scale band that only `rho` and `domain_mask` occupied before.

Table 7.5: Distribution shift with `gnn_angle_representation = edge_diff` and `gnn_physical_scaling = true`, against the uncorrected baseline, over all sampled entries. The offset column is included because scaling moves it: dividing each grid by its own constant is not offset-preserving when the two constants stand in a different relation to typical operation.

	Uncorrected			Corrected		
Channel	offset	ratio	KS	offset	ratio	KS
`load_theta`	+1.194	0.90	0.412	replaced		
`gen_theta`	+0.566	0.93	0.139	replaced		
`theta_diff`		—		−0.383	**1.15**	**0.230**
`gen_p`	+0.017	2.45	0.111	−0.302	**0.86**	0.145
`load_p`	−0.078	2.42	0.055	−0.476	**0.85**	0.307
`rho`	−0.259	1.05	0.141	−0.259	1.05	0.141

The KS increase on the two power channels is the expected cost of the scaling rather than a regression. Scaling slides the continuous part of a channel while its point mass stays pinned at zero, so the two cumulative distributions separate slightly in the bulk even as their dynamic ranges align. Dynamic range is what a frozen encoder's weights are calibrated against, so the trade is favourable, and it accounts for `load_p` reading 0.307.

The offset column records a second cost, and a less comfortable one. The raw power means were nearly identical, at +0.017 and −0.078; after division they are not. `bus14` runs its generators at 39 % of its fleet maximum against WCCI's 20 %, so a divisor taken from the largest unit lands the two grids at different normalized levels and the displacement grows to −0.302 and −0.476. Section 7.5 argued that offsets, not scale errors, are what the architecture cannot absorb, so this is the correction moving a channel onto the axis that matters rather than off it.

The exchange is still favourable on the evidence available: the worst displacement across all channels falls from 1.194 to 0.476, the scale errors of 2.4 resolve to within 15 %, and no channel remains outside both bounds at once. But it is an exchange, not an elimination, and a divisor reflecting typical rather than peak generation might do better on both axes at once. Table 7.3 tested four candidates against the scale ratio alone; selecting one against the offset as well is left open, and the transfer results of Chapter 8 are the test of whether the residual displacement matters in practice.

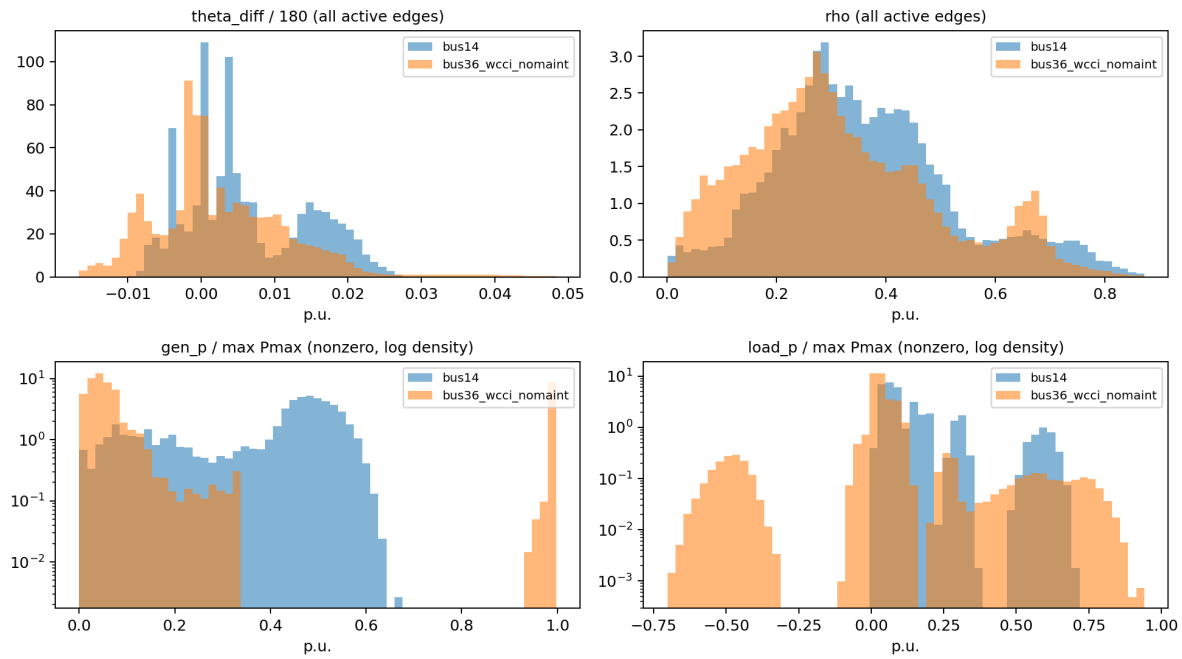

Figure 7.3: Corrected distributions, for comparison with Figure 7.1. The angle channel is now dense and centred on both grids, and the two power channels overlap over most of their range instead of differing by a factor of three in width.

Chapter 8

Transfer to the 36-Substation WCCI Grid

The controlled method comparisons so far were developed on `bus14`. That grid is small enough to iterate on but too small to claim anything about scale: fourteen substations, twenty lines, and three agents. The preliminary WCCI stress test of Section 4.7 reproduced several sparse-control mechanisms with an MLP actor on the larger grid, but even the matched baseline was weak. Because that pilot changed grid size, action space, and maintenance at once, it could not identify the source of failure.

This chapter turns that pilot into a controlled transfer study. It moves to the L2RPN WCCI 2020 grid, more than twice as many substations and three times as many lines, and asks the question that motivates a graph encoder in the first place. If the actor reads the grid as a graph rather than as a flat vector, its weights are independent of how many substations exist, and a representation learned on `bus14` should carry over.

Before interpreting that transfer, the two environments have to be made comparable and the encoder's inputs have to be checked. This chapter does both. The comparison turns out to matter more than expected: the graph features the encoder consumes are raw physical quantities, and they do not follow the same distribution on the two grids.

8.1 The target environment

Table 8.1 contrasts the source and target grids. The agent partitions follow the same principle in both cases, contiguous electrical areas, but WCCI is split into four regions rather than three, and the regions are markedly unequal: one agent controls seventeen substations while another controls five.

Table 8.1: Source and target environments. Both use topology actions, decentralized agents and an 80/20 train/test chronic split with split seed 0.

	`bus14`	`bus36_wcci_nomaint`
Grid2Op environment	`l2rpn_case14_sandbox`	`l2rpn_wcci_2020`
Substations	14	36
Lines	20	59
Agents	3	4
Chronics	1,004	2,880
Episode length	up to 8,064 steps	up to 8,062 steps
Maintenance	none	disabled (Section 8.1.1)

8.1.1 Removing maintenance

Of the environments Grid2Op distributes, `l2rpn_case14_sandbox` is the only one above the toy scale that ships without scheduled maintenance. Every larger environment, WCCI 2020 included, injects planned line outages. That is a problem for a transfer study: a policy trained without maintenance and evaluated with it faces a hazard it has never seen, and the two observation spaces differ as well, since the maintenance countdown features are only present when maintenance exists.

The two settings are therefore aligned by removing maintenance from WCCI rather than inventing a schedule for `bus14`, for which Grid2Op provides no specification.

The effect is measured on the complete held-out test split using the same 576 chronics and a do-nothing policy in both settings.

Table 8.2: Do-nothing performance on the full WCCI test split. The chronics and policy are identical; only scheduled maintenance differs.

Setting	Mean survival	Median survival	Complete episodes
Maintenance enabled	27.3%	12.1%	39/576 (6.8%)
Maintenance disabled	57.3%	100.0%	303/576 (52.6%)
Difference	+30.0 pp	+87.9 pp	+264 (+45.8 pp)

Removing maintenance increases mean survival by 30.0 percentage points and episode completion by 45.8 points, confirming that maintenance substantially changes the baseline difficulty of the target grid.

8.1.2 Action space

The unreduced WCCI action space is not trainable. Enumerating topology actions over each agent's region gives the sizes in the first column of Table 8.3: nearly 67,000 actions in total, of which agent 1 alone accounts for 65,642. All experiments therefore use the reduced action space obtained by the brute-force outcome procedure, at $k = 256$ per agent.

Table 8.3: Per-agent action-space sizes on `bus36_wcci_nomaint`. Agents 0 and 2 are already exhaustive at $k = 256$, so the reduction only binds on agents 1 and 3.

Agent	Substations	Full	Reduced ($k = 256$)
`agent_0`	5	77	77
`agent_1`	5	65,642	256
`agent_2`	9	127	127
`agent_3`	17	1,119	256
Total	36	66,965	716

The choice of k was made by evaluating a one-step simulator-greedy policy over each candidate reduction on the same fifty chronics (Table 8.4). Performance saturates at $k = 256$: the difference between $k = 256$ and $k = 1024$ is four chronics won against three lost, which is noise, while both clearly beat $k \leq 128$. Since $k = 256$ is the smallest reduction in the saturated group, it gives the smallest action heads for no measurable loss in what the action set can achieve.

Table 8.4: One-step simulator-greedy policy over each reduced action space, on the same fifty held-out chronics of `bus36_wcci_nomaint`. The do-nothing row is the common reference; no reduction was evaluated against a different chronic set.

Policy	Mean survival	Full-episode survival	Episodes worse than do-nothing
Do nothing	0.560	52%	—
Greedy, $k = 32$	0.679	60%	5
Greedy, $k = 64$	0.819	64%	3
Greedy, $k = 128$	0.847	76%	3
Greedy, $k = 256$	**0.976**	**92%**	0
Greedy, $k = 512$	0.883	82%	2
Greedy, $k = 1024$	0.953	90%	0

These two rows bracket the transfer experiments. A learned policy that lands near 0.56 mean survival has learned nothing beyond doing nothing; one approaching 0.9 is doing real work. The greedy row is not a strict upper bound, since it queries the simulator at every decision step and is myopic, but it is a strong reference.

8.2 Corrected inputs

Chapter 7 measures the encoder's inputs on both grids and settles two corrections, both applied in every run below. Power channels are divided by each grid's own maximum generator rating, which leaves the structural zeros in place and is fixed at construction rather than estimated online (Section 7.6.2). The absolute voltage angle is replaced by the drop along each line, which removes a displacement that is a property of the slack reference rather than of the grid state (Section 7.6.3). Running standardization is left off: Section 7.6.1 measures it degrading three of the four channels it touches by a factor of five or more. Together these bring the angle channel into the band occupied by `rho` and the power spreads to within 15 % of parity (Table 7.5).

8.3 Transfer experiment design

Four encoder architectures are carried over from Chapter 6, two from the message-direction screen and two from the structural screen. Each runs in two arms at three seeds. In the *frozen* arm the encoder is loaded from the paired `bus14` run and its parameters stop requiring gradients, so only the action heads train; in the *scratch* arm the same architecture starts from random initialization. Every other setting is copied from the corresponding `bus14` configuration, which is what makes the encoder weights loadable at all.

Loading needs care in one place. The encoder registers its edge index and node identifiers as persistent buffers, so they appear in the saved state dictionary at source-grid shape. They describe the source graph and are supplied per agent from the target environment's own specification at call time, so they are dropped during the load, while any other shape disagreement is treated as an architecture mismatch and raises rather than leaving part of the encoder randomly initialized.

Table 8.5: Transfer study design. The two arms of a pair differ only in the two transfer settings.

Component	Setting
Environment	`bus36_wcci_nomaint`, difficulty 0, topology actions, 80/20 split with split seed 0
Action space	Reduced at $k = 256$, giving 77 / 256 / 127 / 256 actions per agent
Architectures	`gb2a_la2b` and `gb2a_lb2a` (heterogeneous); `e0n0v0` and `e1n0v0` (bus)
Arms	Frozen encoder with trained heads; identical architecture from scratch
Seeds	0, 1, 2
Budget	15M steps, matching the `bus14` source runs
Inputs	`edge_diff` angles, physical scaling enabled

A first set of runs was launched with the uncorrected inputs before the analysis above was complete. Those are retained rather than discarded: comparing the scratch arm with and without the input correction isolates whether the correction helps or harms on the target grid, independently of any transfer effect, on otherwise identical settings.

Appendix A

Supplementary Results

This appendix collects detailed result tables and diagnostic figures that support Chapters 4 and 6. The main chapters retain the compact comparisons needed for the discussion.

A.1 Sparse Intervention Control

The joint survival/action-0 comparison is retained in Figure 4.1; the tables below provide the exact aggregate and per-agent values.

The following tables report average full-test episodic survival and action-0 fractions over the three evaluated seeds. The all-agent value is the arithmetic mean of the three per-agent fractions. The do-nothing row is a reference policy that always selects action 0.

Table A.1: Full-test summary for evaluation-time rho heuristics.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Global rho heuristic, $\rho_{safe} = 0.90$	98.36	0.936	0.933	0.936	0.940
Local rho heuristic, $\rho_{safe} = 0.90$	97.07	0.968	0.992	0.985	0.928

Table A.2: Full-test summary for intervention-gated actors.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Intervention gate, final-action MAP	89.82	0.802	0.826	0.889	0.690
Intervention gate, hierarchical greedy	93.66	0.395	0.066	0.844	0.276

Table A.3: Full-test summary for flat-policy fixed-penalty runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Flat policy, $\lambda_{fixed} = 0.000$	98.23	0.494	0.524	0.463	0.496
Flat policy, $\lambda_{fixed} = 0.001$	96.75	0.486	0.642	0.401	0.416
Flat policy, $\lambda_{fixed} = 0.003$	93.97	0.575	0.691	0.346	0.688
Flat policy, $\lambda_{fixed} = 0.010$	97.44	0.905	0.907	0.954	0.853

Table A.4: Full-test summary for gated-policy fixed-penalty runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Gated policy, $\lambda_{fixed} = 0.000$	78.51	0.787	0.747	0.949	0.665
Gated policy, $\lambda_{fixed} = 0.001$	73.82	0.862	0.956	0.832	0.799
Gated policy, $\lambda_{fixed} = 0.003$	94.59	0.887	0.934	0.929	0.799
Gated policy, $\lambda_{fixed} = 0.010$	85.69	0.974	0.990	0.984	0.947

Table A.5: Full-test summary for adaptive intervention budget runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Baseline flat policy	93.67	0.434	0.554	0.385	0.363
Flat local-safe AIB, $d = 0.20$	98.31	0.978	0.980	0.989	0.966
Flat local-safe AIB, $d = 0.10$	98.39	0.979	0.992	0.995	0.949
Flat local-safe AIB, $d = 0.35$	96.61	0.934	0.959	0.947	0.896
Gated local-safe AIB, $d = 0.20$, hierarchical greedy	33.87	0.999	1.000	1.000	0.998
Flat non-idle AIB, $d = 0.20$	95.08	0.986	0.997	0.992	0.970

Table A.6: Full-test summary for the teacher-student distillation runs.

Run type	Avg. episodic survival (%)	Avg. action-0 all agents	Avg. action-0 agent 0	Avg. action-0 agent 1	Avg. action-0 agent 2
Do-nothing policy	20.25	1.000	1.000	1.000	1.000
Teacher: MAPPO with local rho heuristic	97.07	0.968	0.992	0.985	0.928
Student BC, balanced non-idle 0.50, weight 5, aux 0.50	92.14	0.939	0.974	0.925	0.919
Student BC, balanced non-idle 0.20, weight 3, aux 0.25	90.76	0.957	0.982	0.948	0.940

A.2 Screen A: Encoder Depth and Width

Table A.7 gives the complete endpoint grid, while Table A.8 summarizes each factor across the other. Figures A.1 and A.2 provide the corresponding training-curve and parameter-count diagnostics.

Table A.7: Screen A. Full-test survival of the best checkpoint, one seed per cell, by message-passing depth and encoder width. Screen B reuses `mp2_h128` as its all-visible/mean reference.

	Encoder width d_h			
Layers K	16	32	64	128
1	84.02	92.72	65.52	92.96
2	92.12	91.37	53.38	94.72
3	89.11	**97.80**	65.57	86.48

Table A.8: Screen A. Marginal effect of each factor, averaged over the other. Four runs per depth, three per width.

Level	Mean (%)	Min (%)	Max (%)
Message-passing depth			
$K = 1$	83.80	65.52	92.96
$K = 2$	82.90	53.38	94.72
$K = 3$	84.74	65.57	97.80
Encoder width			
$d_h = 16$	88.42	84.02	92.12
$d_h = 32$	93.97	91.37	97.80
$d_h = 64$	61.49	53.38	65.57
$d_h = 128$	91.39	86.48	94.72

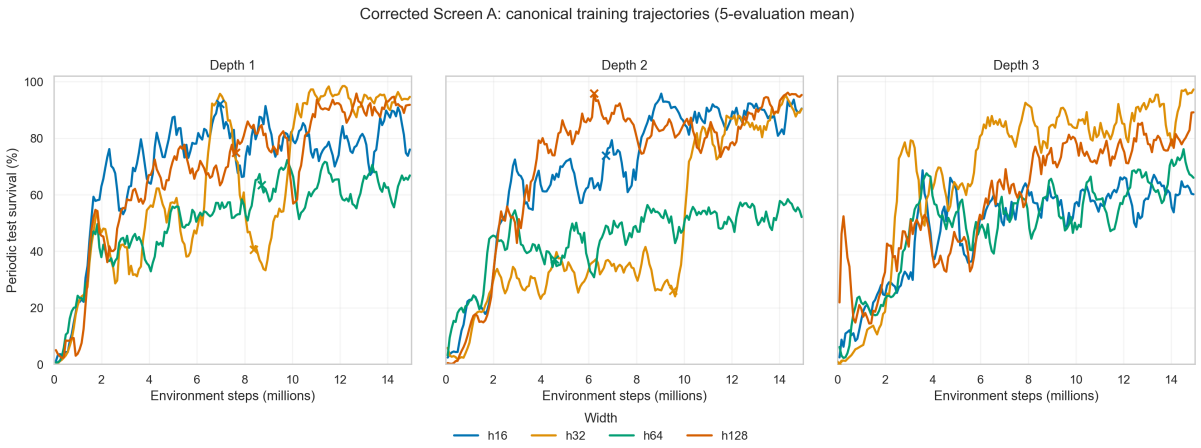

Figure A.1: Screen A. Test episodic survival during training, one panel per message-passing depth, with the four widths compared inside each panel. Restarted runs use the canonical history in which the continuation replaces the overlapping post-checkpoint branch.

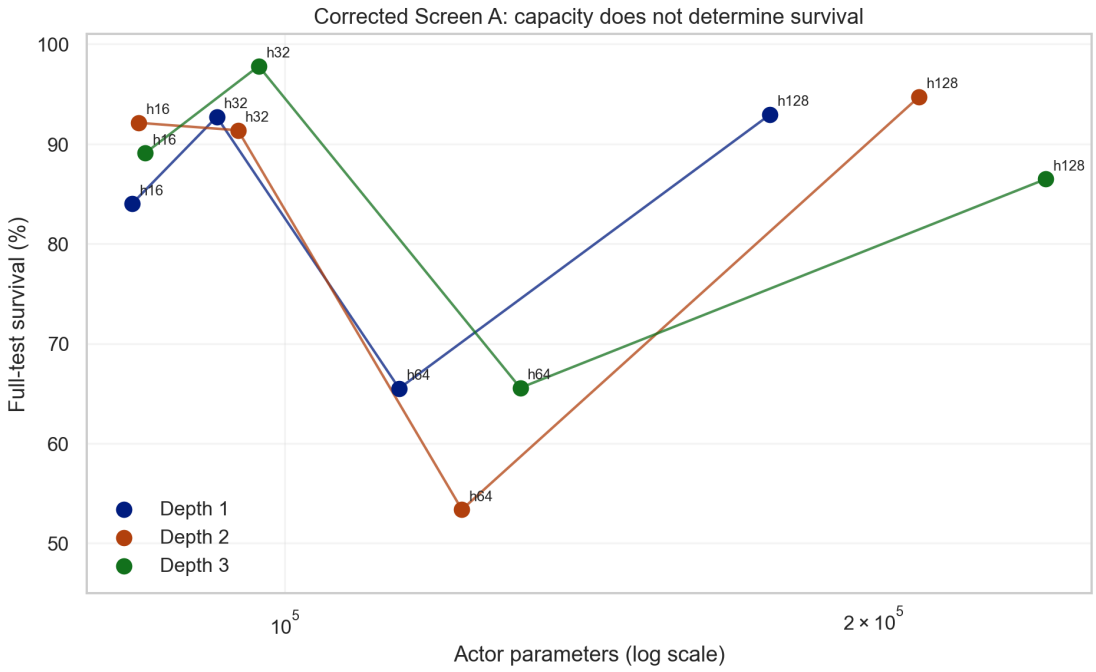

Figure A.2: Screen A. Full-test survival against actor parameter count on a logarithmic axis, joined within each depth. The grid spans 83.6k to 244.9k actor parameters, a factor of three.

A.3 Screen B: Graph Readout Scope and Reducer

Figures A.3 and A.4 complement the endpoint comparison in Table 6.5 with the eight training trajectories and the full-test interaction between readout scope and reducer.

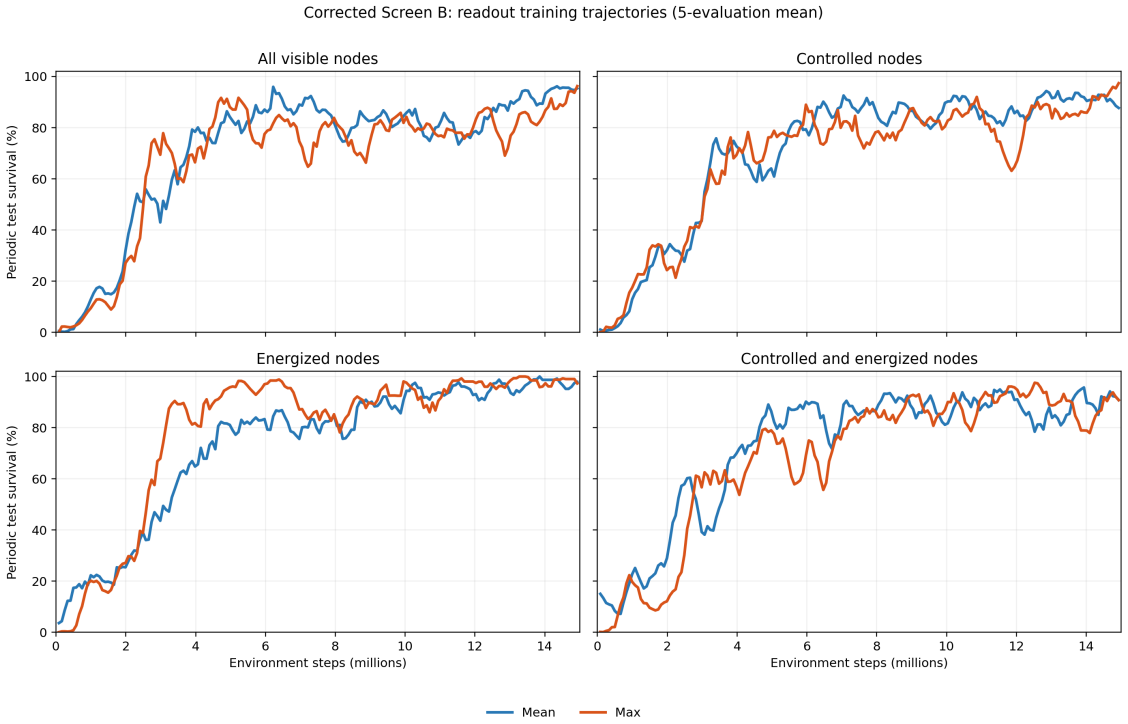

Figure A.3: Screen B. Smoothed training episodic survival curves by readout scope and reducer (seed 0).

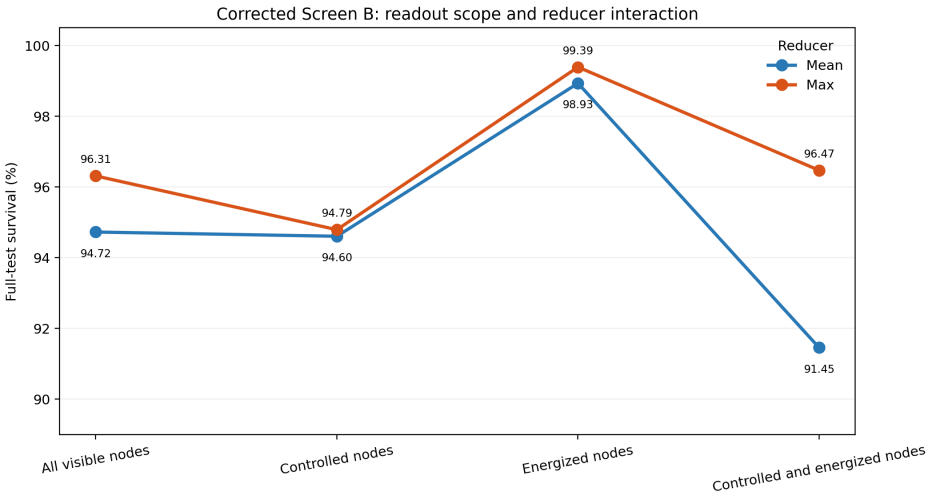

Figure A.4: Screen B. Full-test survival of each best checkpoint. Connecting the two reducers within a scope makes the scope–reducer interaction visible; energized-only readout leads under both reductions.

A.4 Screen C: Structural Augmentations of the Busbar Graph

Tables A.9 and A.10 and Figures A.5 and A.6 complement the factorial cube retained in Section 6.6 with the complete results.

Table A.9: Screen C. Cells ranked by full-test survival, with the difference from the unaugmented `e0n0v0` baseline.

Cell	Mean (%)	Std (pp)	Min (%)	Max (%)	Δ baseline (pp)
`e0n0v0`	97.31	1.22	96.55	98.72	0.00
`e1n0v0`	96.20	3.94	91.66	98.68	−1.11
`e0n0v1`	95.63	6.19	88.52	99.82	−1.68
`e0n1v0`	93.51	6.39	86.72	99.40	−3.80
`e1n1v0`	90.01	4.92	85.42	95.19	−7.30
`e1n0v1`	89.92	9.42	80.48	99.33	−7.39
`e0n1v1`	68.91	5.77	62.59	73.91	−28.40
`e1n1v1`	66.11	33.95	27.86	92.68	−31.20

Table A.10: Screen C. Main effect of each factor (12 runs per level) and the four single-factor flips of the cube, each one enabling that factor with the other two held fixed. `Spread` is the range of those four flips.

Factor	Off (%)	On (%)	Effect (pp)	Flips (pp)	Mean flip (pp)	Spread (pp)
e (substation edges)	88.84	85.56	−3.28	−1.1, −5.7, −3.5, −2.8	−3.28	4.60
v (virtual readout)	94.26	80.14	−14.11	−1.7, −24.6, −6.3, −23.9	−14.11	22.92
n (summary nodes)	94.77	79.63	−15.13	−3.8, −26.7, −6.2, −23.8	−15.13	22.92

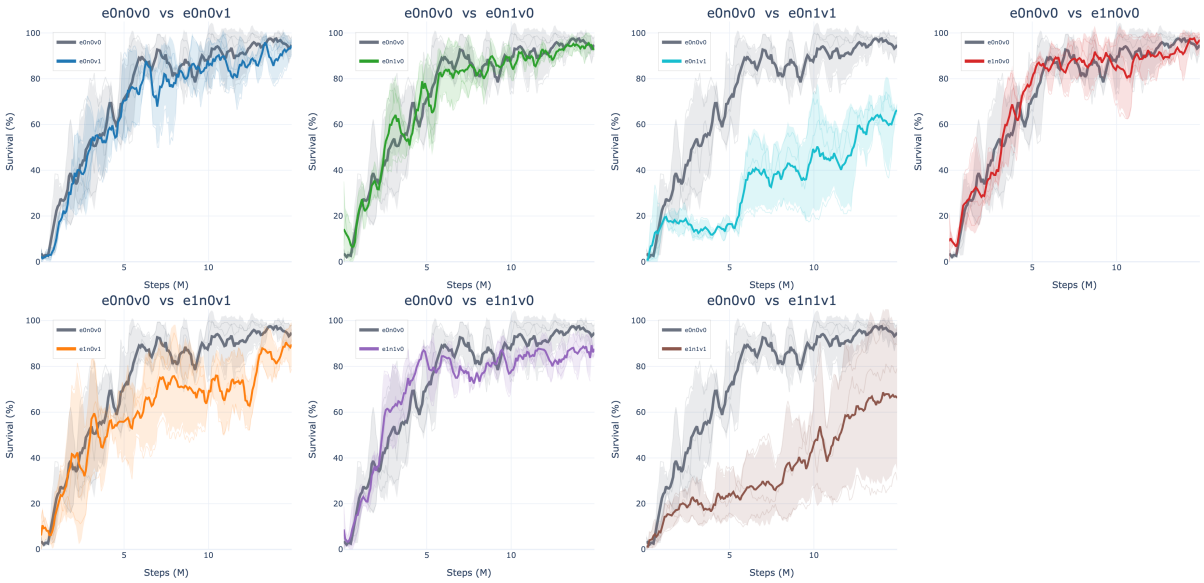

Figure A.5: Screen C. Test episodic survival during training, one panel per augmented cell against the unaugmented `e0n0v0` baseline in grey. Thin lines are the three seeds, the thick line is their mean, and the band is one standard deviation.

Figure A.6: Screen C. The same eight cells as a heatmap of the full-test survival of each run's best checkpoint, with the three individual seed values printed under each cell mean.

A.5 Screen D: Generator and Load Message Direction

Figure A.7 shows the training trajectories behind the endpoint heatmap retained in Section 6.7, while Table A.11 gives the complete ranking.

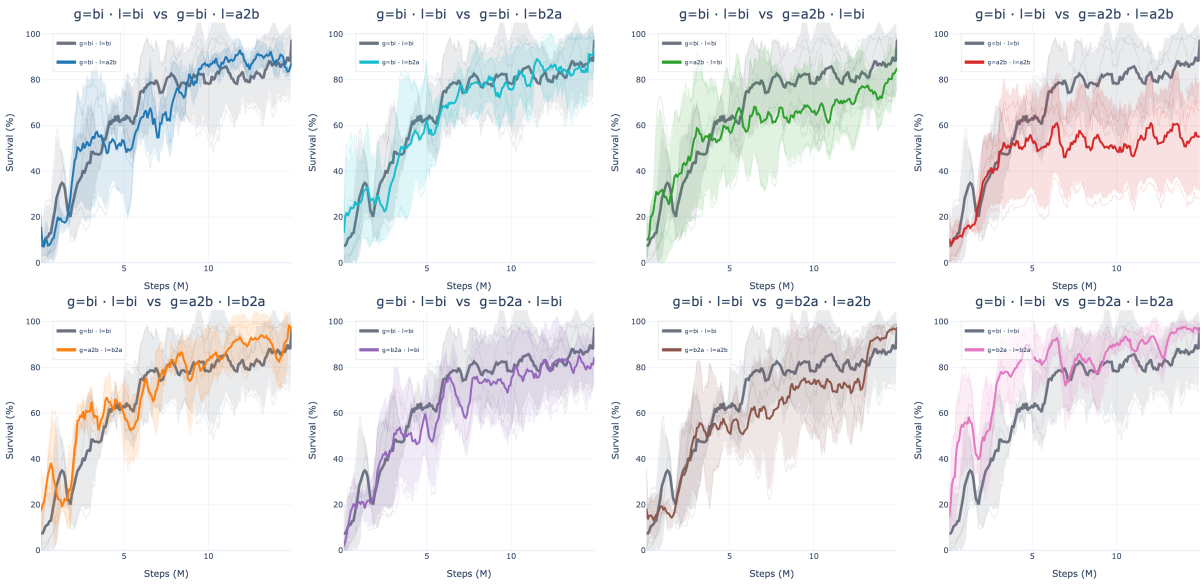

Figure A.7: Screen D. Test episodic survival during training, one panel per direction pair against the bidirectional baseline in grey. Thin lines are the three seeds, the thick line is their mean, and the band is one standard deviation.

Table A.11: Screen D. Direction pairs ranked by full-test survival, with the difference from the `bi/bi` baseline.

Generator / load	Mean (%)	Std (pp)	Min (%)	Max (%)	Δ baseline (pp)
`b2a / b2a`	97.94	2.20	95.40	99.25	+5.30
`b2a / a2b`	97.43	1.42	96.32	99.03	+4.79
`a2b / b2a`	94.47	7.53	85.78	99.03	+1.83
`bi / bi`	92.65	8.24	83.21	98.40	0.00
`bi / a2b`	91.35	5.54	86.52	97.40	−1.30
`bi / b2a`	87.76	11.04	77.64	99.53	−4.89
`b2a / bi`	84.93	13.01	70.11	94.46	−7.72
`a2b / bi`	83.16	12.75	72.27	97.19	−9.49
`a2b / a2b`	56.41	22.12	30.88	69.60	−36.24

A.6 Screen E: Candidate-Action Scoring

Figures A.8 and A.9 complement the full-test table retained in Section 6.8 with the training trajectories and endpoint ordering.

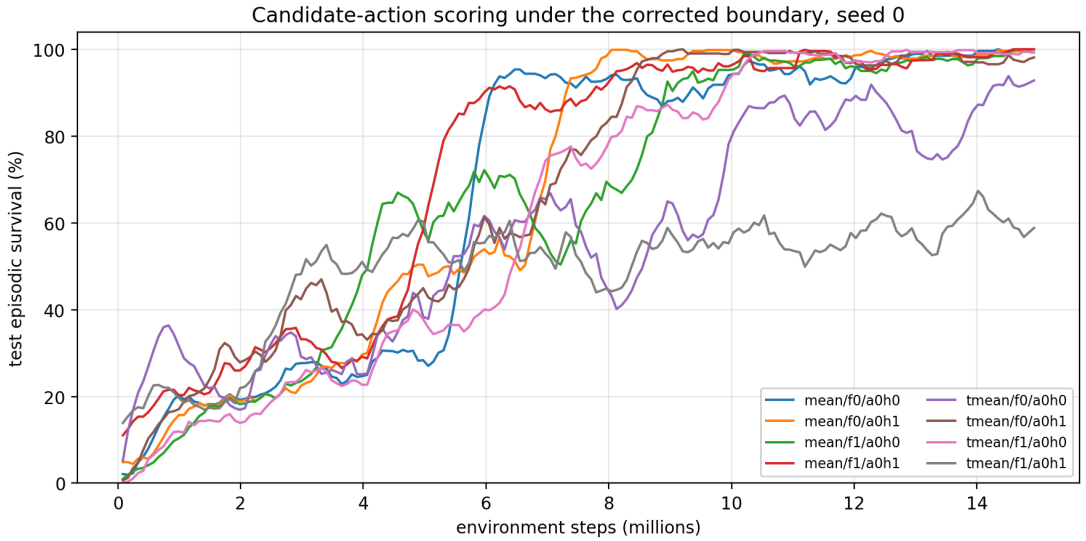

Figure A.8: Screen E. Test episodic survival during training, smoothed over nine evaluations. Six of the eight cells converge to the top of the range and stay there; `typed_mean/f1/a0h1` never does.

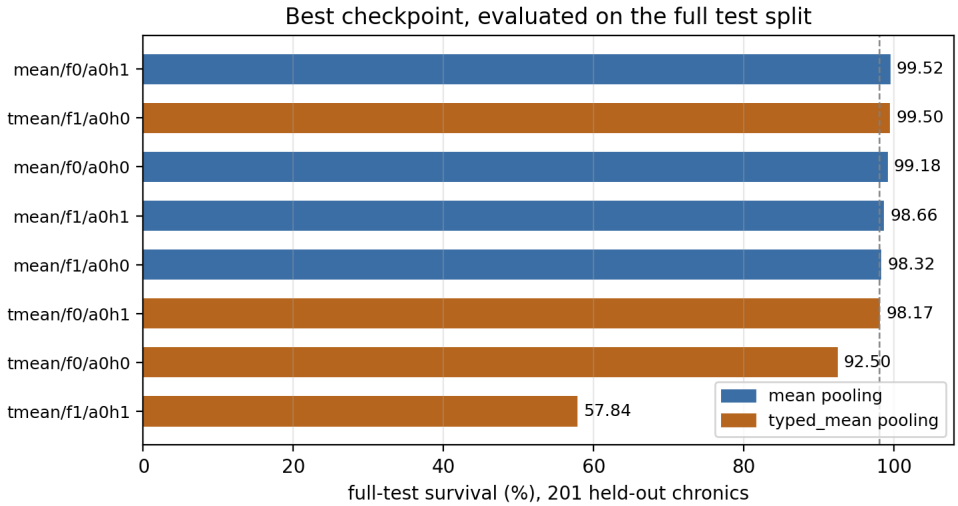

Figure A.9: Screen E. Full-test survival of each best checkpoint. The dashed line marks 98%.

A.7 Encoder Input Distributions

Chapter 7 retains the compact shift table (Table 7.1) and the consolidated distribution panel (Figure 7.1). This section provides the channel definitions, the raw moments behind the shift statistics, and the tail behaviour that linear axes compress.

Table A.12: Node and edge features of the `bus` graph with their units, and the fraction of sampled entries equal to zero on each grid. A busbar only carries generation or load when an asset is attached to it, which makes four of the six node channels structurally sparse.

Feature	Meaning	Unit	Zeros (%)	
			`bus14`	WCCI
Node features				
`gen_p`	Active power injected by attached generators	MW	80.1	85.4
`gen_theta`	Voltage angle at the generator connection	deg	81.1	76.1
`load_p`	Net active power of attached load objects (positive withdrawal)	MW	45.9	46.7
`load_theta`	Voltage angle at the load connection	deg	45.9	48.9
`time_before_cooldown_sub`	Steps until this substation may be reconfigured	steps	100	100
`domain_mask`	1 for the agent's own substations, 0 for connected context	—	24.3	21.7
Edge features				
`line_status`	1 connected, 0 disconnected	—	0	0
`rho`	Current flow divided by thermal limit	p.u.	0	0
`timestep_overflow`	Consecutive steps above the thermal limit	steps	> 99.99	> 99.99
`time_before_cooldown_line`	Steps until this line may be switched	steps	100	100

Table A.13: Raw moments behind Table 7.1, with the Wasserstein distance. W_1/σ_p tracks the offset almost exactly on the displaced channels, which is why the main table omits it: for distributions that differ mainly by a shift the two statistics carry the same information.

		`bus14`		WCCI				
Channel	Restriction	mean	sd	mean	sd	offset	ratio	W_1/σ_p
`load_theta`	all	-2.987	3.283	0.747	2.963	$+1.194$	0.903	1.194
	carrying	-5.527	2.431	1.462	4.018	$+2.105$	1.653	2.104
`gen_theta`	all	-0.787	1.925	0.265	1.786	$+0.566$	0.928	0.566
	carrying	-4.158	2.361	1.108	3.522	$+1.756$	1.492	1.756
`gen_p`	all	10.763	23.911	11.530	58.620	$+0.017$	2.452	0.300
	carrying	54.038	23.069	78.738	134.810	$+0.255$	5.844	0.746
`load_p`	all	12.104	20.192	9.184	48.766	-0.078	2.415	0.270
	carrying	22.393	22.888	17.243	65.774	-0.105	2.874	0.379
`rho`	all	0.364	0.168	0.320	0.176	-0.259	1.050	0.260
`domain_mask`	all	0.757	0.429	0.783	0.413	$+0.061$	0.961	0.062

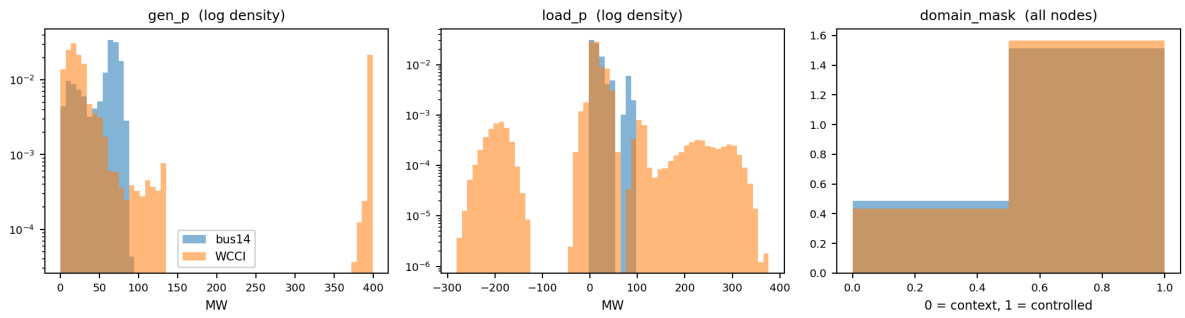

Figure A.10: Tail behaviour of the power channels on log density, and the structural `domain_mask`. WCCI carries a generator pinned near its 399 MW rating and a load region reaching −281 MW, neither of which occurs anywhere on `bus14`: a frozen encoder has no calibration for either. `domain_mask` is near-identical on the two grids, so the small difference in Table 7.1 carries no signal.

Appendix B

GINE and GAT Graph-Actor Architectures

This appendix specifies the complete edge-aware actor used in the graph experiments. GINE and GAT receive the same graph inputs and use the same pooling and action head. Their only architectural difference is the message-passing operator: GINE inserts an edge embedding into the message content, whereas GAT uses an edge embedding to decide how strongly the message is weighted.

B.1 Shared input and encoder pipeline

For agent a, the encoder receives the local graph $\mathcal{G}_t^{(a)}$. Candidate edges with `edge_mask`$= 0$ are removed before message passing, so the layer sees only the active directed set $\mathcal{E}_t^{(a)}$. The source of $u \to v$ sends a message and v aggregates it. Node and edge masks do not become learned features.

Each raw node vector $\mathbf{x}_v$ first passes through

$$\mathbf{h}_v^{(0)} = \mathrm{LayerNorm}(\mathrm{ReLU}(\mathbf{W}_x \mathbf{x}_v + \mathbf{b}_x)), \tag{B.1}$$

which produces a 128-dimensional node state. Learned global node-identifier embeddings and the optional external edge pre-encoder are disabled. Every GINE or GAT layer nevertheless contains its own trainable projection of the edge vector to the dimension required by that layer.

Two message-passing layers are used. After each layer, the implementation applies ReLU followed by LayerNorm:

$$\mathbf{h}_v^{(k+1)} = \mathrm{LayerNorm}\Big(\mathrm{ReLU}(\widetilde{\mathbf{h}}_v^{(k+1)})\Big). \tag{B.2}$$

The same encoder parameters are shared by all agents. Each agent nevertheless passes its own local graph, with its own node set, active edges, and one-hop context, through that shared encoder.

B.2 GINE message passing

For an active edge $u \to v$, layer k projects the complete edge feature vector and adds it to the sender state:

$$\widetilde{\mathbf{e}}_{uv}^{(k)} = \mathbf{W}_e^{(k)} \mathbf{e}_{uv} + \mathbf{b}_e^{(k)}, \tag{B.3}$$

$$\mathbf{m}_{u\to v}^{(k)} = \mathrm{ReLU}\Big(\mathbf{h}_u^{(k)} + \widetilde{\mathbf{e}}_{uv}^{(k)}\Big), \tag{B.4}$$

$$\widetilde{\mathbf{h}}_v^{(k+1)} = \mathrm{MLP}^{(k)}\left((1+\epsilon)\mathbf{h}_v^{(k)} + \sum_{u:(u,v)\in\mathcal{E}_t} \mathbf{m}_{u\to v}^{(k)}\right). \tag{B.5}$$

The implementation uses the PyTorch Geometric default $\epsilon = 0$ without a trainable epsilon parameter. Each internal GINE MLP is Linear–ReLU–Linear and outputs 128 features. Equation B.5 shows why

edge attributes are part of the message content. Two identical node states can send different messages when one edge is an overloaded physical line and the other is an asset-attachment relation. Likewise, reversing a typed heterogeneous edge activates a different relation-one-hot column and can produce a different message.

GINE uses an unweighted sum over incoming active edges. It does not learn a softmax competition between neighbors. The edge embedding changes what is sent; the number of incoming messages and their vector sum determine the aggregate.

B.3 GAT message passing

GAT first computes a score for every active incoming edge and attention head. For head r, the implementation has the form

$$s_{uv}^{(k,r)} = \mathrm{LeakyReLU}\Big(\mathbf{a}_{s,r}^{\top}\mathbf{W}_{s,r}\mathbf{h}_u^{(k)} + \mathbf{a}_{d,r}^{\top}\mathbf{W}_{d,r}\mathbf{h}_v^{(k)} + \mathbf{a}_{e,r}^{\top}\mathbf{W}_{e,r}\mathbf{e}_{uv}\Big), \tag{B.6}$$

$$\alpha_{uv}^{(k,r)} = \frac{\exp s_{uv}^{(k,r)}}{\sum_{w:(w,v)\in\mathcal{E}_t} \exp s_{wv}^{(k,r)}}, \tag{B.7}$$

$$\widetilde{\mathbf{h}}_v^{(k+1)} = \frac{1}{H}\sum_{r=1}^{H}\sum_{u:(u,v)\in\mathcal{E}_t} \alpha_{uv}^{(k,r)}\mathbf{W}_r\mathbf{h}_u^{(k)}. \tag{B.8}$$

GAT uses $H = 4$ heads and `concat=false`, so the four head outputs are averaged and the layer still returns 128 features. The edge embedding affects α_{uv}: it changes how much attention the receiver assigns to the sender. In the standard GAT operator used here, the edge vector is not added to the value vector $\mathbf{W}_r\mathbf{h}_u$. This is the central difference from GINE.

GAT internally adds a self-loop for every node. That computational self-loop is separate from any explicit self edge in the graph schema, which these representations do not create.

Table B.1: Exact architectural difference between the implemented GINE and GAT actors.

Component	GINE	GAT
Edge use	Added to the sender state inside every message	Enters the attention score for every incoming edge
Neighbor aggregation	Unweighted sum	Softmax-weighted sum
Self information	Explicit $(1+\epsilon)\mathbf{h}_v$ term	Internal GAT self-loop
Multiple heads	Not used	Four heads, averaged rather than concatenated
Post-layer operation	ReLU, then LayerNorm	ReLU, then LayerNorm
Pooling and action head	Shared implementation	Shared implementation

B.4 Pooling and graph output

With mean pooling, the graph representation after the second message-passing layer is

$$\overline{\mathbf{h}}_G = \frac{\sum_v m_v\mathbf{h}_v^{(K)}}{\max(1, \sum_v m_v)}, \tag{B.9}$$

where m_v is the dynamic visibility mask. This is one mean over all visible node types, not a separate mean per type. Therefore:

- the busbar graph pools controlled busbars and only the far-end busbars currently attached by live boundary lines;
- the direct heterogeneous graph additionally pools controlled generators and loads, but never neighboring private assets;

- the explicit-line graph additionally pools transmission-line nodes and contains no neighboring equipment.

An inactive contextual candidate contributes neither to the numerator nor to the denominator. The ordinary GINE/GAT mean readout includes visible context; `controlled_mean` replaces m_v by the product of the visibility and controlled-node masks. Visible context can still influence controlled nodes through message passing, but it is not aggregated directly. Sum and maximum pooling, including their `controlled_*` variants, change only the readout and do not change message passing or the information boundary.

The pooled 128-dimensional vector then passes through the shared graph output projection

$$\mathbf{z}_a = \mathrm{ReLU}(\mathbf{W}_o \overline{\mathbf{h}}_G + \mathbf{b}_o), \qquad \mathbf{z}_a \in \mathbb{R}^{128}. \tag{B.10}$$

When virtual-node readout is selected, the final virtual-node state replaces $\overline{\mathbf{h}}_G$; the output projection and its dimension stay the same.

B.5 Agent-specific action head

The flat observation is not concatenated to $\mathbf{z}_a$. Each agent has its own two-layer MLP action head:

$$\mathbf{q}_a^{(1)} = \mathrm{ReLU}(\mathbf{W}_a^{(1)} \mathbf{z}_a + \mathbf{b}_a^{(1)}), \tag{B.11}$$

$$\mathbf{q}_a^{(2)} = \mathrm{ReLU}(\mathbf{W}_a^{(2)} \mathbf{q}_a^{(1)} + \mathbf{b}_a^{(2)}), \tag{B.12}$$

$$\boldsymbol{\ell}_a = \mathbf{W}_a^{(3)} \mathbf{q}_a^{(2)} + \mathbf{b}_a^{(3)}, \qquad \boldsymbol{\ell}_a \in \mathbb{R}^{|\mathcal{A}_a|}, \tag{B.13}$$

$$\pi_a(\cdot \mid \mathcal{G}_t^{(a)}) = \mathrm{Categorical}(\mathrm{logits} = \boldsymbol{\ell}_a). \tag{B.14}$$

Both hidden layers contain 128 units. During training, an action is sampled from this categorical distribution. Deterministic evaluation selects $\arg\max_j \ell_{a,j}$. Action zero is the local do-nothing action.

Only the GNN encoder and graph output projection are shared across agents. The action heads are agent-specific because agents can have different local action sets and therefore different output widths $|\mathcal{A}_a|$. The critic is a separate MLP over the centralized flat state and does not reuse the actor GINE/GAT embedding.

Equation B.14 is the default head. The optional candidate-action head of Section 5.5 keeps this pipeline up to $\mathbf{z}_a$ and additionally consumes the post-message-passing node states $\mathbf{h}_v^{(K)}$, which are the same states that Equation B.9 averages.

B.6 Exact node inputs by representation

All node types of one representation receive the same feature columns. Columns not listed for a node type are zero. Type identity reaches GINE and GAT through the `is_*` columns. The inventory below describes visible nodes. For an inactive contextual candidate, the complete row—including type and role columns—is zero, all incident edges are inactive, and the node is excluded from readout.

Table B.2: Node types and nonzero input features received by GINE and GAT, under the default `gnn_angle_representation = node`. Under `edge_diff` the `gen_theta`, `load_theta` and `theta` entries leave the node rows in every representation; the busbar and disaggregated graphs move the angle drop onto the physical line edge, while the explicit-line graph writes it on the transmission-line row as `theta_diff` (Section 5.1.2).

Representation	Node type	Populated input features
`bus`	Busbar	`gen_p`, `gen_theta`, `load_p`, `load_theta`, `time_before_cooldown_sub`, `domain_mask`
`heterogeneous`	Busbar	`is_busbar`, `time_before_cooldown_sub`, `domain_mask`
	Generator	`is_generator`, `p`=`gen_p`, `theta`=`gen_theta`, `domain_mask`
	Load	`is_load`, `p`=`load_p`, `theta`=`load_theta`, `domain_mask`
`heterogeneous_line`	Busbar	`is_busbar`, `time_before_cooldown_sub`, `domain_mask`
	Generator	`is_generator`, `p`=`gen_p`, `theta`=`gen_theta`, `domain_mask`
	Load	`is_load`, `p`=`load_p`, `theta`=`load_theta`, `domain_mask`
	Transmission line	`is_transmission_line`, `line_status`, `rho`, `timestep_overflow`, `time_before_cooldown_line`, optional maintenance times, `domain_mask`
Any augmented schema	Substation summary	Learned projection of the substation structural descriptor; no raw physical feature row
	Virtual node	One learned vector shared across graphs; no raw physical feature row

B.7 Exact edge inputs by representation

The edge vector contains all edge information consumed by GINE and GAT. Inactive candidates are filtered out before the vectors in Table B.3 reach a convolution.

Table B.3: Directed edge relations and edge-feature vectors received by GINE and GAT, under the default `gnn_angle_representation = node`. "Relation" denotes one active relation-one-hot column. Under `edge_diff` the physical-line block of the busbar and disaggregated graphs gains a `theta_diff` column; the explicit-line rows are unchanged, because there the drop is a line-node feature and the attachment edges stay free of physical channels.

Representation	Directed relation	Edge features
`bus`	Physical busbar ↔ busbar	`line_status`, `rho`, `timestep_overflow`, `time_before_cooldown_line`, optional maintenance times; no relation one-hot
	Optional self or same-substation	Same width; all physical values zero; no relation one-hot
`heterogeneous`	Physical busbar ↔ busbar	Complete measured physical-line block plus `relation_physical_line`
	Generator ↔ busbar	Neutral physical block plus direction-specific generator relation
	Load ↔ busbar	Neutral physical block plus direction-specific load relation
	Optional self or same-substation	Neutral physical block plus the corresponding relation
`heterogeneous_line`	Busbar ↔ line node	One origin/extremity-and-direction relation; no line-state attributes
	Generator ↔ busbar	One direction-specific generator relation
	Load ↔ busbar	One direction-specific load relation
	Optional self or same-substation	One corresponding relation
Any augmented schema	Busbar → summary/virtual, with optional reverse	Zero edge vector; the relation carries no flow

The complete relation names and activation rules are given in Tables 5.7 and 5.9. Those tables distinguish, for example, generator-to-busbar from busbar-to-generator and origin from extremity line attachments. These distinctions matter directly to both edge-aware models: GINE changes message content and GAT changes attention.

Appendix C

Graph Construction Details

This appendix collects the construction and sizing details of the three graph schemas of Section 5.2: how large each candidate graph is, how inactive candidates are neutralized, and how a local actor graph is cut out of the full grid. None of it changes the representations themselves.

C.1 Candidate graph sizes

Let S be the number of included substations, B the busbars per substation, L the included physical lines, and N_g, N_d the generators and loads. Write $d_g, d_d, d_\ell \in \{1, 2\}$ for the number of retained directions of the generator, load, and line-node attachments: 2 for `bidirectional` and 1 for either one-way mode. Let P_ℓ be the number of represented line endpoints: an internal line contributes two and a boundary line in a local explicit-line graph contributes only its controlled endpoint. Thus $L \le P_\ell \le 2L$. Table C.1 gives the size of the fixed candidate graph before any mask is applied.

Table C.1: Size of the candidate graph of each schema. Enabling self edges adds $|\mathcal{V}|$ directed edges and enabling same-substation edges adds $SB(B-1)$, in every schema.

Schema	Candidate nodes	Candidate directed edges
`bus`	SB	$2B^2L$
`heterogeneous`	$SB + N_g + N_d$	$2B^2L + B(d_gN_g + d_dN_d)$
`heterogeneous_line`	$SB + N_g + N_d + L$	$Bd_\ell P_\ell + B(d_gN_g + d_dN_d)$

Three consequences are worth naming. A busbar graph with both optional relations enabled therefore has $2B^2L + SB + SB(B-1)$ candidate directed edges. The disaggregated schema adds $N_g + N_d$ nodes and between $B(N_g + N_d)$ and $2B(N_g + N_d)$ asset candidates to the busbar graph, and buys individual asset states with them. For a complete state graph, $P_\ell = 2L$, so the explicit-line schema replaces the $2B^2L$ busbar-to-busbar term by $2Bd_\ell L$: identical at $B = 2$ and cheaper for $B > 2$. A local boundary line has candidates only at its controlled endpoint, which reduces P_ℓ and prevents construction of a far-end attachment. The schema adds L line nodes and one message-passing hop for internal endpoint exchange.

The hierarchical augmentations of Section 5.3 add on top of any of these: one substation-summary node per substation, with SB directed edges for `toward_summary` or $2SB$ for `bidirectional`; and one virtual node, with N_{bus} or $2N_{\text{bus}}$ edges, or S or $2S$ when summary nodes are also enabled.

C.2 Indexing and masking conventions

Grid2Op numbers busbars from one in its topology vector; those identifiers are converted to the zero-based index b used in Equation 5.4.

An inactive candidate edge keeps its row in the edge-feature tensor, so the tensor shape is fixed for an episode, but the row is zero-filled and its `edge_mask` entry is 0. Masked edges are removed before the

encoder is called, so those zero rows never generate a message. This is what allows a topology change to alter connectivity without rebuilding the graph. An inactive contextual node likewise retains its candidate row, but its complete feature vector is zero and its `node_mask` entry is 0. Every incident physical or structural edge is deactivated. The row therefore participates in neither message passing nor graph readout; a mean readout excludes it from both numerator and denominator.

C.3 Local actor graphs

A local actor graph always contains the controlled substations and every line touching them. Additional membership is schema-specific. The `bus` graph contains only the far-end busbar currently attached by each live boundary line, and that busbar exposes aggregated neighboring power and angle. The direct `heterogeneous` graph contains the same far-end busbar, but never its generator or load nodes. The `heterogeneous_line` graph represents the shared line itself and therefore constructs no neighboring busbar or private asset and no far-end attachment edge. The centralized state graph is intentionally unrestricted for centralized training. Ordinary pooling aggregates all visible nodes, whereas `controlled_*` pooling restricts readout to action-domain nodes; neither choice changes the construction boundary.

C.4 Boundary verification

The regression suite checks that contextual visibility follows a live busbar attachment, moves when the attachment is switched, and disappears after a line outage; controlled nodes remain present; structural relations cannot make a hidden candidate visible; inactive rows are zero; neighboring private assets are absent from direct heterogeneous graphs; and all neighboring equipment is absent from explicit-line local graphs. Candidate-action tests additionally verify that a boundary line remains addressable through its line node without adding far-end equipment. Controlled-readout tests verify both that the pooled set is fixed and that gradients can still reach visible context through message passing.

Appendix D

Experiment Configurations

This appendix provides the complete run configurations for the GINE encoder and heavy GINE ablations.

Table D.1: GINE rerun configuration summary. The reference run is `Heavy GINE Baseline`.

Run	Actor/critic encoder	GNN hidden/layers	Actor/critic heads	Concat flat	Edge pre-enc.	Entropy	Init action-0	Opt. critic	Difference from `Heavy GINE Baseline`
`Heavy GINE Baseline`	`gnn / gnn`	128 / 2	3×256 / 3×256	true	true	0.02	0.7	false	Baseline: heavy concat-flat GINE actor, GNN critic, and legacy critic updates
`Light GINE (Concat, MLP Critic)`	`gnn / mlp`	64 / 1	2×128 / 2×128	true	false	0.02	0.7	true	Light GINE actor, MLP critic, smaller heads, no edge pre-encoder, optimized critic updates
`Heavy GINE (No Bias)`	`gnn / gnn`	128 / 2	3×256 / 3×256	true	true	0.02	0.0	false	Bias ablation: same as baseline, but the initial do-nothing prior is 0.0
`No Bias, Opt Critic`	`gnn / gnn`	128 / 2	3×256 / 3×256	true	true	0.02	0.0	true	Bias 0.0 plus optimized critic updates; otherwise heavy concat-flat GINE with GNN critic
`Optimized Heavy GINE`	`gnn / gnn`	128 / 2	3×256 / 3×256	true	true	0.01	0.0	true	Optimized heavy GINE: bias 0.0, lower entropy 0.01, optimized critic updates
`Optimized Light GINE`	`gnn / mlp`	64 / 1	2×128 / 2×128	false	false	0.01	0.0	true	Light optimized GINE: MLP critic, no concat-flat, lower entropy 0.01, bias 0.0, optimized critic updates

Table D.2: Heavy-GINE parameter comparison from `configs/gine_s0_s1_s2`.

Run	Main change from baseline	GNN	GNN size	Critic	Actor/critic heads	Concat	Shared	Edge pre.	Node ID	Entropy	Init action-0	Opt. critic
`No Flat Concat`	Removes flat-observation concatenation	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	no	yes	yes	yes	0.02–0.02	0.7	no
`Heavy GINE Baseline`	Reference: heavy concat-flat GINE with GNN critic	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Opt. Critic Updates`	Enables optimized critic updates	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	yes
`MLP Critic`	Replaces the GNN critic with an MLP critic	`gine`	2 × 128	`mlp`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Unshared Actor`	Uses non-shared actor GNN weights	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	no	yes	yes	0.02–0.02	0.7	no
`Light GINE Encoder`	Uses a lighter GINE encoder	`gine`	1 × 64	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Zero Entropy Decay`	Decays entropy from 0.02 to 0.0	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.00	0.7	no
`No Node-ID Embs`	Removes learned node-ID embeddings	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	no	0.02–0.02	0.7	no
`GAT Message Passing`	Replaces GINE with GAT message passing	`gat`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Weighted GCN`	Replaces GINE with weighted GCN message passing	`gcn`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.7	no
`Light GINE (No Concat, MLP Critic)`	Light no-concat GINE with MLP critic and optimized critic updates	`gine`	1 × 64	`mlp`	2 × 128 / 2 × 128	no	yes	no	yes	0.02–0.02	0.7	yes
`Light GINE (Concat, MLP Critic)`	Light concat-flat GINE with MLP critic and optimized critic updates	`gine`	1 × 64	`mlp`	2 × 128 / 2 × 128	yes	yes	no	yes	0.02–0.02	0.7	yes
`No Initial Bias`	Removes the initial do-nothing bias	`gine`	2 × 128	`gnn`	3 × 256 / 3 × 256	yes	yes	yes	yes	0.02–0.02	0.0	no

Appendix E

Full-Test Checkpoint Registry

Table E.1 reports the checkpoint step used for each full-test result table. The values are the `checkpoint_global_step` fields stored in the corresponding JSON files under `Topology_Task/outputs/full_test_eval`. The do-nothing policy has no learned checkpoint. For the behavioral cloning students, the stored step is the student checkpoint's optimizer-step count rather than a MAPPO environment step.

Table E.1: Checkpoint steps used for the full-test evaluations.

Run type	Seed 0 checkpoint step	Seed 1 checkpoint step	Seed 2 checkpoint step
Do-nothing policy	–	–	–
Baseline flat policy	13,600,000	13,040,000	13,760,000
Global rho heuristic, $\rho_{safe} = 0.90$	13,920,000	14,320,000	14,800,000
Local rho heuristic, $\rho_{safe} = 0.90$	14,160,000	14,400,000	14,160,000
Intervention gate, final-action MAP	12,000,000	12,720,000	13,760,000
Intervention gate, hierarchical greedy	7,120,000	14,720,000	13,760,000
Flat Sparse16, $\lambda_{fixed} = 0.000$	9,600,000	12,160,000	7,920,000
Flat Sparse16, $\lambda_{fixed} = 0.001$	10,640,000	11,520,000	11,600,000
Flat Sparse16, $\lambda_{fixed} = 0.003$	12,000,000	12,960,000	12,480,000
Flat Sparse16, $\lambda_{fixed} = 0.010$	8,960,000	5,760,000	12,560,000
Gated Sparse16, $\lambda_{fixed} = 0.000$	8,480,000	4,320,000	6,000,000
Gated Sparse16, $\lambda_{fixed} = 0.001$	5,760,000	7,600,000	3,280,000
Gated Sparse16, $\lambda_{fixed} = 0.003$	8,960,000	7,120,000	7,600,000
Gated Sparse16, $\lambda_{fixed} = 0.010$	12,000,000	7,840,000	7,920,000
Flat local-safe AIB, $d = 0.20$	13,120,000	9,360,000	12,160,000
Flat local-safe AIB, $d = 0.10$	12,880,000	13,200,000	12,720,000
Flat local-safe AIB, $d = 0.35$	12,800,000	5,360,000	11,520,000
Gated local-safe AIB, $d = 0.20$, hierarchical greedy	14,960,000	5,280,000	16,800,000
Flat non-idle AIB, $d = 0.20$	12,480,000	14,880,000	11,920,000
Teacher: MAPPO with local rho heuristic	14,160,000	14,400,000	14,160,000
Student BC, balanced non-idle 0.50, weight 5, aux 0.50	12,375	14,994	13,617
Student BC, balanced non-idle 0.20, weight 3, aux 0.25	12,375	14,994	13,617

Bibliography

[1] E. B. Fisher, R. P. O'Neill, and M. C. Ferris, "Optimal transmission switching," *IEEE Transactions on Power Systems*, vol. 23, no. 3, pp. 1346–1355, 2008.

[2] A. Marot et al., "Learning to run a power network challenge for training topology controllers," *Electric Power Systems Research*, 2021.

[3] J. Viebahn, M. Naglic, A. Marot, B. Donnot, and S. H. Tindemans, "Potential and challenges of AI-powered decision support for short-term system operations," in *CIGRE Paris Session*, 2022.

[4] B. Donnot, "Grid2op: A testbed platform to model sequential decision making in power systems," *arXiv preprint arXiv:2011.07929*, 2020.

[5] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in *International Conference on Machine Learning*, 1993.

[6] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems*, 2017.

[7] C. Yu et al., "The surprising effectiveness of ppo in cooperative multi-agent games," *Advances in Neural Information Processing Systems*, 2022.

[8] M. Hassouna, C. Holzhüter, P. Lytaev, J. Thomas, B. Sick, and C. Scholz, *Graph reinforcement learning for power grids: A comprehensive survey*, 2024. arXiv: 2407.04522. [Online]. Available: https://arxiv.org/abs/2407.04522

[9] M. Subramanian, J. Viebahn, S. H. Tindemans, B. Donnot, and A. Marot, "Exploring grid topology reconfiguration using a simple deep reinforcement learning approach," in *IEEE Madrid PowerTech*, 2021.

[10] D. Yoon, S. Hong, B.-J. Lee, and K.-E. Kim, "Winning the L2RPN challenge: Power grid management via semi-Markov afterstate actor-critic," in *International Conference on Learning Representations*, 2021.

[11] A. Chauhan, M. Baranwal, and A. Basumatary, "PowRL: A reinforcement learning framework for robust management of power networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, 2023, pp. 14 757–14 764.

[12] M. Lehna, J. Viebahn, A. Marot, S. Tomforde, and C. Scholz, "Managing power grids through topology actions: A comparative study between advanced rule-based and reinforcement learning agents," *Energy and AI*, vol. 14, p. 100 276, 2023.

[13] B. Manczak, J. Viebahn, and H. van Hoof, *Hierarchical reinforcement learning for power network topology control*, 2023. arXiv: 2311.02129. [Online]. Available: https://arxiv.org/abs/2311.02129

[14] E. van der Sar, A. Zocca, and S. Bhulai, *Multi-agent reinforcement learning for power grid topology optimization*, 2023. arXiv: 2310.02605. [Online]. Available: https://arxiv.org/abs/2310.02605

[15] B. de Mol, D. Barbieri, J. Viebahn, and D. Grossi, "Centrally coordinated multi-agent reinforcement learning for power grid topology control," in *Proceedings of the 16th ACM International Conference on Future and Sustainable Energy Systems (e-Energy)*, 2025.

[16] E. Marchesini et al., "MARL2Grid-TR: A multi-agent RL benchmark in power grid operations," in *International Conference on Learning Representations*, 2026.

[17] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *International Conference on Learning Representations*, 2017.

[18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations*, 2018.

[19] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" *International Conference on Learning Representations*, 2019.

[20] M. de Jong, J. Viebahn, and Y. Shapovalova, *Generalizable graph neural networks for robust power grid topology control*, 2025. arXiv: 2501.07186. [Online]. Available: https://arxiv.org/abs/2501.07186

[21] E. Marchesini et al., *RL2Grid: Benchmarking reinforcement learning in power grid operations*, 2025. arXiv: 2503.23101. [Online]. Available: https://arxiv.org/abs/2503.23101

[22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[23] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," *International Conference on Machine Learning*, 2017. [Online]. Available: https://arxiv.org/abs/1705.10528

[24] A. Stooke, J. Achiam, and P. Abbeel, "Responsive safety in reinforcement learning by PID lagrangian methods," *International Conference on Machine Learning*, 2020. [Online]. Available: https://arxiv.org/abs/2007.03964

[25] N. Carrara, E. Leurent, R. Laroche, T. Urvoy, and O.-A. Maillard, "Budgeted reinforcement learning in continuous state space," *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: https://arxiv.org/abs/1903.01004

[26] L. Lautenbacher et al., *Multi-objective reinforcement learning for power grid topology control*, 2025. arXiv: 2502.00040. [Online]. Available: https://arxiv.org/abs/2502.00040

[27] W. Hu et al., "Strategies for pre-training graph neural networks," in *International Conference on Learning Representations*, 2020.